

Graph Signal Processing



OUTLINE

- ▶ Signal processing
- ▶ Graph theory
- ▶ Graph signal processing
- ▶ Focus :
Graph convolutional networks



OUTLINE

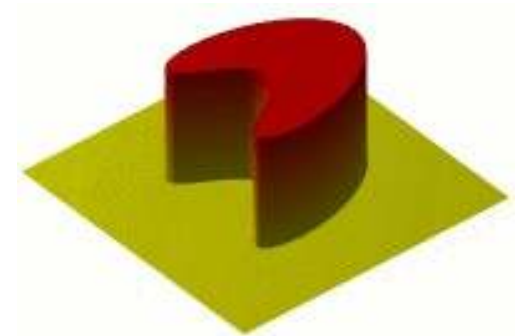
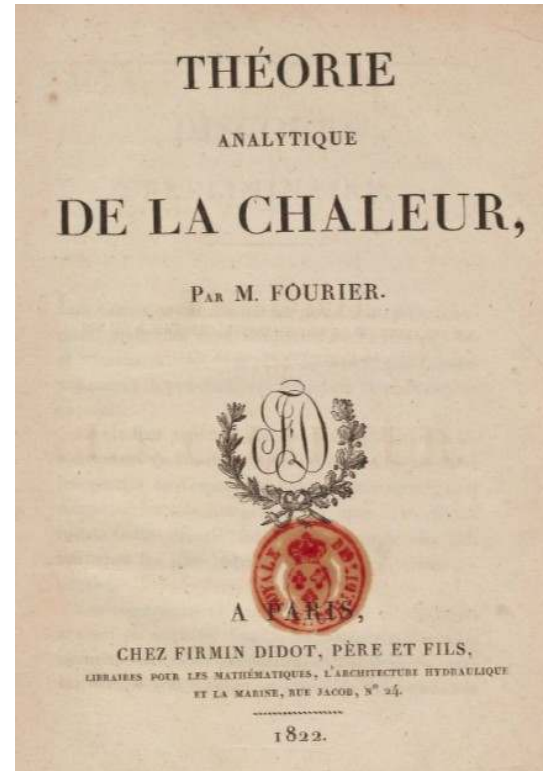
- ▶ Signal processing
- ▶ Graph theory
- ▶ Graph signal processing
- ▶ Focus :
Graph convolutional networks



The work of Fourier

└ A long, long time ago

https://fr.wikipedia.org/wiki/Joseph_Fourier#/media/Fichier:Fourier2.jpg
https://en.wikipedia.org/wiki/Heat_equation#/media/File:Heat_eqn.gif
<https://gallica.bnf.fr/ark:/12148/bpt6k1045508v>



The work of Fourier

└ Fourier decomposition

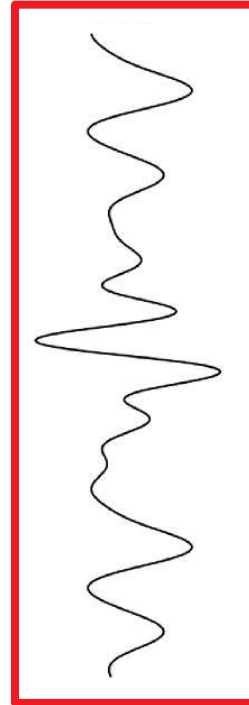
<https://agilescientific.com/blog/2012/10/23/hooray-for-fourier.htm>

https://images-na.ssl-images-amazon.com/images/I/61O%2BA3Wc5XL._AC_SL1001_.jpg

https://www.notretemps.com/cache/com_zoo_images/21/apple-watch-ecg-france_62c1ccf0952604677cc2cb95bec7ea13.jpg

<https://www.cardmarket.com/fr/Magic/Products/Singles/Unlimited/Black-Lotus>

<https://tel.archives-ouvertes.fr/tel-01708824>



The work of Fourier

└ Fourier decomposition

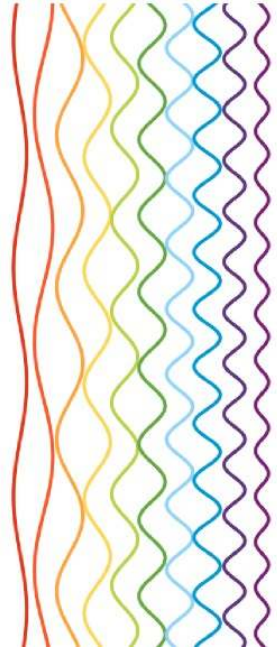
<https://agilescientific.com/blog/2012/10/23/hooray-for-fourier.htm>

https://images-na.ssl-images-amazon.com/images/I/61O%2BA3Wc5XL._AC_SL1001_.jpg

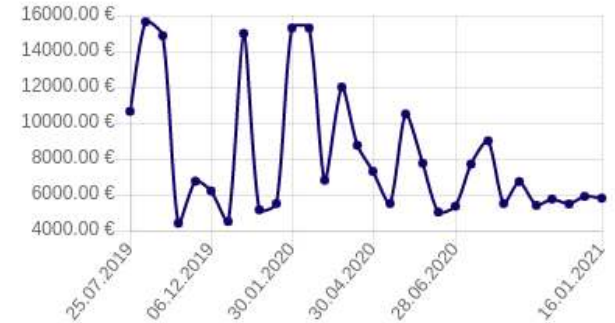
https://www.notretemps.com/cache/com_zoo_images/21/apple-watch-ecg-france_62c1ccf0952604677cc2cb95bec7ea13.jpg

<https://www.cardmarket.com/fr/Magic/Products/Singles/Unlimited/Black-Lotus>

<https://tel.archives-ouvertes.fr/tel-01708824>



=



The work of Fourier

└ Fourier decomposition

<https://agilescientific.com/blog/2012/10/23/hooray-for-fourier.htm>

https://images-na.ssl-images-amazon.com/images/I/61O%2BA3Wc5XL._AC_SL1001_.jpg

https://www.notretemps.com/cache/com_zoo_images/21/apple-watch-ecg-france_62c1ccf0952604677cc2cb95bec7ea13.jpg

<https://www.cardmarket.com/fr/Magic/Products/Singles/Unlimited/Black-Lotus>

<https://tel.archives-ouvertes.fr/tel-01708824>

Definition 33: Fourier transform

The *Fourier transform* $\hat{s} = \text{FT}(s)$ of a continuous, integrable, signal $s(t)$ is:

$$\text{FT}(s) : \begin{cases} \mathbb{R} \rightarrow \mathbb{C} \\ \omega \mapsto \hat{s}(\omega) = \int_{\mathbb{R}} s(t) e^{-i2\pi\omega t} dt \end{cases} \quad (23)$$

with t in seconds, and ω the frequency in Hz.

The work of Fourier

└ A change of basis

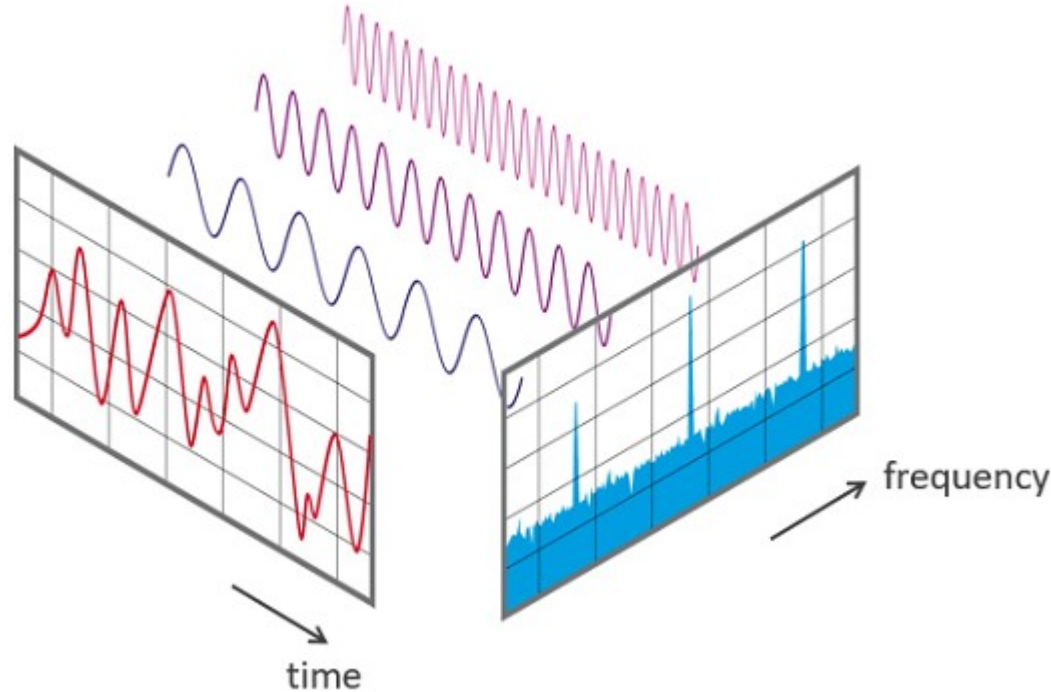
Definition 33: Fourier transform

The **Fourier transform** $\hat{s} = \text{FT}(s)$ of a continuous, integrable, signal $s(t)$ is:

$$\text{FT}(s) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ \omega & \mapsto \hat{s}(\omega) = \int_{\mathbb{R}} s(t) e^{-i2\pi\omega t} dt \end{cases}, \quad (23)$$

with t in seconds, and ω the frequency in Hz.

[s://dev.to/trekhleb/playing-with-discrete-fourier-transform-algorithm-in-javascript-53n5](https://dev.to/trekhleb/playing-with-discrete-fourier-transform-algorithm-in-javascript-53n5)
<https://tel.archives-ouvertes.fr/tel-01708824>



The work of Fourier

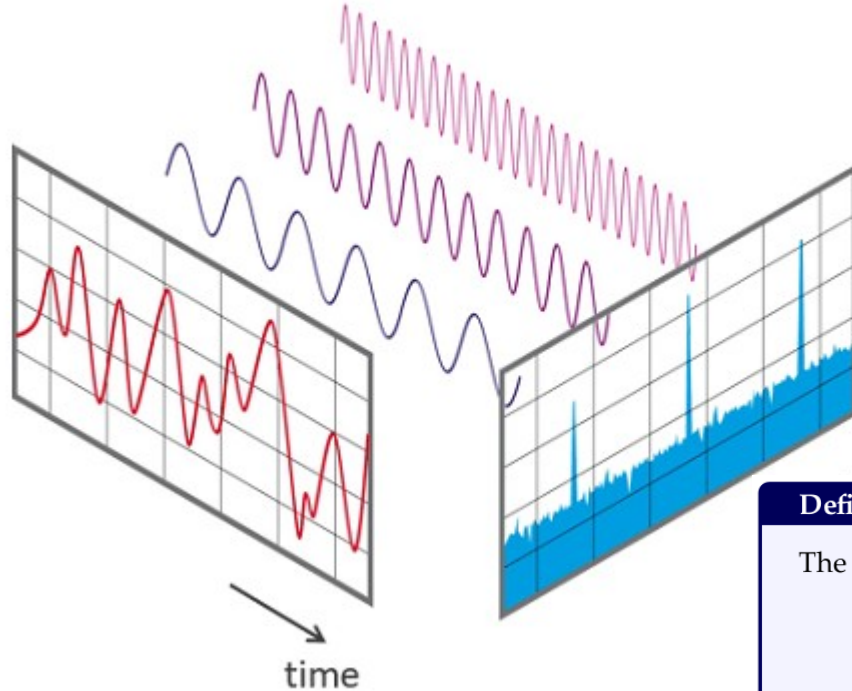
A change of basis

Definition 33: Fourier transform

The *Fourier transform* $\hat{s} = \text{FT}(s)$ of a continuous, integrable, signal $s(t)$ is:

$$\text{FT}(s) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ \omega & \mapsto \hat{s}(\omega) = \int_{\mathbb{R}} s(t) e^{-i2\pi\omega t} dt \end{cases} \quad (23)$$

with t in seconds, and ω the frequency in Hz.



<https://dev.to/trekhleb/playing-with-discrete-fourier-transform-algorithm-in-javascript-53n5>
<https://tel.archives-ouvertes.fr/tel-01708824>

Time domain

Fourier
transform
↓

Inv. Fourier
transform
↑

Frequency domain

Definition 34: Inverse Fourier transform

The *inverse Fourier transform* $s = \text{FT}^{-1}(\hat{s})$ of an integrable spectrum $\hat{s}(\omega)$ is:

$$\text{FT}^{-1}(\hat{s}) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ t & \mapsto s(t) = \int_{\mathbb{R}} \hat{s}(\omega) e^{+i2\pi\omega t} d\omega \end{cases} \quad (24)$$

The work of Fourier

└ A change of basis

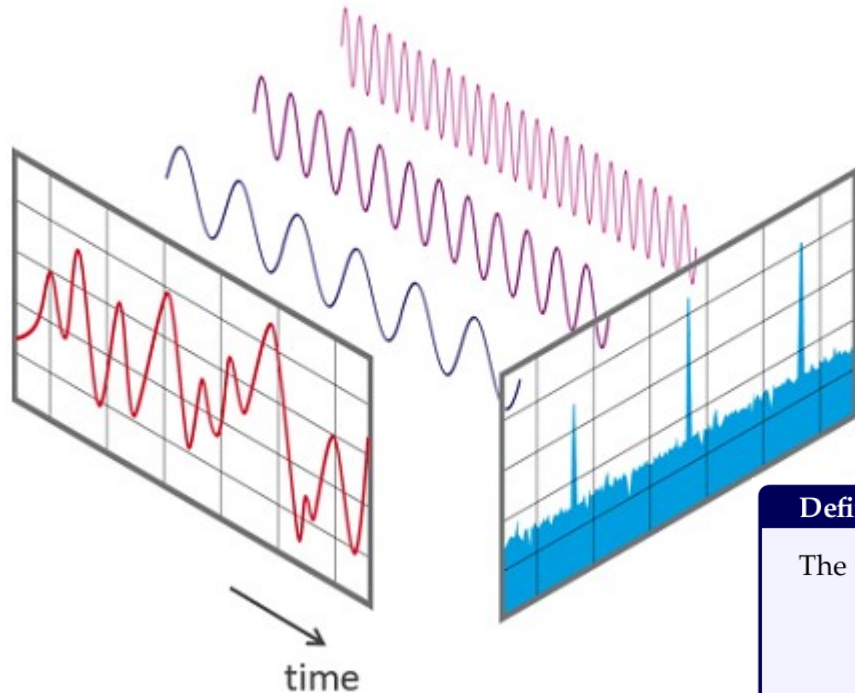
IMPORTANT!!!

Definition 33: Fourier transform

The **Fourier transform** $\hat{s} = \text{FT}(s)$ of a continuous, integrable, signal $s(t)$ is:

$$\text{FT}(s) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ \omega & \mapsto \hat{s}(\omega) = \int_{\mathbb{R}} s(t) e^{-i2\pi\omega t} dt \end{cases} \quad (23)$$

with t in seconds, and ω the frequency in Hz.



Time domain

Fourier
transform
↓

Inv. Fourier
transform
↑

Frequency domain

Definition 34: Inverse Fourier transform

The **inverse Fourier transform** $s = \text{FT}^{-1}(\hat{s})$ of an integrable spectrum $\hat{s}(\omega)$ is:

$$\text{FT}^{-1}(\hat{s}) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ t & \mapsto s(t) = \int_{\mathbb{R}} \hat{s}(\omega) e^{+i2\pi\omega t} d\omega \end{cases} \quad (24)$$

The work of Fourier

└ An innovation that led to many practical applications

https://en.wikipedia.org/wiki/Fourier_analysis

Applications [[edit](#)]

Fourier analysis has many scientific applications – in physics, partial differential equations, number theory, combinatorics, signal processing, digital image processing, probability theory, statistics, forensics, option pricing, cryptography, numerical analysis, acoustics, oceanography, sonar, optics, diffraction, geometry, protein structure analysis, and other areas.



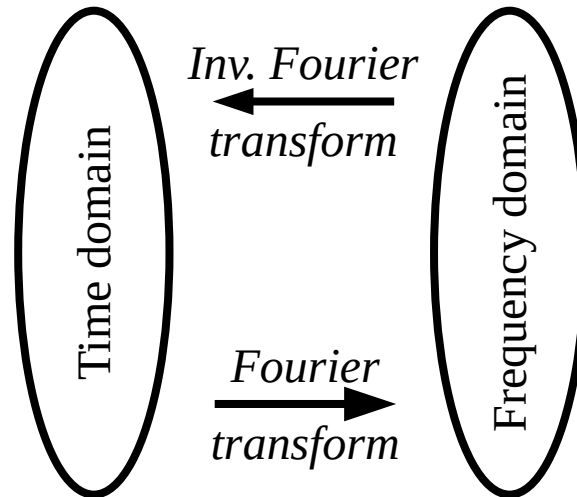
The work of Fourier

└ An innovation that led to many practical applications

https://en.wikipedia.org/wiki/Fourier_analysis

Applications [[edit](#)]

Fourier analysis has many scientific applications – in physics, partial differential equations, number theory, combinatorics, signal processing, digital image processing, probability theory, statistics, forensics, option pricing, cryptography, numerical analysis, acoustics, oceanography, sonar, optics, diffraction, geometry, protein structure analysis, and other areas.



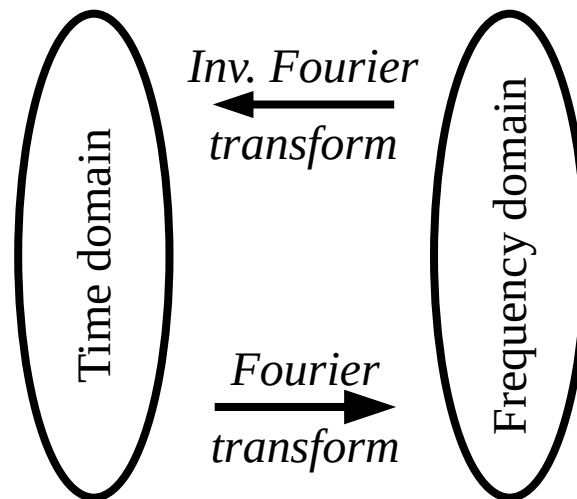
The work of Fourier

└ An innovation that led to many practical applications

https://en.wikipedia.org/wiki/Fourier_analysis

Applications [edit]

Fourier analysis has many scientific applications – in physics, partial differential equations, number theory, combinatorics, signal processing, digital image processing, probability theory, statistics, forensics, option pricing, cryptography, numerical analysis, acoustics, oceanography, sonar, optics, diffraction, geometry, protein structure analysis, and other areas.



Signal to process
(e.g., denoising)



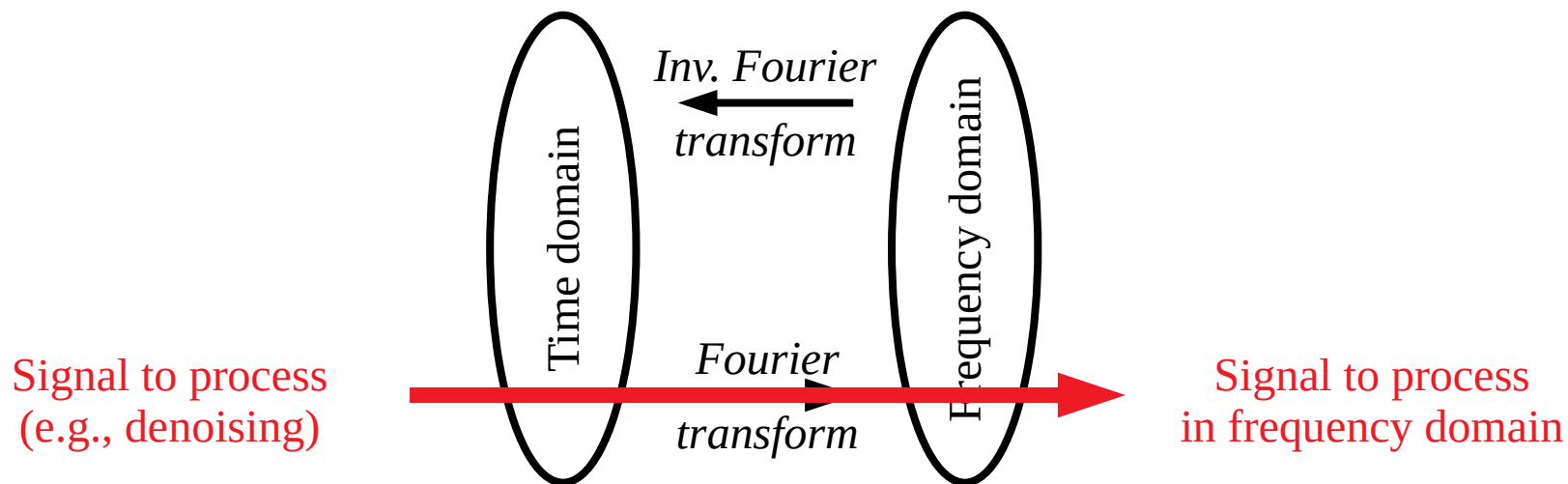
The work of Fourier

└ An innovation that led to many practical applications

https://en.wikipedia.org/wiki/Fourier_analysis

Applications [[edit](#)]

Fourier analysis has many scientific applications – in physics, partial differential equations, number theory, combinatorics, signal processing, digital image processing, probability theory, statistics, forensics, option pricing, cryptography, numerical analysis, acoustics, oceanography, sonar, optics, diffraction, geometry, protein structure analysis, and other areas.



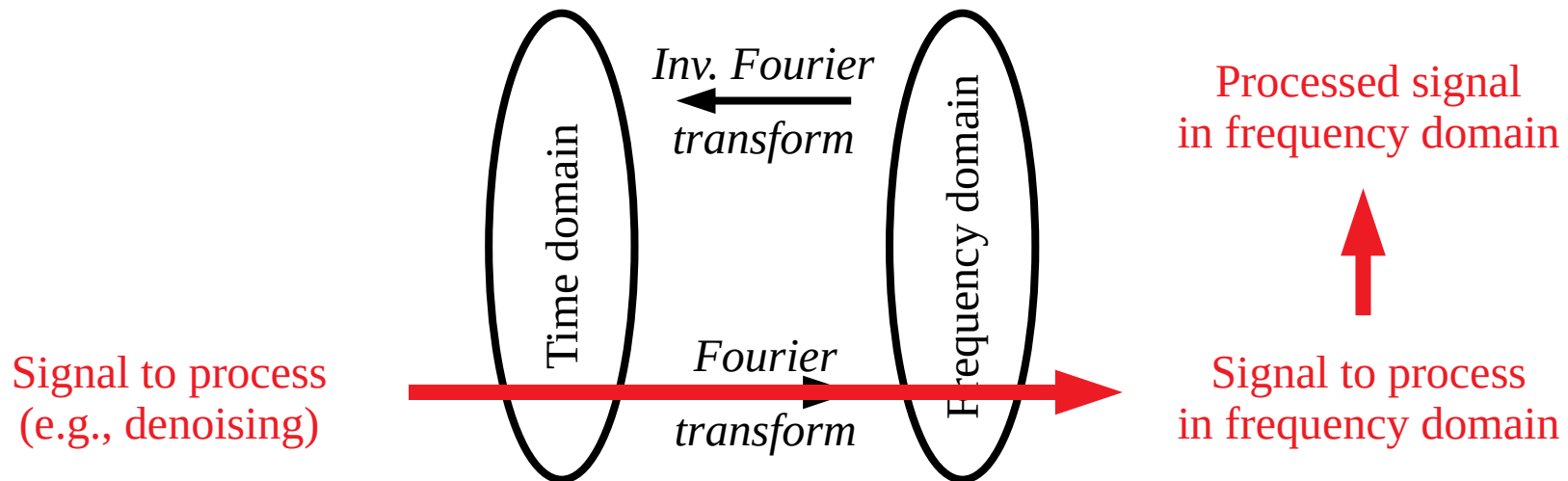
The work of Fourier

└ An innovation that led to many practical applications

https://en.wikipedia.org/wiki/Fourier_analysis

Applications [[edit](#)]

Fourier analysis has many scientific applications – in physics, partial differential equations, number theory, combinatorics, signal processing, digital image processing, probability theory, statistics, forensics, option pricing, cryptography, numerical analysis, acoustics, oceanography, sonar, optics, diffraction, geometry, protein structure analysis, and other areas.



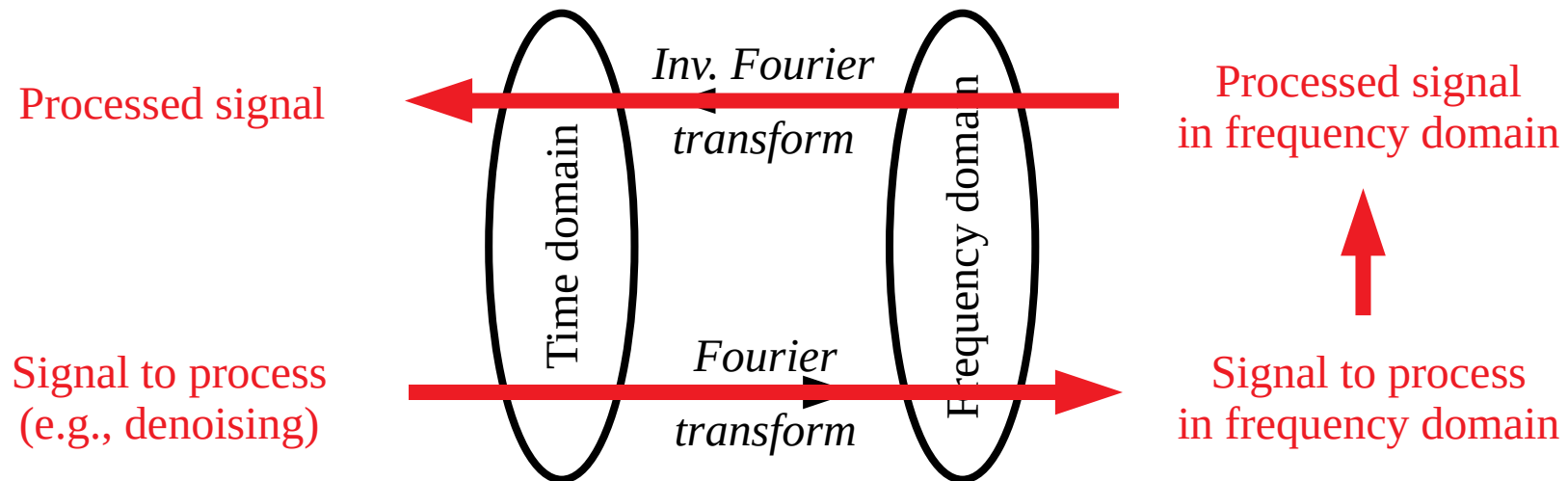
The work of Fourier

└ An innovation that led to many practical applications

https://en.wikipedia.org/wiki/Fourier_analysis

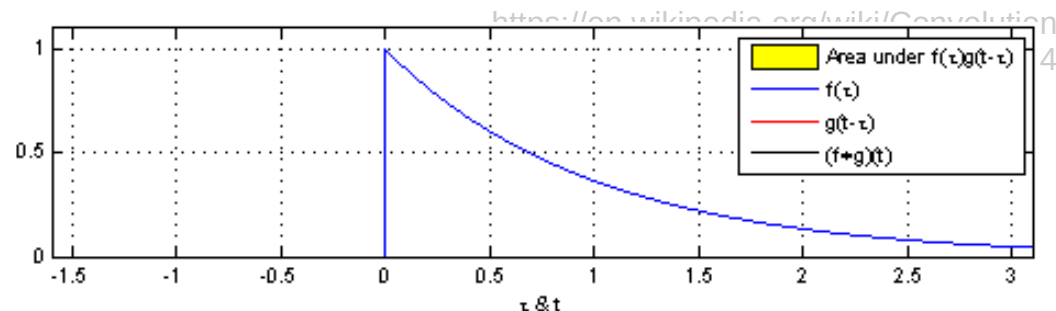
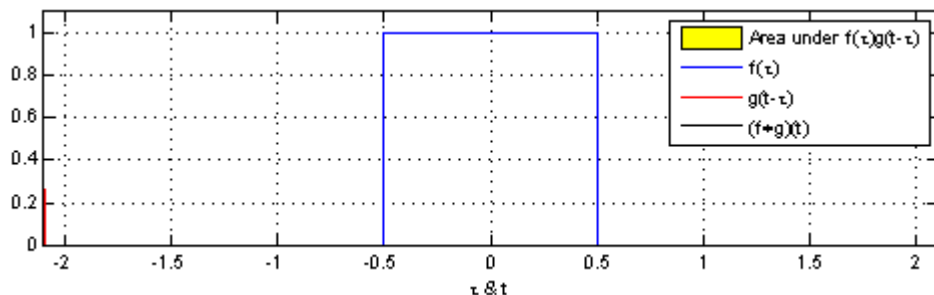
Applications [edit]

Fourier analysis has many scientific applications – in physics, partial differential equations, number theory, combinatorics, signal processing, digital image processing, probability theory, statistics, forensics, option pricing, cryptography, numerical analysis, acoustics, oceanography, sonar, optics, diffraction, geometry, protein structure analysis, and other areas.



Signal processing

Operators on temporal signals: convolution



Definition 38: Convolution of signals

Convolution of two signals, noted $s_1 * s_2$, is a function that associates with each time instant the sum of one signal, weighted by the other around origin, *i.e.*:

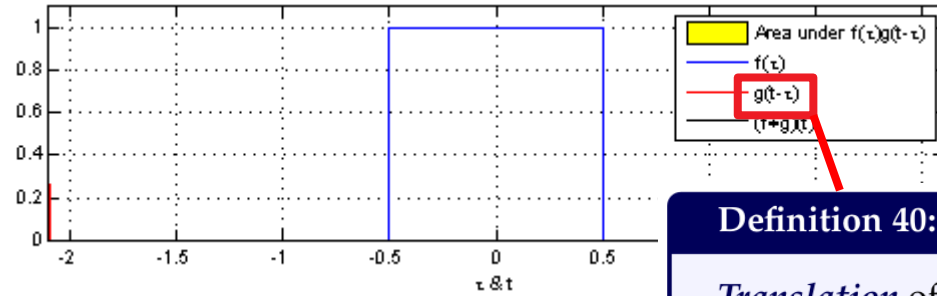
$$(s_1 * s_2)(t) = \int_{\mathbb{R}} s_1(t - \tau) s_2(\tau) d\tau = \int_{\mathbb{R}} s_1(\tau) s_2(t - \tau) d\tau . \quad (27)$$

What makes convolution an essential tool to filter signals is the fact that convolving signals in the time domain is equivalent to multiplying their spectrums:

$$s_1 * s_2 = \text{FT}^{-1}(\text{FT}(s_1)\text{FT}(s_2)) = \text{FT}^{-1}(\hat{s}_1 \hat{s}_2) . \quad (28)$$

Signal processing

Operators on temporal signals: translation



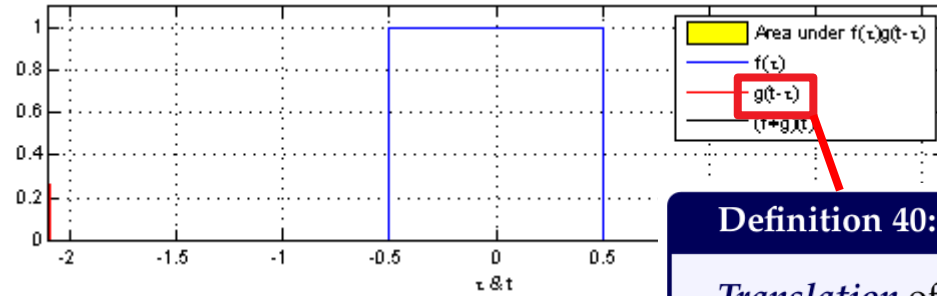
<https://tel.archives-ouvertes.fr/tel-01708824>
<https://en.wikipedia.org/wiki/Convolution>
https://en.wikipedia.org/wiki/Dirac_delta_function

Definition 40: Signal translation

Translation of a signal $s(t)$ by a quantity τ is performed by a change of variable, *i.e.*, by replacing $s(t)$ with $s(t - \tau)$.

Signal processing

Operators on temporal signals: translation



<https://tel.archives-ouvertes.fr/tel-01708824>
<https://en.wikipedia.org/wiki/Convolution>
https://en.wikipedia.org/wiki/Dirac_delta_function

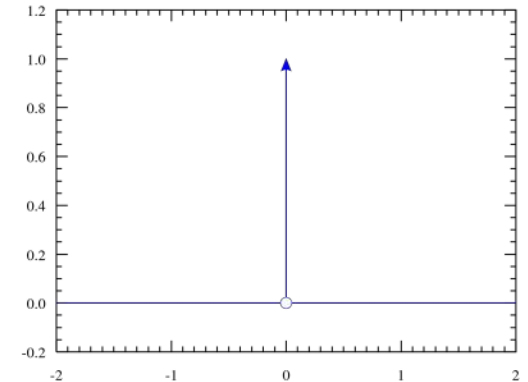
Definition 40: Signal translation

Translation of a signal $s(t)$ by a quantity τ is performed by a change of variable, i.e., by replacing $s(t)$ with $s(t - \tau)$.

Definition 39: Dirac delta signal

A *Dirac delta signal* at time 0 is a signal $\delta(t)$ such that:

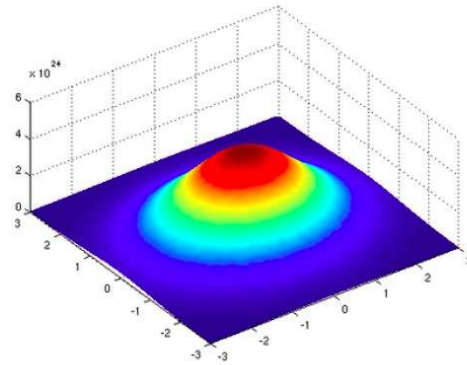
$$\delta(t) = \begin{cases} +\infty & \text{if } t = 0 \\ 0 & \text{otherwise} \end{cases} \quad (29)$$



Signal processing

Operators on temporal signals: translation

<https://en.wikipedia.org/wiki/Convolution>



In mathematics, **Schwartz space** $\mathcal{S}$ is the function space of all functions whose derivatives are rapidly decreasing. This space has the important property that the Fourier transform is an automorphism on this space. This property enables one, by duality, to define the Fourier transform for elements



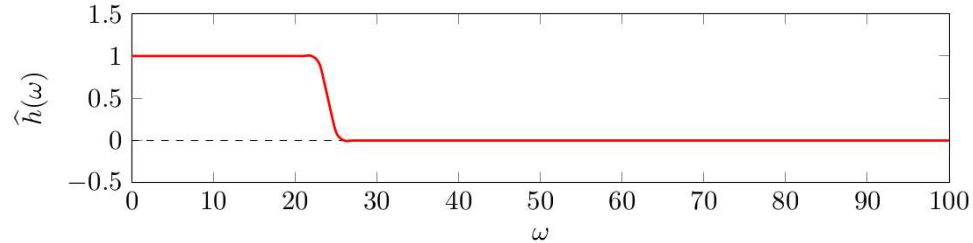
If f is a [Schwartz function](#), then $\tau_x f$ is the convolution with a translated Dirac delta function $\tau_x f = f * \tau_x \delta$. So translation invariance of the convolution of Schwartz functions is a consequence of the associativity of convolution.

Signal processing

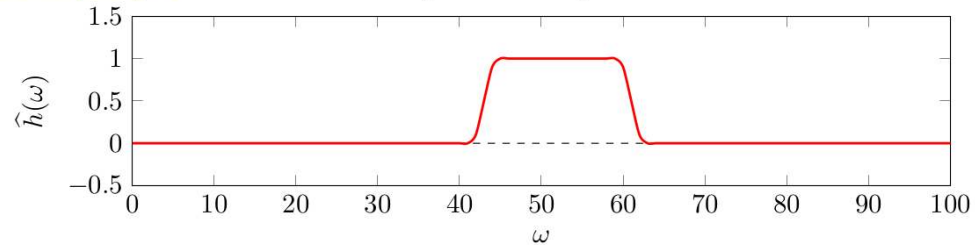
Filtering signals

<https://tel.archives-ouvertes.fr/tel-01708824>

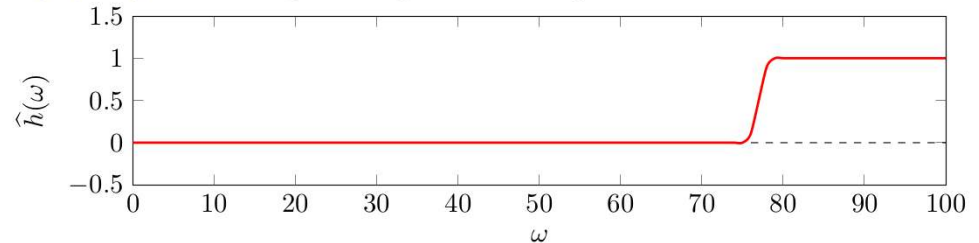
Low-pass filter: The lowest frequencies are kept, and the others are attenuated:



Band-pass filter: An interval of frequencies is kept, and the others are attenuated:

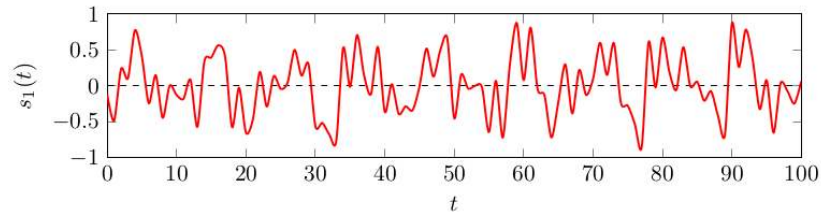


High-pass filter: The highest frequencies are kept, and the others are attenuated:



Signal processing

└ Filtering signals



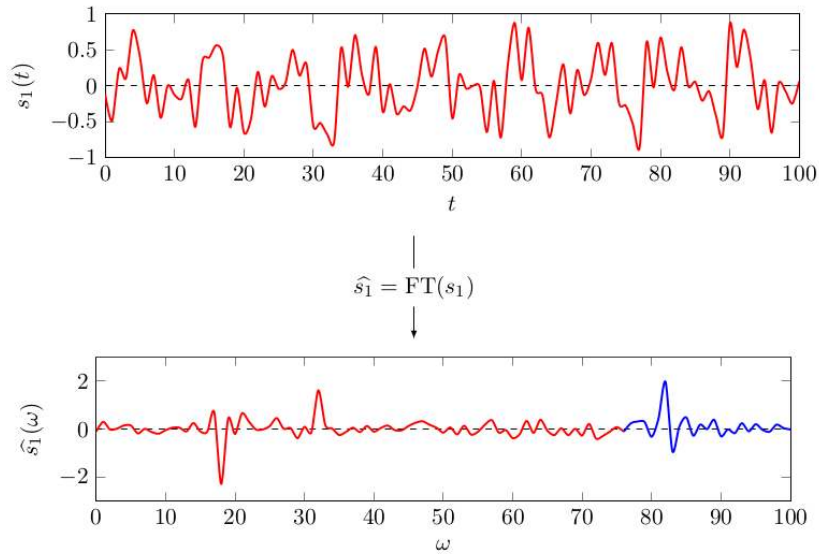
<https://tel.archives-ouvertes.fr/tel-01708824v246/how-to-get-the-low-middle-and-high-frequency-components-of-an-image>

Signal processing

Filtering signals

<https://tel.archives-ouvertes.fr/tel-01708824>

246/how-to-get-the-low-middle-and-high-frequency-components-of-an-image

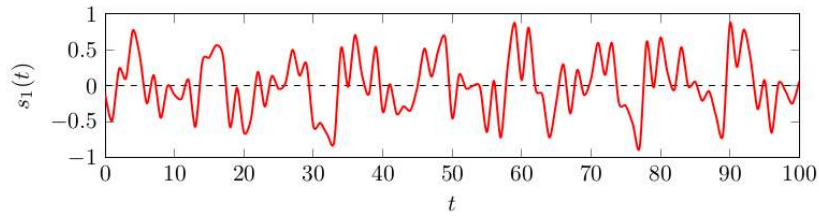


Signal processing

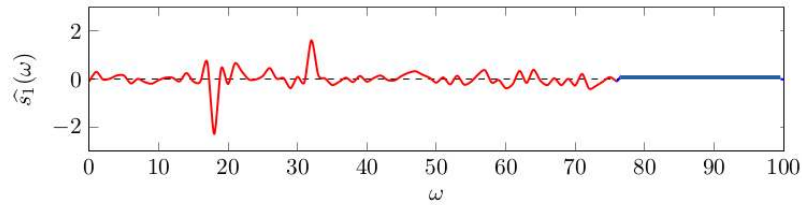
Filtering signals

<https://tel.archives-ouvertes.fr/tel-01708824>

246/how-to-get-the-low-middle-and-high-frequency-components-of-an-image



$$\hat{s}_1 = \text{FT}(s_1)$$

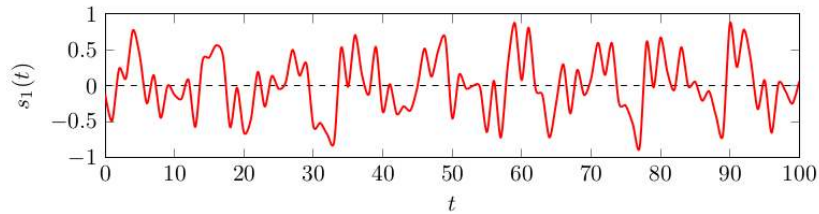


Signal processing

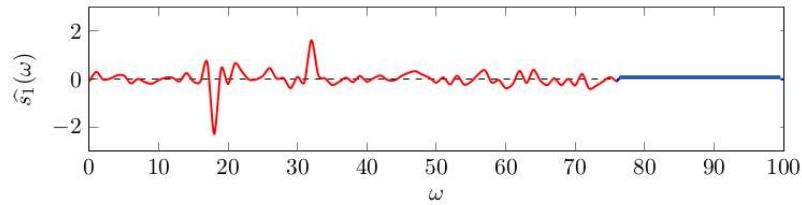
Filtering signals

<https://tel.archives-ouvertes.fr/tel-01708824>

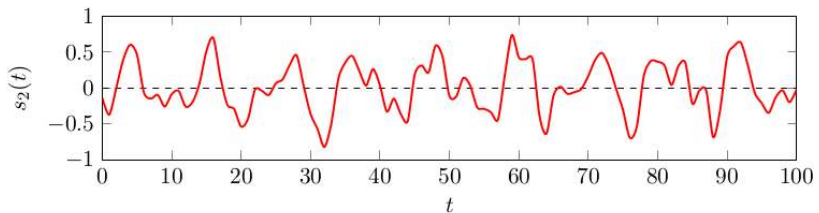
246/how-to-get-the-low-middle-and-high-frequency-components-of-an-image



$$\hat{s}_1 = \text{FT}(s_1)$$

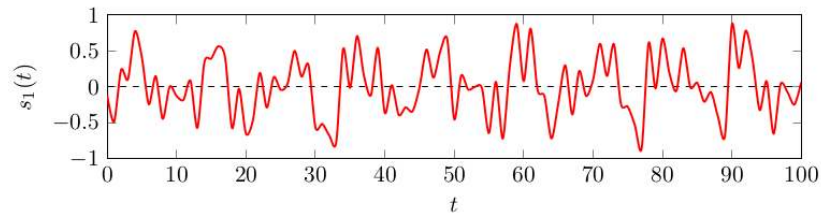


$$s_2 = \text{FT}^{-1}(\hat{h}\hat{s}_1)$$

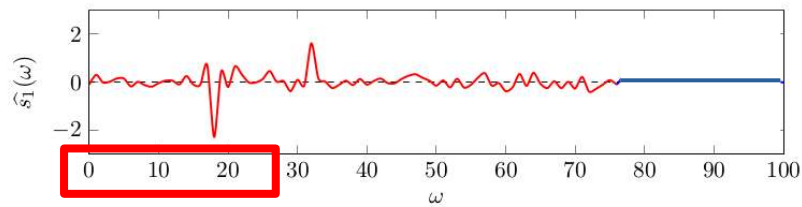


Signal processing

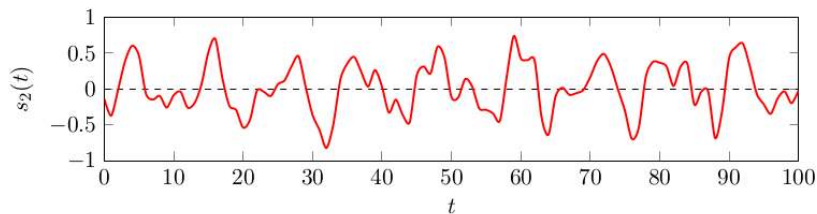
Filtering signals



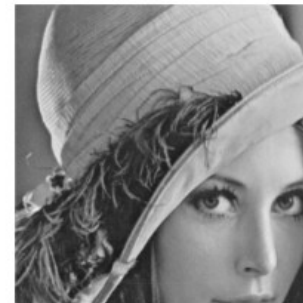
$$\hat{s}_1 = \text{FT}(s_1)$$



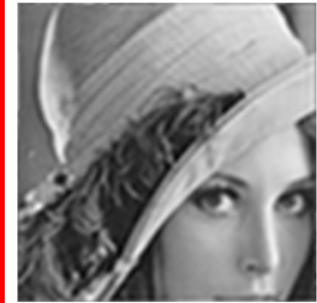
$$s_2 = \text{FT}^{-1}(\hat{h}\hat{s}_1)$$



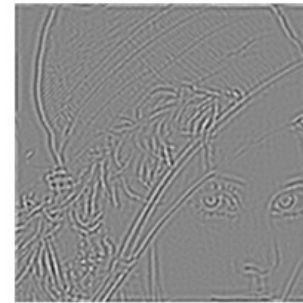
<https://tel.archives-ouvertes.fr/tel-01708824/246/how-to-get-the-low-middle-and-high-frequency-components-of-an-image>



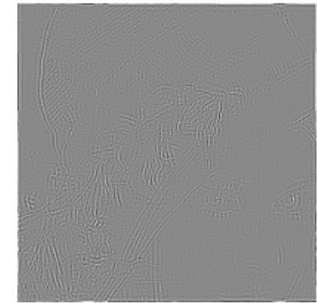
(a) Original image



(b) Low frequency component



(c) Middle frequency component



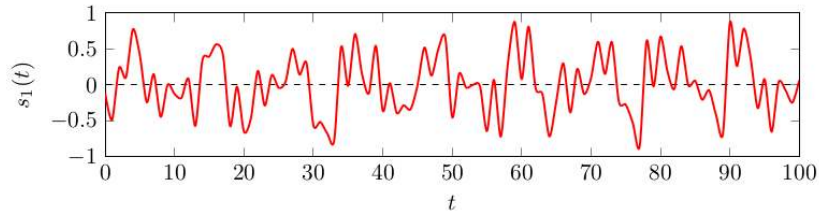
(d) High frequency component

Signal processing

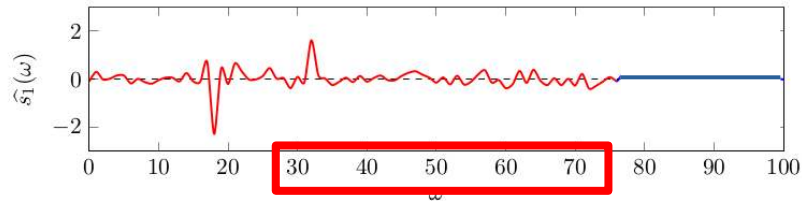
Filtering signals

<https://tel.archives-ouvertes.fr/tel-01708824>

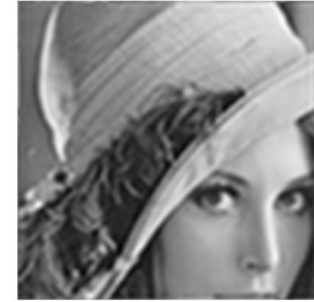
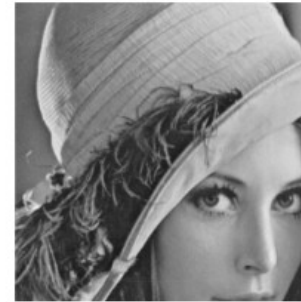
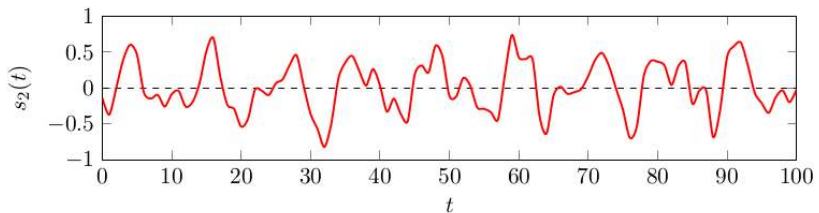
246/how-to-get-the-low-middle-and-high-frequency-components-of-an-image



$$\hat{s}_1 = \text{FT}(s_1)$$

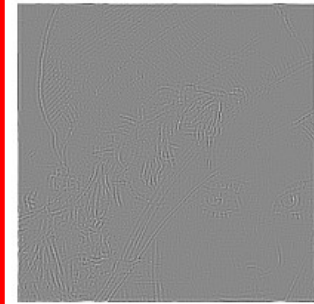
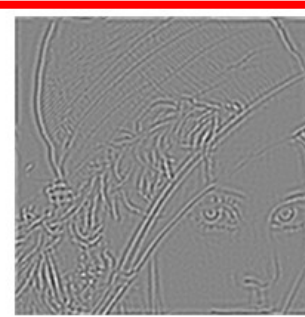


$$s_2 = \text{FT}^{-1}(\hat{h}\hat{s}_1)$$



(a) Original image

(b) Low frequency component



(c) Middle frequency component

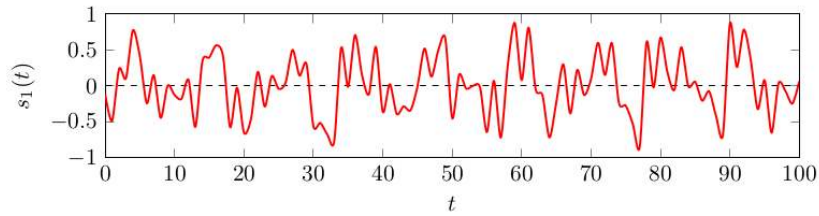
(d) High frequency component

Signal processing

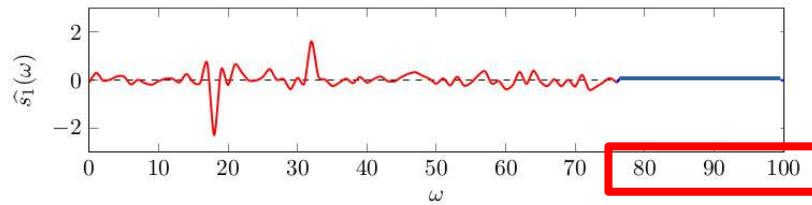
Filtering signals

<https://tel.archives-ouvertes.fr/tel-01708824>

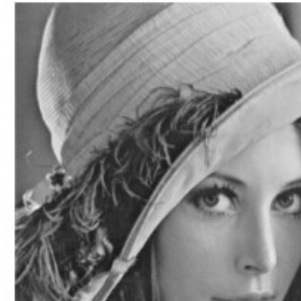
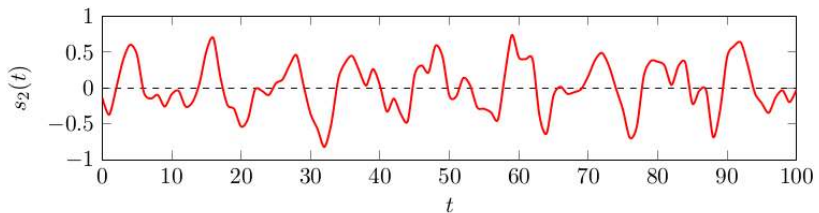
246/how-to-get-the-low-middle-and-high-frequency-components-of-an-image



$$\hat{s}_1 = \text{FT}(s_1)$$



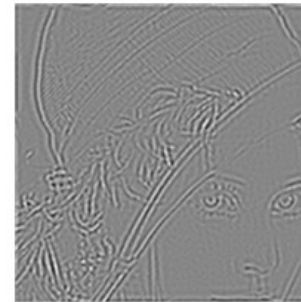
$$s_2 = \text{FT}^{-1}(\hat{h}\hat{s}_1)$$



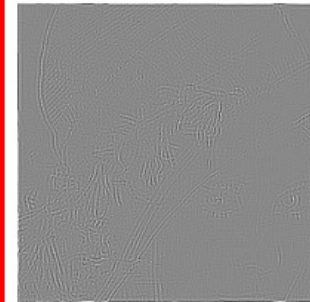
(a) Original image



(b) Low frequency component



(c) Middle frequency component

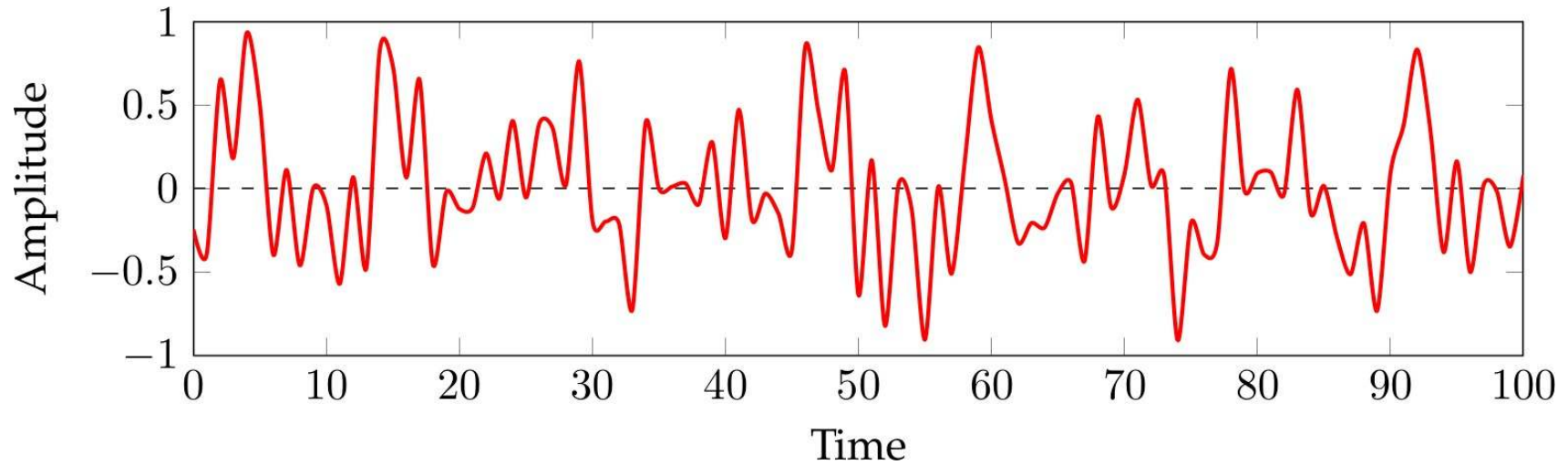


(d) High frequency component

Signal processing

└ Continuous vs. Discrete signal processing

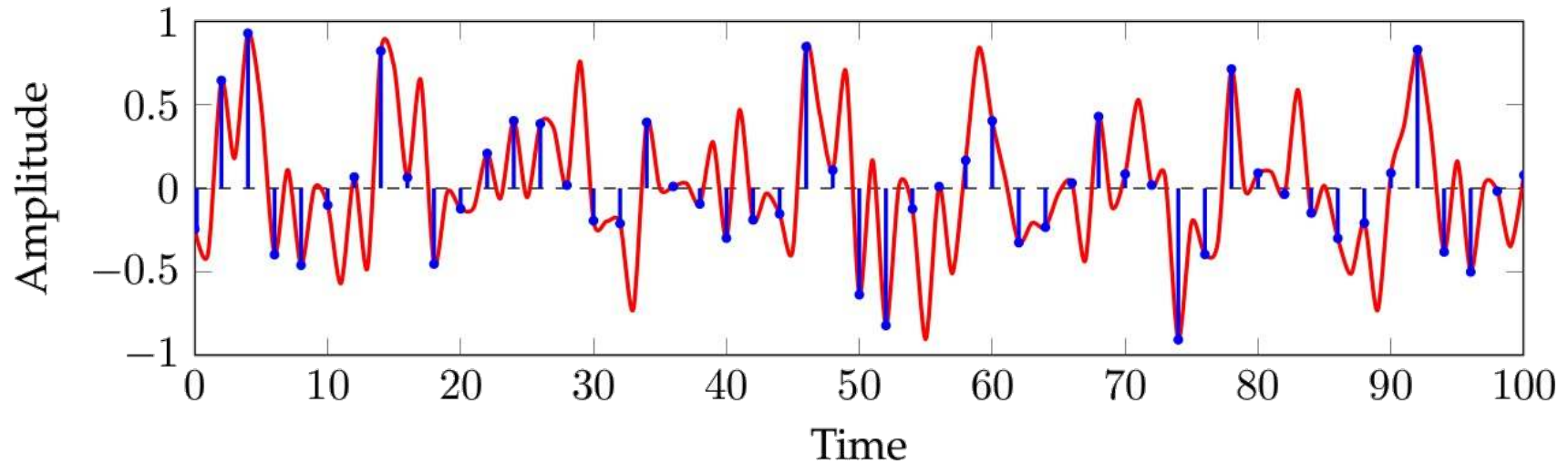
<https://tel.archives-ouvertes.fr/tel-01708824>



Signal processing

Continuous vs. Discrete signal processing

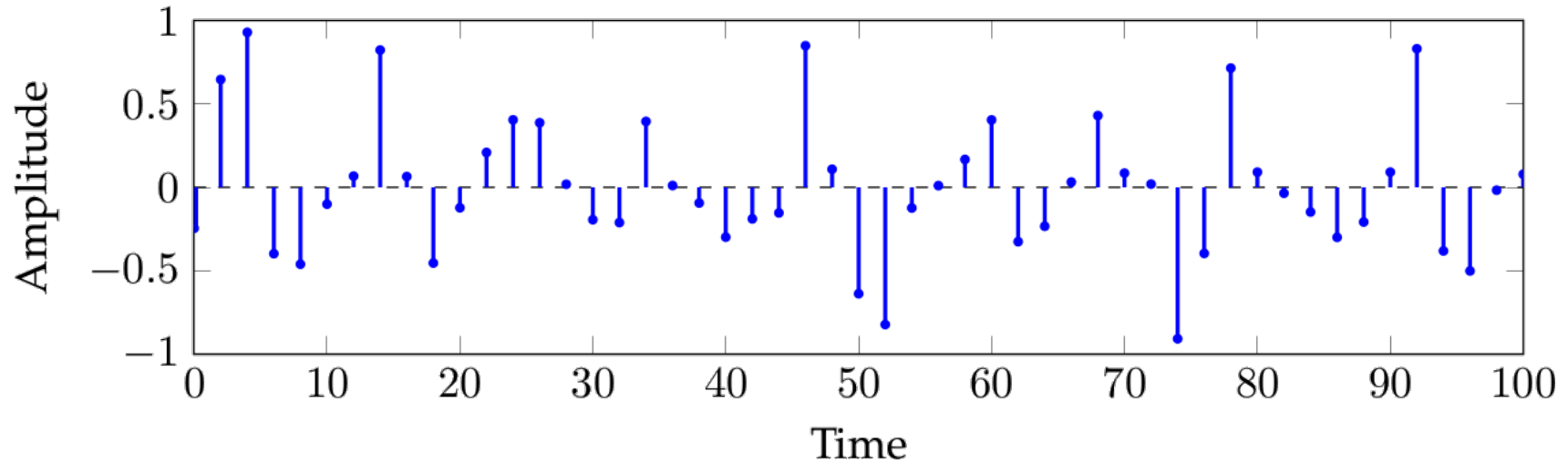
<https://tel.archives-ouvertes.fr/tel-01708824>



Signal processing

Continuous vs. Discrete signal processing

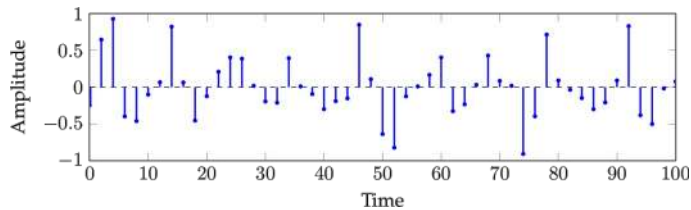
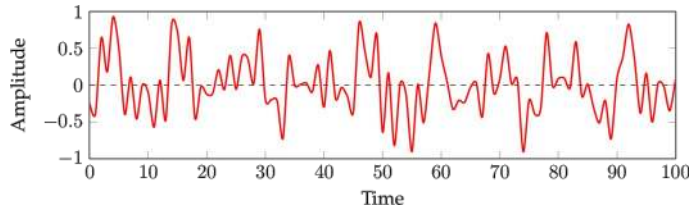
<https://tel.archives-ouvertes.fr/tel-01708824>



Signal processing

Continuous vs. Discrete signal processing

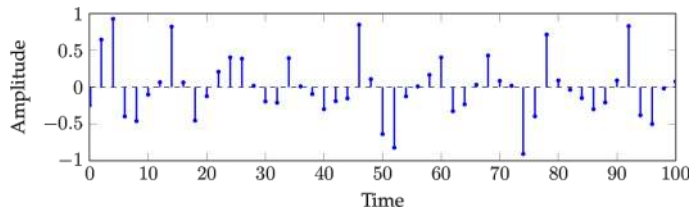
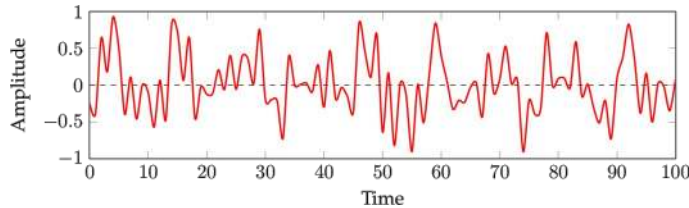
<https://tel.archives-ouvertes.fr/tel-01708824>



Signal processing

Continuous vs. Discrete signal processing

<https://tel.archives-ouvertes.fr/tel-01708824>



Definition 35: Signal sampling

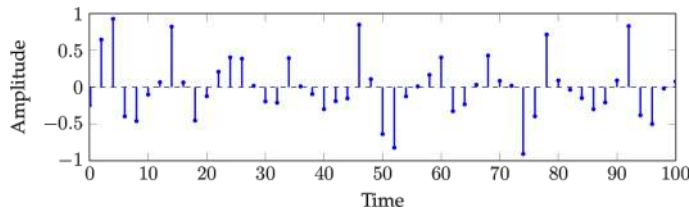
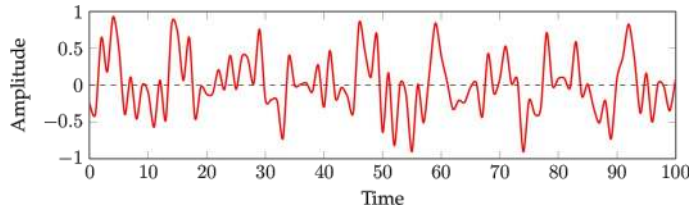
Signal sampling is the process of approximating a continuous signal by a discrete signal with entries taking their values in a finite set $\mathcal{S}$. It is done in two steps:

1. *Discretization* of the signal consists in dividing it into equal intervals of time, each interval being represented by a single amplitude;
2. *Quantization* of the discretized signal consists in approximating the amplitude by a finite value in $\mathcal{S}$.

Signal processing

Continuous vs. Discrete signal processing

<https://tel.archives-ouvertes.fr/tel-01708824>



Definition 35: Signal sampling

Signal sampling is the process of approximating a continuous signal by a discrete signal with entries taking their values in a finite set $\mathcal{S}$. It is done in two steps:

1. *Discretization* of the signal consists in dividing it into equal intervals of time, each interval being represented by a single amplitude;
2. *Quantization* of the discretized signal consists in approximating the amplitude by a finite value in $\mathcal{S}$.

Theorem 1: Nyquist-Shannon sampling theorem

A signal $s(t)$ of highest frequency $\omega_{\max}$ Hz is fully determined by subsampling it with a period of $\frac{1}{2\omega_{\max}}$.



Signal processing

Continuous vs. Discrete signal processing

<https://tel.archives-ouvertes.fr/tel-01708824>
[ipedia.org/wiki/Memory_cell_\(computing\)](https://ipedia.org/wiki/Memory_cell_(computing))

Definition 33: Fourier transform

The *Fourier transform* $\hat{s} = \text{FT}(s)$ of a continuous, integrable, signal $s(t)$ is:

$$\text{FT}(s) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ \omega & \mapsto \hat{s}(\omega) = \int_{\mathbb{R}} s(t) e^{-i2\pi\omega t} dt \end{cases}, \quad (23)$$

with t in seconds, and ω the frequency in Hz.

Definition 36: Discrete Fourier transform

The *discrete Fourier transform* $\hat{\mathbf{t}} = \text{DFT}(\mathbf{t})$ of a discrete signal $\mathbf{t}[t]$ of N samples is defined by:

$$\text{DFT}(\mathbf{t}) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ \omega & \mapsto \hat{\mathbf{t}}[\omega] = \sum_{t=1}^N \mathbf{t}[t] e^{-i2\pi\omega t} \end{cases}, \quad (25)$$

where $\hat{\mathbf{t}}$ can be seen as a vector with non-integer indices associated with the various frequencies.

Signal processing

Continuous vs. Discrete signal processing

<https://tel.archives-ouvertes.fr/tel-01708824>
[.wikipedia.org/wiki/Memory_cell_\(computing\)](https://en.wikipedia.org/wiki/Memory_cell_(computing))

Definition 33: Fourier transform

The *Fourier transform* $\hat{s} = \text{FT}(s)$ of a continuous, integrable, signal $s(t)$ is:

$$\text{FT}(s) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ \omega & \mapsto \hat{s}(\omega) = \int_{\mathbb{R}} s(t) e^{-i2\pi\omega t} dt \end{cases}, \quad (23)$$

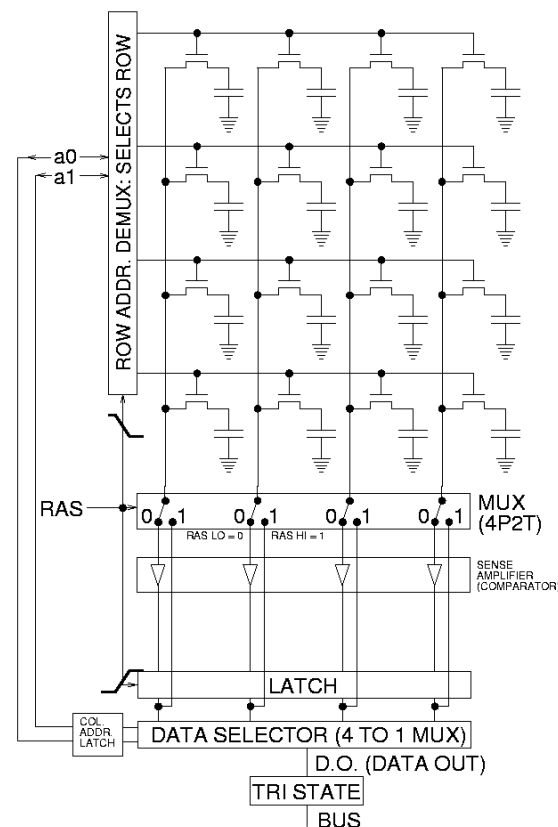
with t in seconds, and ω the frequency in Hz.

Definition 36: Discrete Fourier transform

The *discrete Fourier transform* $\hat{\mathbf{t}} = \text{DFT}(\mathbf{t})$ of a discrete signal $\mathbf{t}[t]$ of N samples is defined by:

$$\text{DFT}(\mathbf{t}) : \begin{cases} \mathbb{R} & \rightarrow \mathbb{C} \\ \omega & \mapsto \hat{\mathbf{t}}[\omega] = \sum_{t=1}^N \mathbf{t}[t] e^{-i2\pi\omega t} \end{cases}, \quad (25)$$

where $\hat{\mathbf{t}}$ can be seen as a vector with non-integer indices associated with the various frequencies.



Signal processing

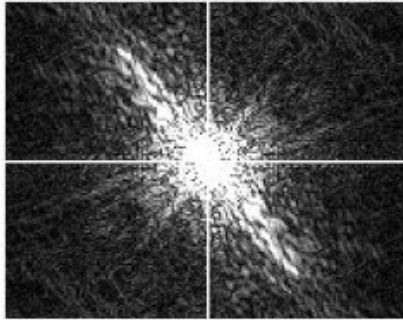
└ Extension to images

<https://www.youtube.com/watch?v=4gp-3pFFyPk>

<https://math.stackexchange.com/questions/1672584/fourier-transform-of-a-2d-image-and-noise-cancellation>



Spatial domain

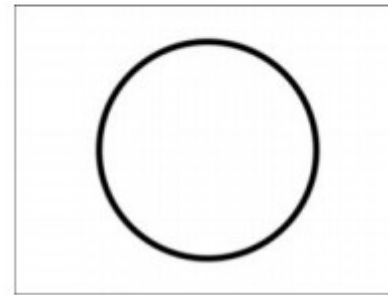
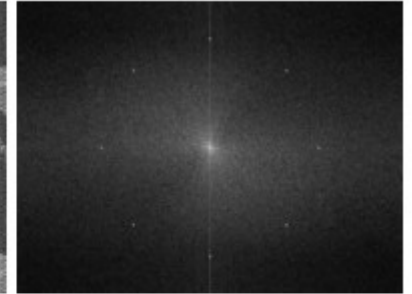


Frequency domain

a b
c d

FIGURE 2.40

(a) Image corrupted by sinusoidal interference. (b) Magnitude of the Fourier transform showing the bursts of energy responsible for the interference. (c) Mask used to eliminate the energy bursts. (d) Result of computing the inverse of the modified Fourier transform. (Original image courtesy of NASA.)



Signal processing

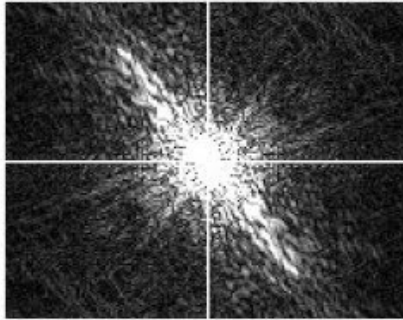
└ Extension to images

<https://www.youtube.com/watch?v=4gp-3pFFyPk>

<https://math.stackexchange.com/questions/1672584/fourier-transform-of-a-2d-image-and-noise-cancellation>



Spatial domain

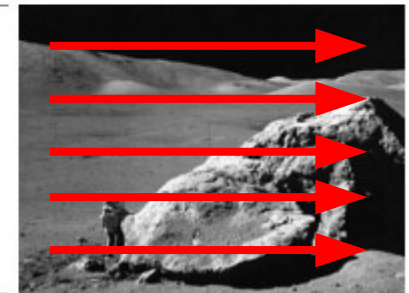
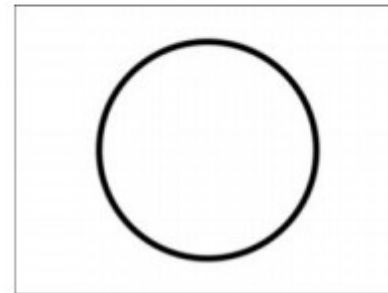
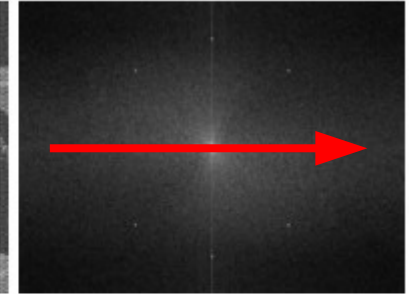


Frequency domain

a b
c d

FIGURE 2.40

(a) Image corrupted by sinusoidal interference. (b) Magnitude of the Fourier transform showing the bursts of energy responsible for the interference. (c) Mask used to eliminate the energy bursts. (d) Result of computing the inverse of the modified Fourier transform. (Original image courtesy of NASA.)



Signal processing

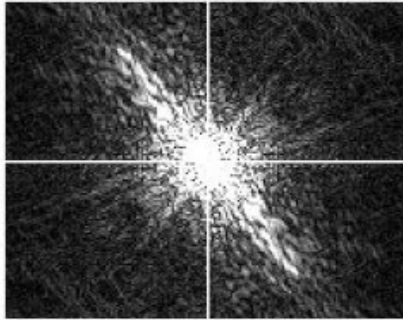
└ Extension to images

<https://www.youtube.com/watch?v=4gp-3pFFyPk>

<https://math.stackexchange.com/questions/1672584/fourier-transform-of-a-2d-image-and-noise-cancellation>



Spatial domain

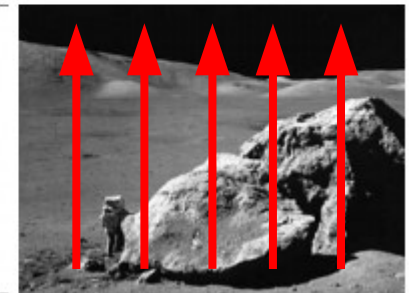
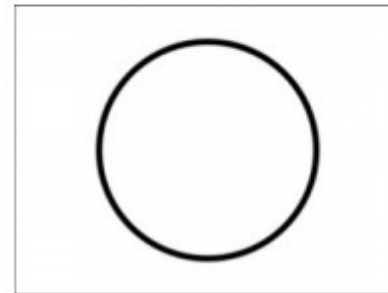
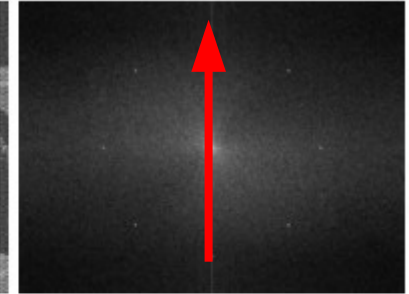


Frequency domain

a b
c d

FIGURE 2.40

(a) Image corrupted by sinusoidal interference. (b) Magnitude of the Fourier transform showing the bursts of energy responsible for the interference. (c) Mask used to eliminate the energy bursts. (d) Result of computing the inverse of the modified Fourier transform. (Original image courtesy of NASA.)



Signal processing

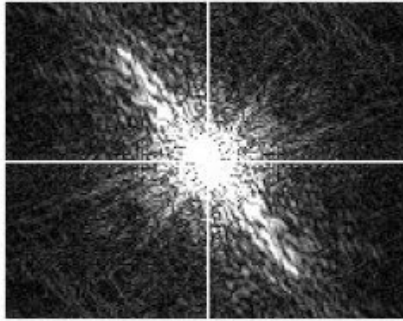
└ Extension to images

<https://www.youtube.com/watch?v=4gp-3pFFyPk>

<https://math.stackexchange.com/questions/1672584/fourier-transform-of-a-2d-image-and-noise-cancellation>



Spatial domain

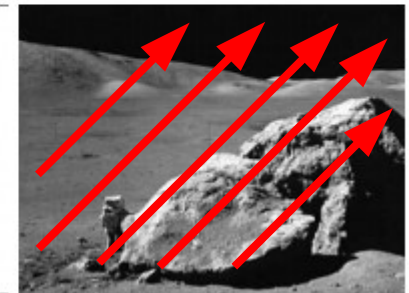
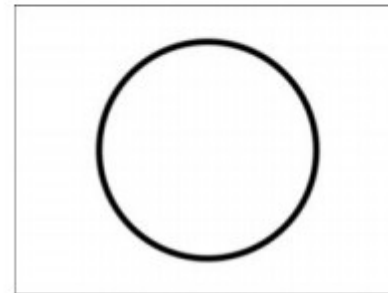
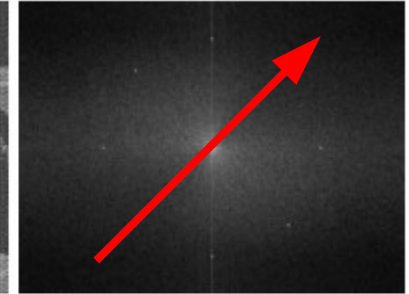


Frequency domain

a b
c d

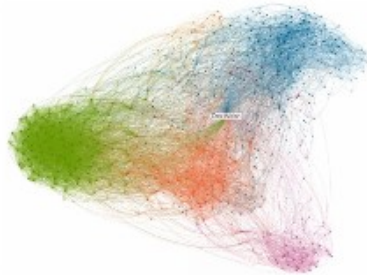
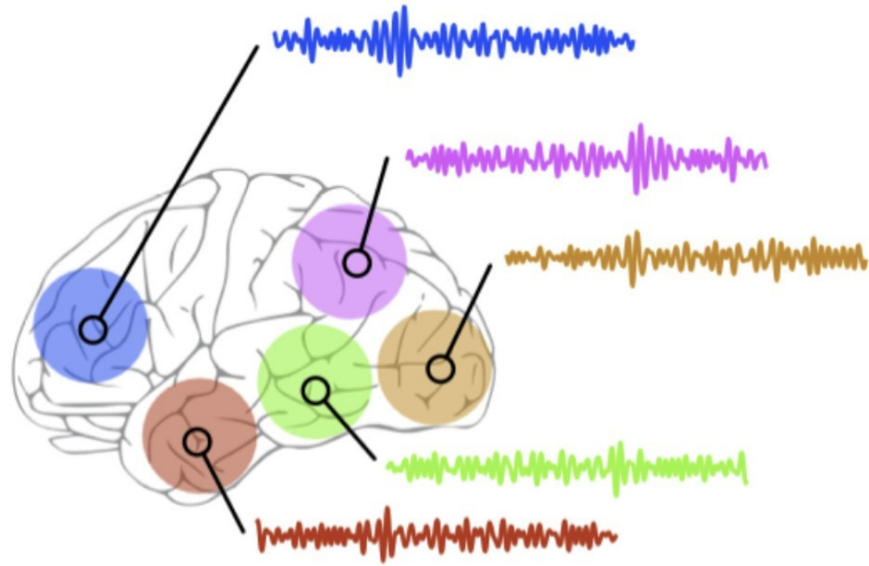
FIGURE 2.40

(a) Image corrupted by sinusoidal interference. (b) Magnitude of the Fourier transform showing the bursts of energy responsible for the interference. (c) Mask used to eliminate the energy bursts. (d) Result of computing the inverse of the modified Fourier transform. (Original image courtesy of NASA.)

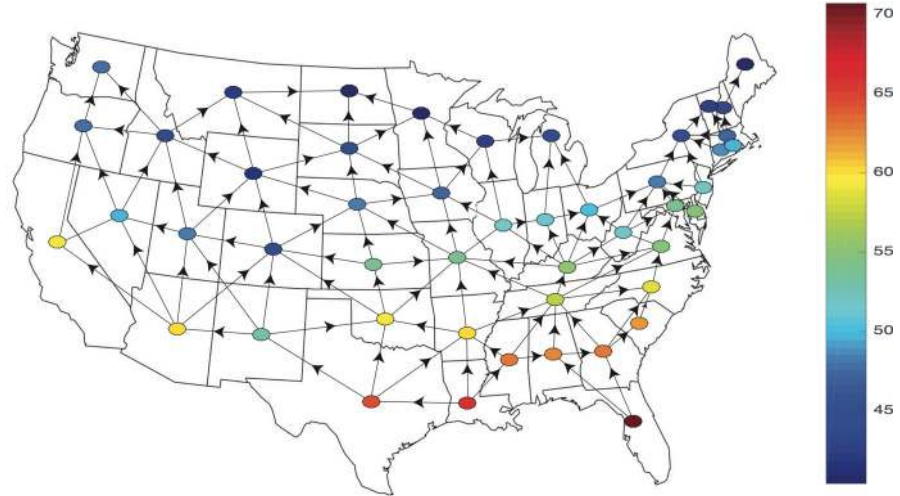


Signal processing

What data looks like today



<http://www2.ece.rochester.edu/~rshafipo/research.htm>
<https://actu.epfl.ch/news/graph-learning-overview/>
<https://pygsp.readthedocs.io/en/stable/>



Signal processing

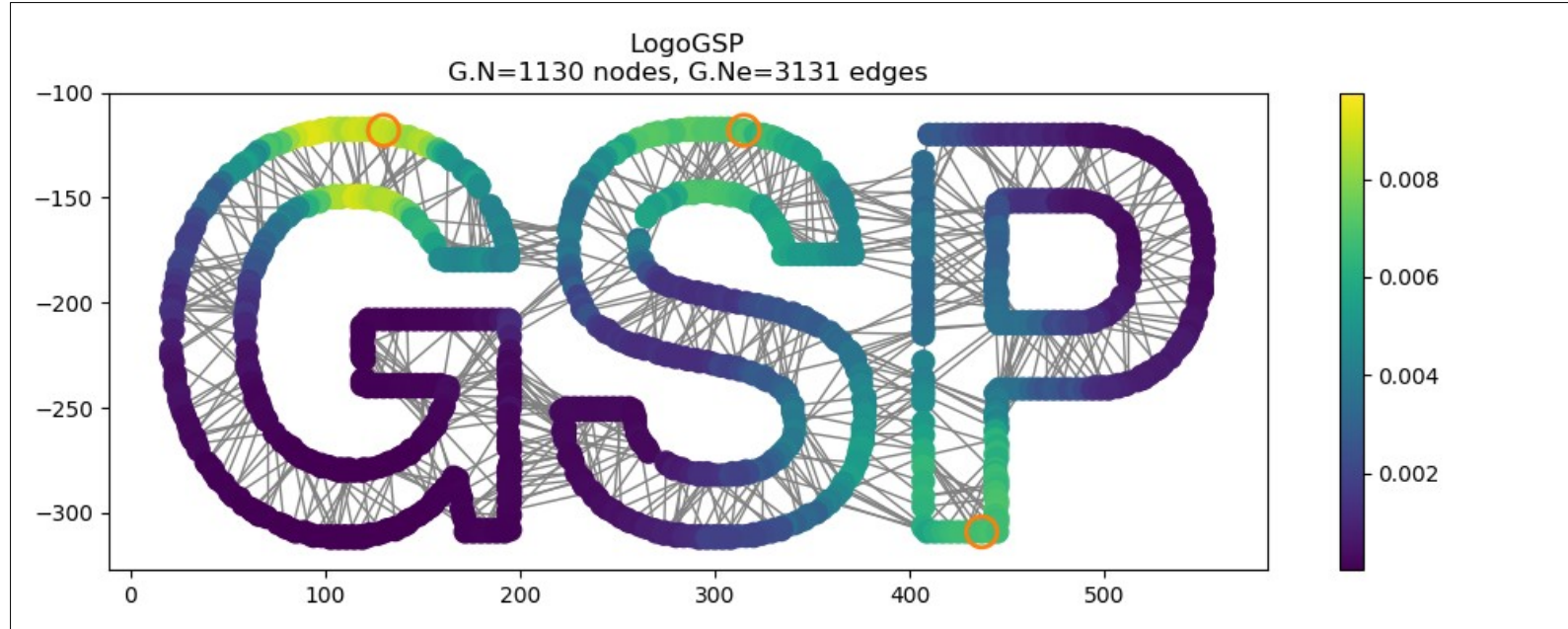
└ What data looks like today

<http://www2.ece.rochester.edu/~rshafipo/research.html>

<https://actu.epfl.ch/news/graph-learning-overview/>

<http://www.ixxi.fr/actualites/phd-thesis-in-graph-signal-processing-multi-scale-classification-and-analysis-of-data-on-networks>

<https://pygsp.readthedocs.io/en/stable/>



OUTLINE

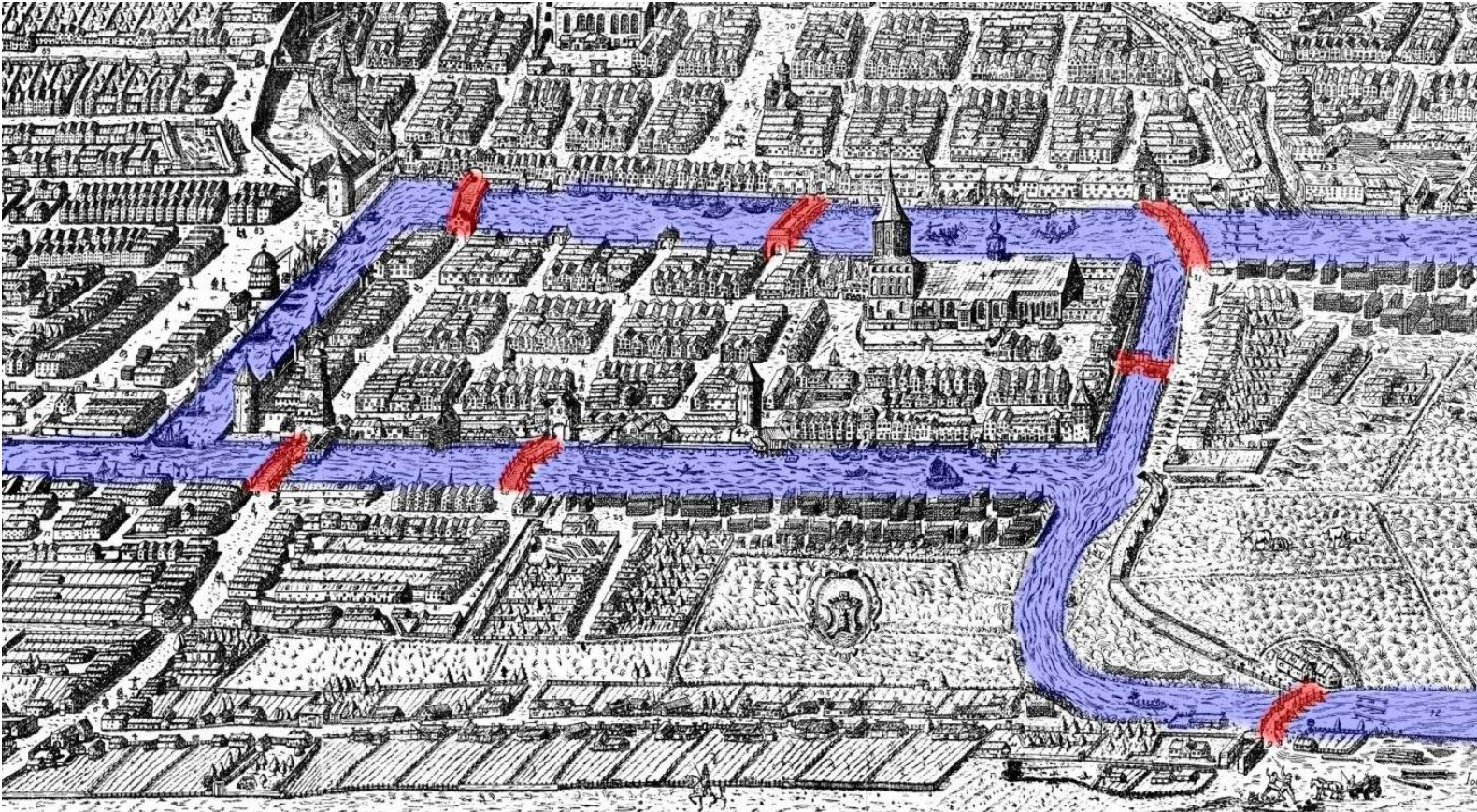
- ▶ Signal processing
- ▶ **Graph theory**
- ▶ Graph signal processing
- ▶ Focus :
Graph convolutional networks



Graph theory

└ Euler & the city of Königsberg (1736)

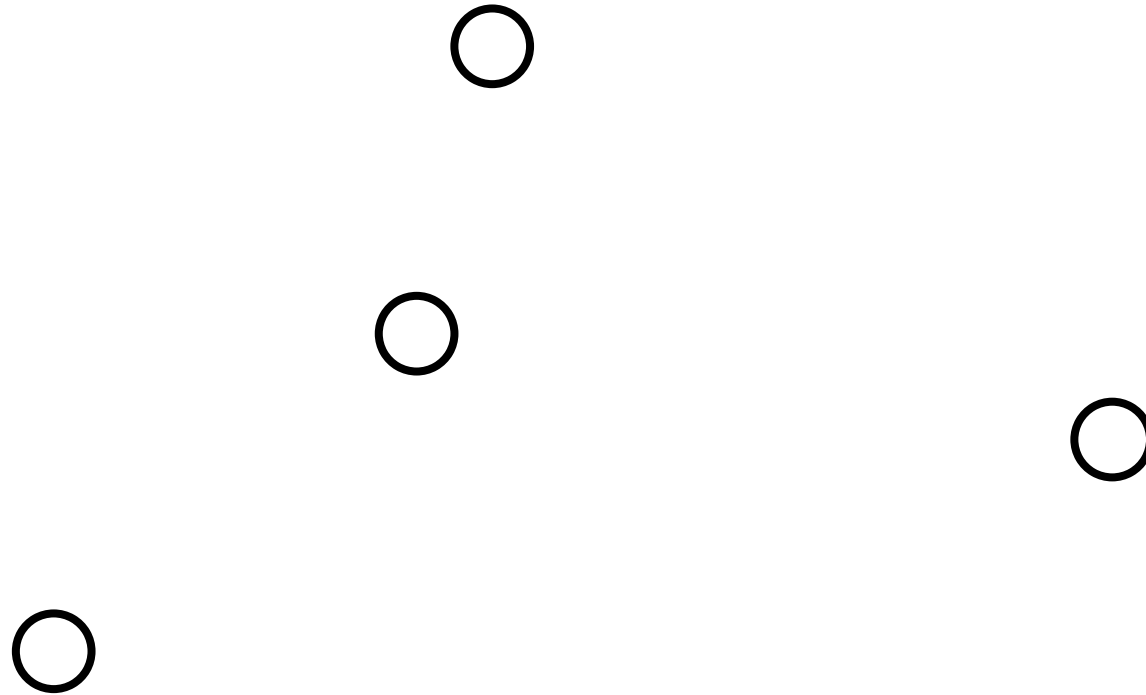
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph theory

└ Euler & the city of Königsberg (1736)

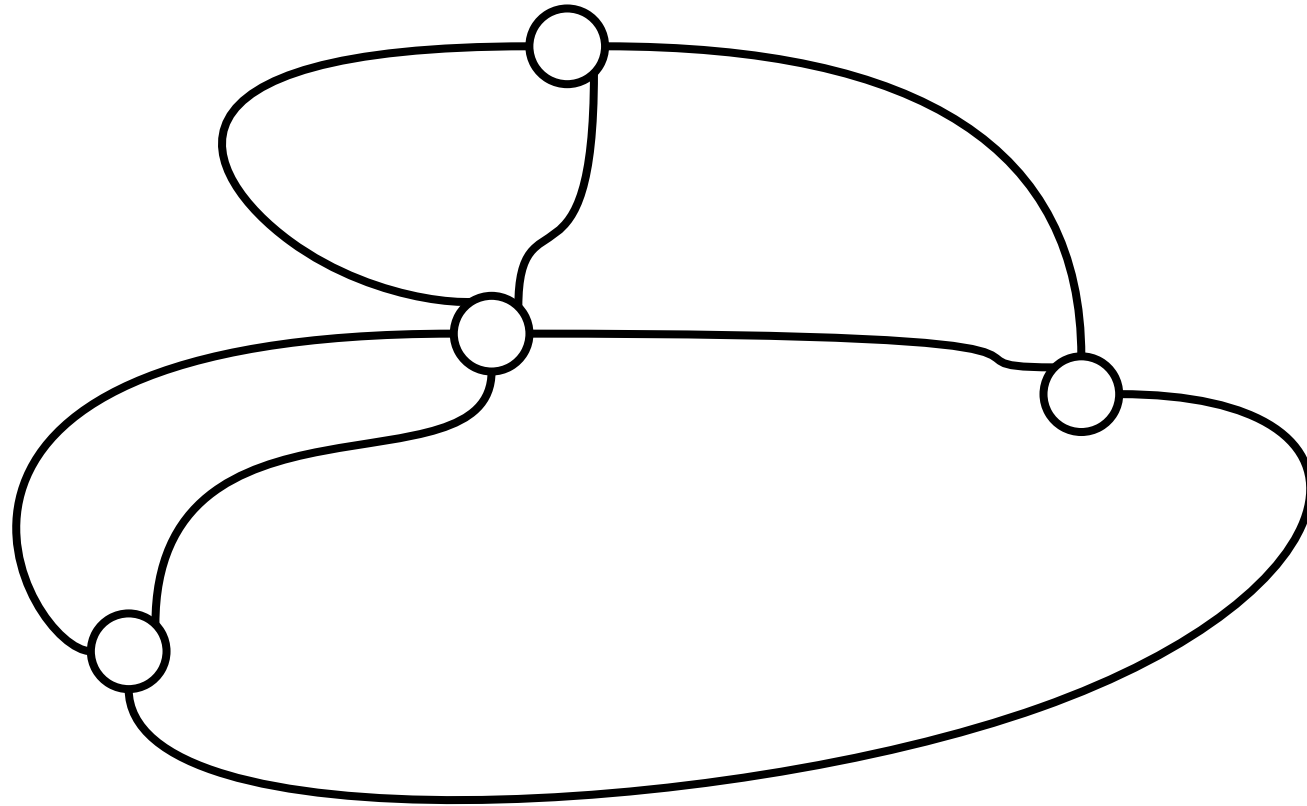
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph theory

└ Euler & the city of Königsberg (1736)

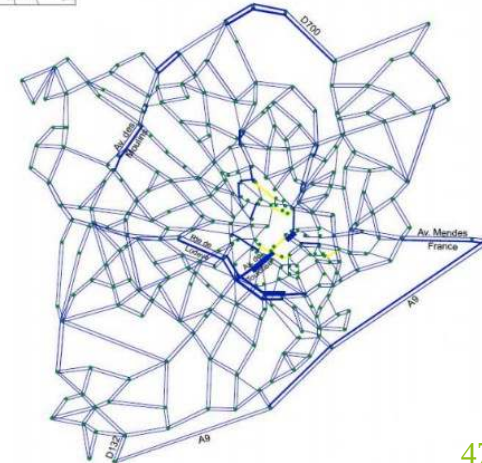
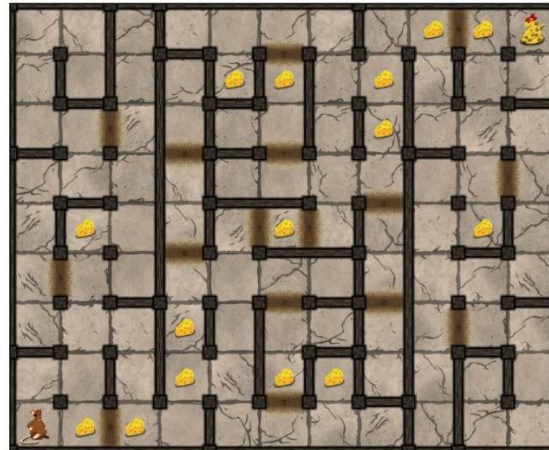
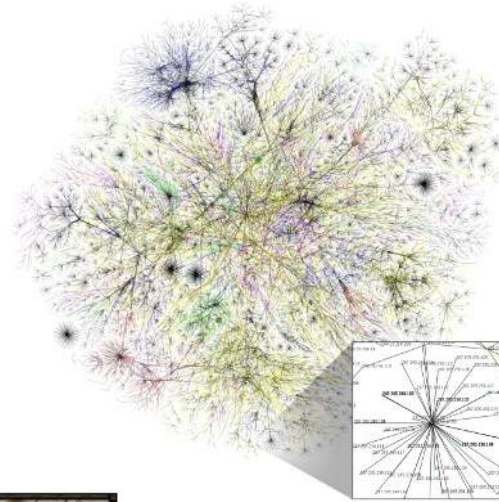
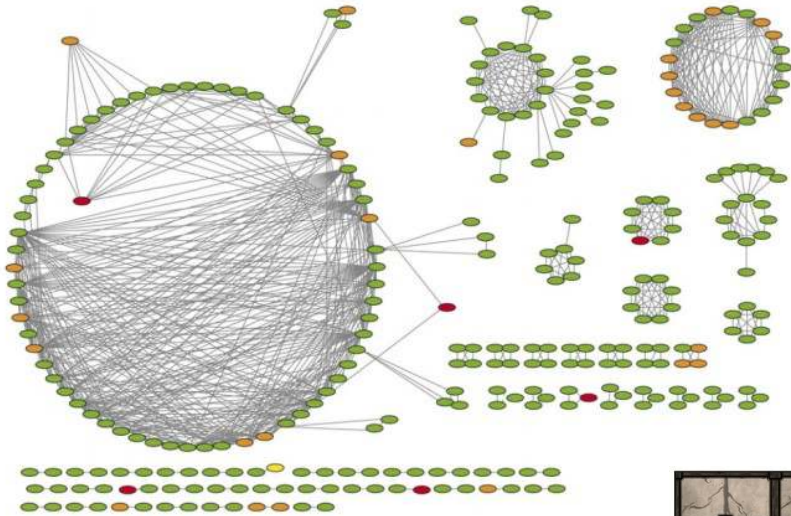
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph theory

└ Graphs in everyday life

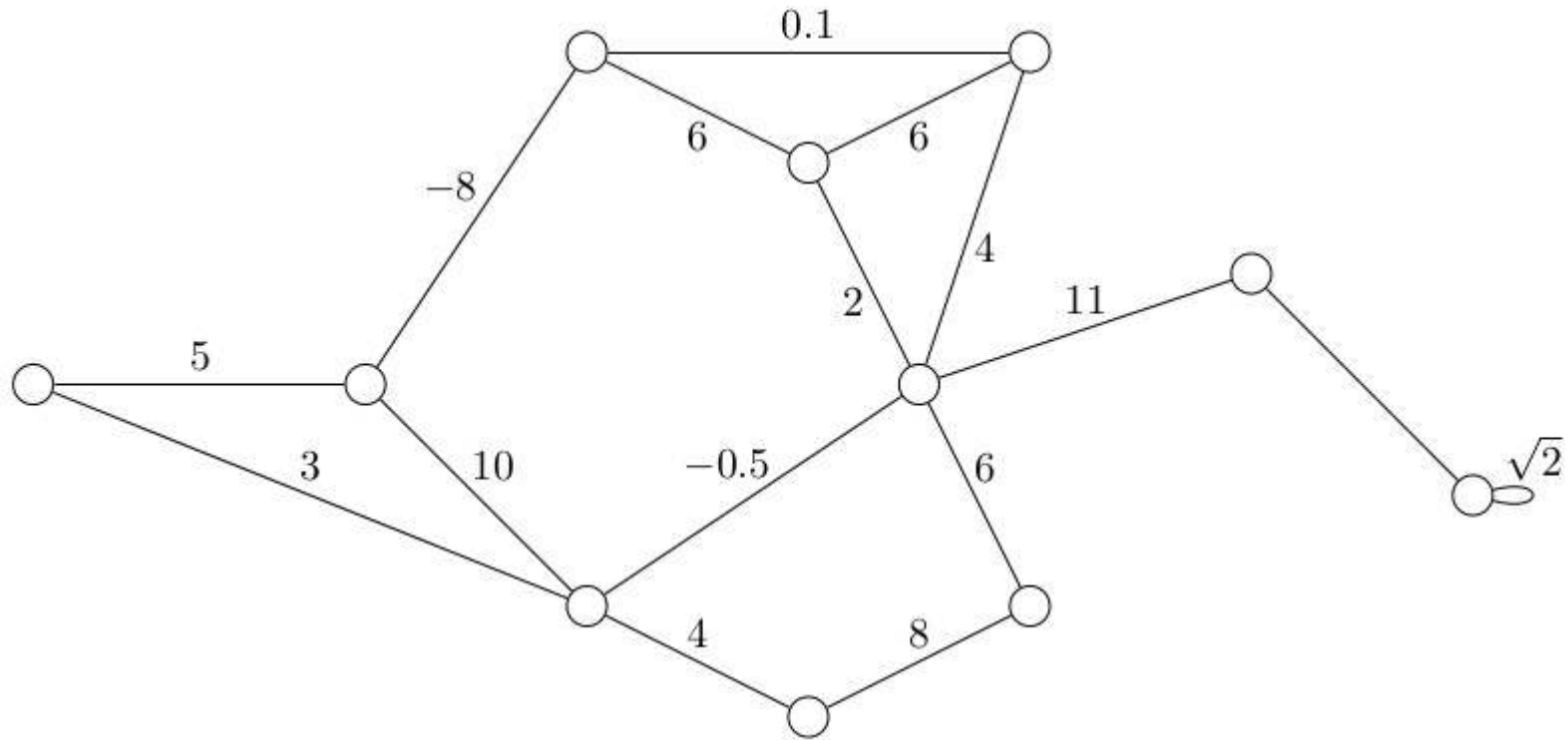
ives-ouvertes.fr/tel-01708824



Graph theory

└ What is a graph ?

<https://tel.archives-ouvertes.fr/tel-01708824>



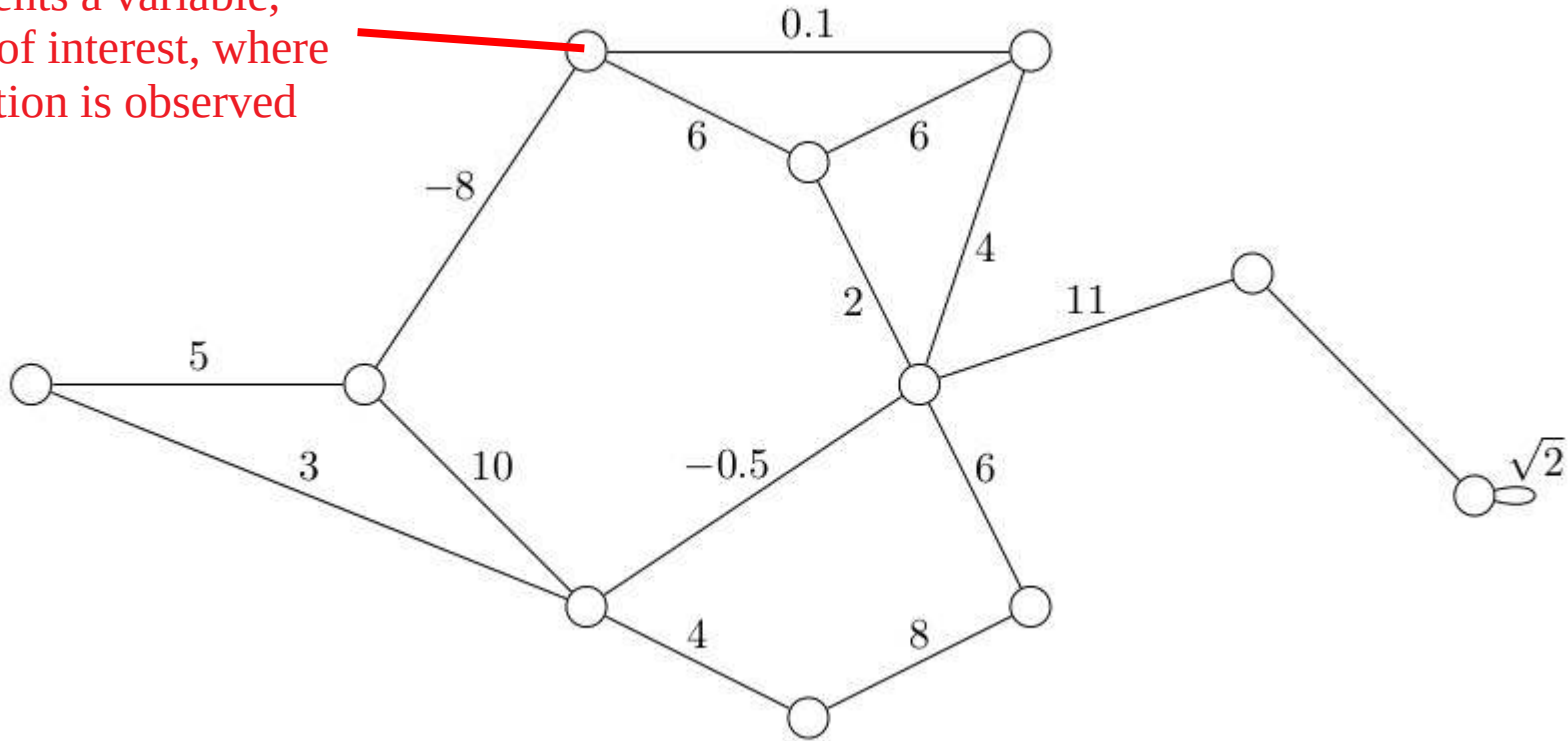
Graph theory

└ What is a graph ?

<https://tel.archives-ouvertes.fr/tel-01708824>

Vertex

Represents a variable,
an object of interest, where
information is observed



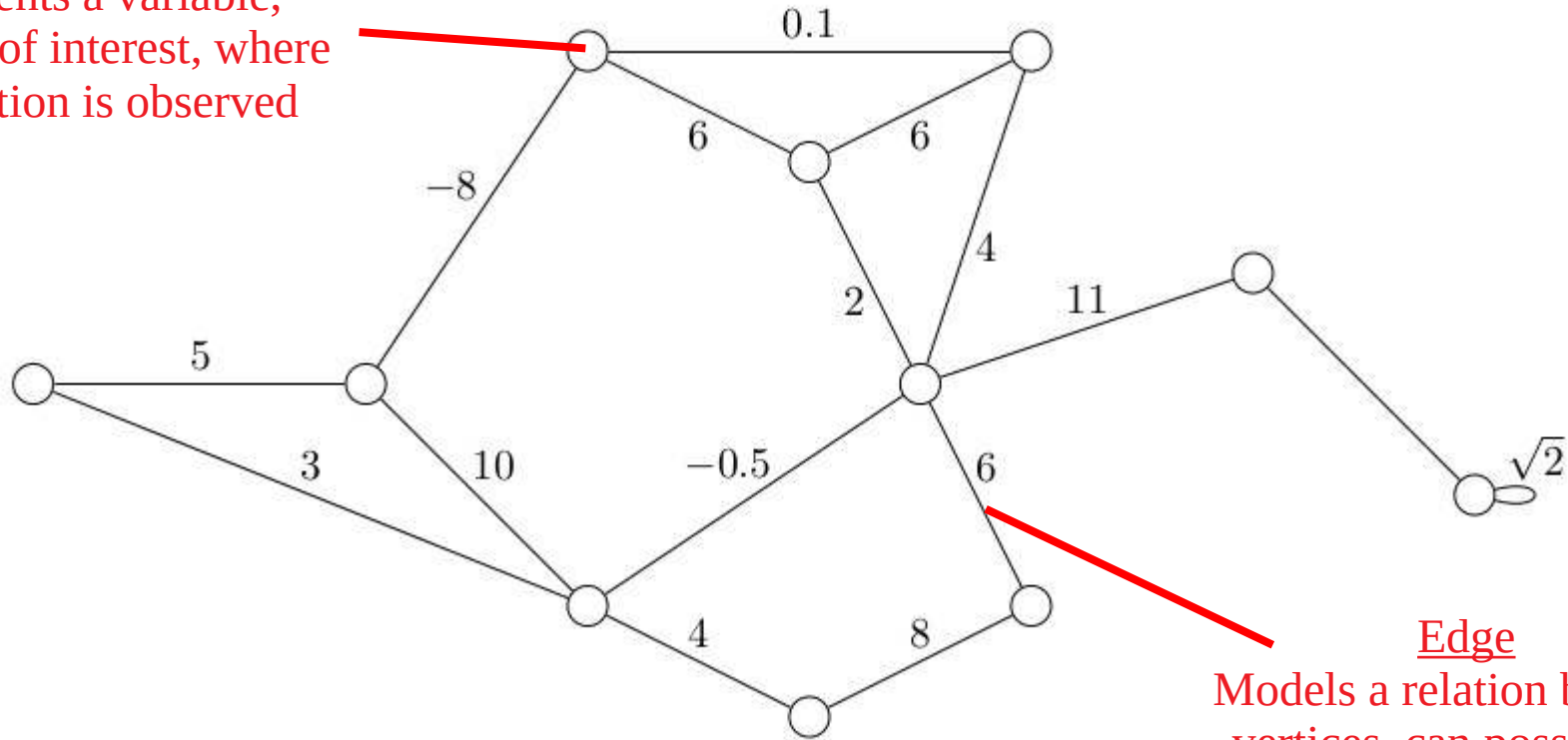
Graph theory

What is a graph ?

<https://tel.archives-ouvertes.fr/tel-01708824>

Vertex

Represents a variable,
an object of interest, where
information is observed



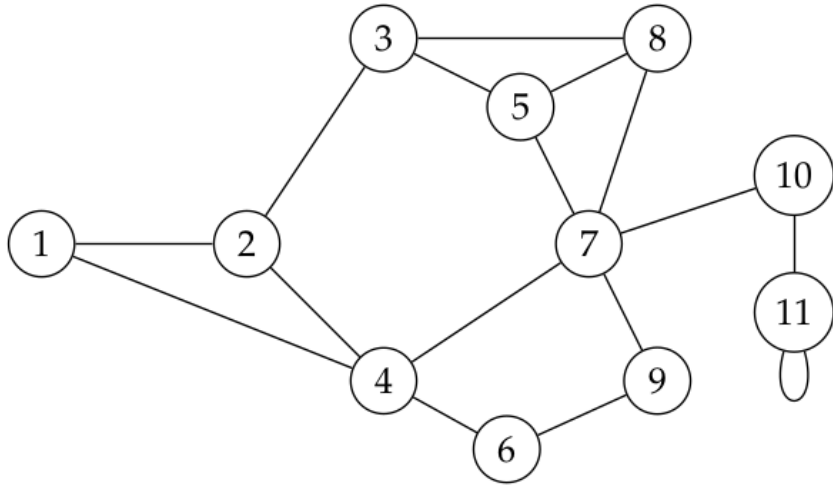
Edge

Models a relation between
vertices, can possibly be
weighted or directed

Graph theory

Adjacency/weights matrix

<https://tel.archives-ouvertes.fr/tel-01708824>



	①	②	③	④	⑤	⑥	⑦	⑧	⑨	⑩	⑪
①	0	1	0	1	0	0	0	0	0	0	0
②	1	0	1	1	0	0	0	0	0	0	0
③	0	1	0	0	1	0	0	1	0	0	0
④	1	1	0	0	0	1	1	0	0	0	0
⑤	0	0	1	0	0	0	1	1	0	0	0
⑥	0	0	0	1	0	0	0	0	1	0	0
⑦	0	0	0	1	1	0	0	1	1	1	0
⑧	0	0	1	0	1	0	1	0	0	0	0
⑨	0	0	0	0	0	1	1	0	0	0	0
⑩	0	0	0	0	0	0	1	0	0	0	1
⑪	0	0	0	0	0	0	0	0	0	1	1

Definition 12: Weights matrix

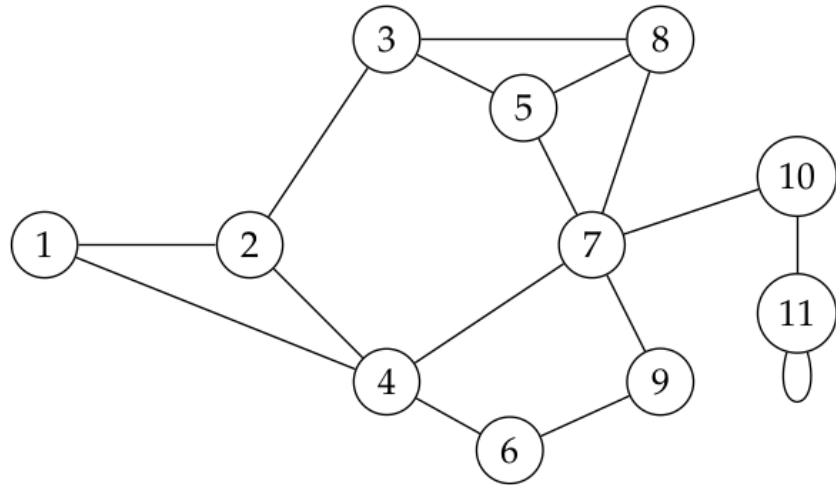
The *weights matrix* $\mathbf{W}$ of a graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E}, f \rangle$ is an $N \times N$ matrix with:

$$\forall v_1, v_2 \in \mathcal{V} : \mathbf{W}[v_1, v_2] = \begin{cases} f(\{v_1, v_2\}) & \text{if } \{v_1, v_2\} \in \mathcal{E} \\ 0 & \text{otherwise} \end{cases} . \quad (5)$$

Graph theory

└ Degrees matrix

<https://tel.archives-ouvertes.fr/tel-01708824>



	①	②	③	④	⑤	⑥	⑦	⑧	⑨	⑩	⑪
①	2	0	0	0	0	0	0	0	0	0	0
②	0	3	0	0	0	0	0	0	0	0	0
③	0	0	3	0	0	0	0	0	0	0	0
④	0	0	0	4	0	0	0	0	0	0	0
⑤	0	0	0	0	3	0	0	0	0	0	0
⑥	0	0	0	0	0	2	0	0	0	0	0
⑦	0	0	0	0	0	0	5	0	0	0	0
⑧	0	0	0	0	0	0	0	3	0	0	0
⑨	0	0	0	0	0	0	0	0	2	0	0
⑩	0	0	0	0	0	0	0	0	0	2	0
⑪	0	0	0	0	0	0	0	0	0	0	2

Definition 21: Indegrees matrix

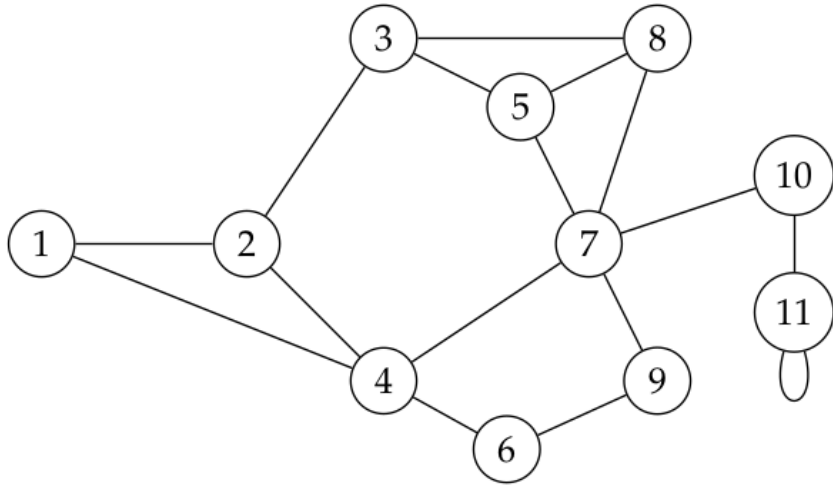
The *indegrees matrix* $\mathbf{D}_{\text{in}}$ of a weighted digraph $\vec{\mathcal{G}} = \langle \mathcal{V}, \vec{\mathcal{E}}, f \rangle$ of order N and weights matrix $\mathbf{W}$ is an $N \times N$ diagonal matrix with:

$$\forall v_1, v_2 \in \mathcal{V} : \mathbf{D}_{\text{in}}[v_1, v_2] = \begin{cases} \sum_{v_3 \in \mathcal{V}} \mathbf{W}[v_3, v_1] & \text{if } v_1 = v_2 \\ 0 & \text{otherwise} \end{cases} . \quad (15)$$

Graph theory

Laplacian matrix

<https://tel.archives-ouvertes.fr/tel-01708824>



	①	②	③	④	⑤	⑥	⑦	⑧	⑨	⑩	⑪
①	2	-1	0	-1	0	0	0	0	0	0	0
②	-1	3	-1	-1	0	0	0	0	0	0	0
③	0	-1	3	0	-1	0	0	-1	0	0	0
④	-1	-1	0	4	0	-1	-1	0	0	0	0
⑤	0	0	-1	0	3	0	-1	-1	0	0	0
⑥	0	0	0	-1	0	2	0	0	-1	0	0
⑦	0	0	0	-1	-1	0	5	-1	-1	-1	0
⑧	0	0	-1	0	-1	0	-1	3	0	0	0
⑨	0	0	0	0	0	-1	-1	0	2	0	0
⑩	0	0	0	0	0	0	-1	0	0	2	-1
⑪	0	0	0	0	0	0	0	0	0	-1	2

Definition 22: Laplacian matrix

The **Laplacian matrix** L of a simple weighted digraph of order N , weights matrix W and degrees matrix D is an $N \times N$ matrix computed as follows:

$$L = D - W . \quad (16)$$

If the graph is undirected, then $\mathbf{L}$ is symmetric and real-valued, thus diagonalizable by an orthonormal matrix. Let us denote its eigenvalues $\lambda = (\lambda_1, \dots, \lambda_N)$, sorted by increasing value. This matrix has numerous interesting properties, among which the following ones [Chu97]:

- $\forall i \in \llbracket 1, N \rrbracket : \lambda_i \geq 0$;
- The number of occurrences of eigenvalue 0 is called *algebraic multiplicity* of the graph, and corresponds to the number of connected components in the graph;
- λ_2 is called *algebraic connectivity* of the graph, and measures how connected a graph is.

The Laplacian matrix can also be given in its normalized version as follows:

Definition 23: Normalized Laplacian matrix

The *normalized Laplacian matrix* $\mathcal{L}$ of a simple weighted digraph of order N , weights matrix $\mathbf{W}$ and degrees matrix $\mathbf{D}$ is an $N \times N$ matrix computed as follows:

$$\mathcal{L} = \mathbf{I}_N - \mathbf{D}^{-\frac{1}{2}} \mathbf{W} \mathbf{D}^{-\frac{1}{2}} . \quad (17)$$

OUTLINE

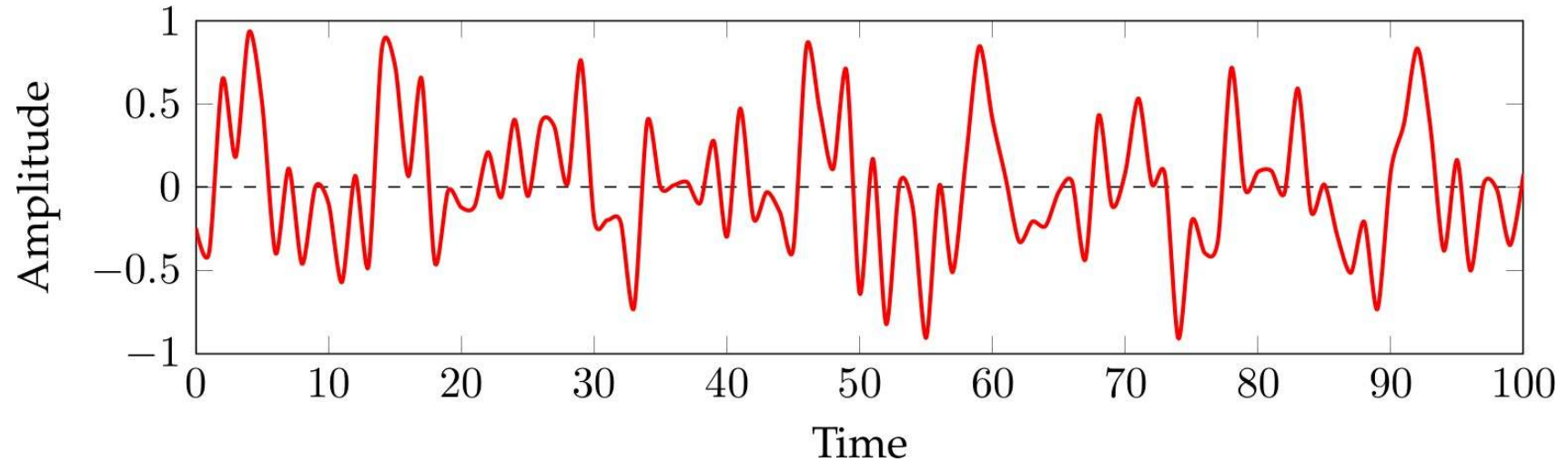
- ▶ Signal processing
- ▶ Graph theory
- ▶ **Graph signal processing**
- ▶ Focus :
Graph convolutional networks



Graph signal processing

└ From signals to signals on graphs

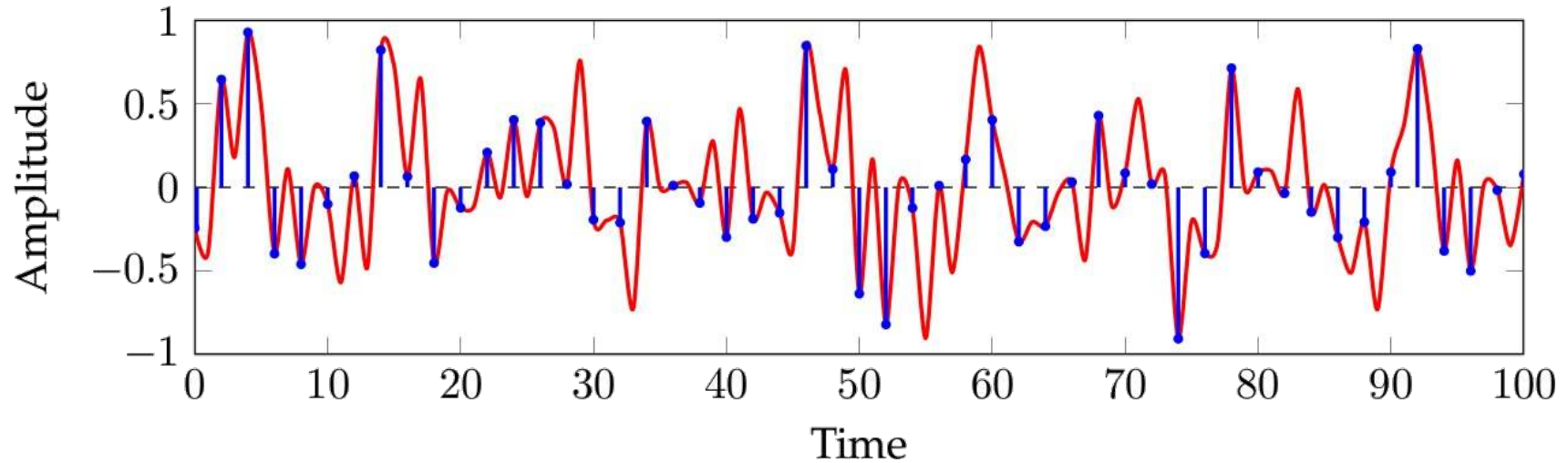
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ From signals to signals on graphs

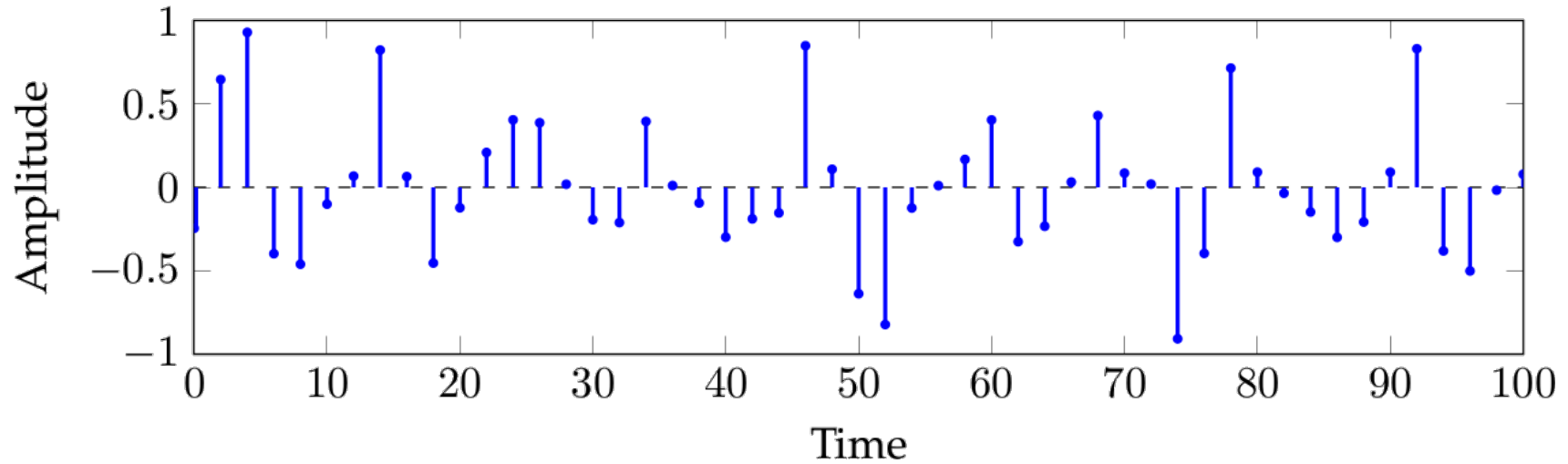
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ From signals to signals on graphs

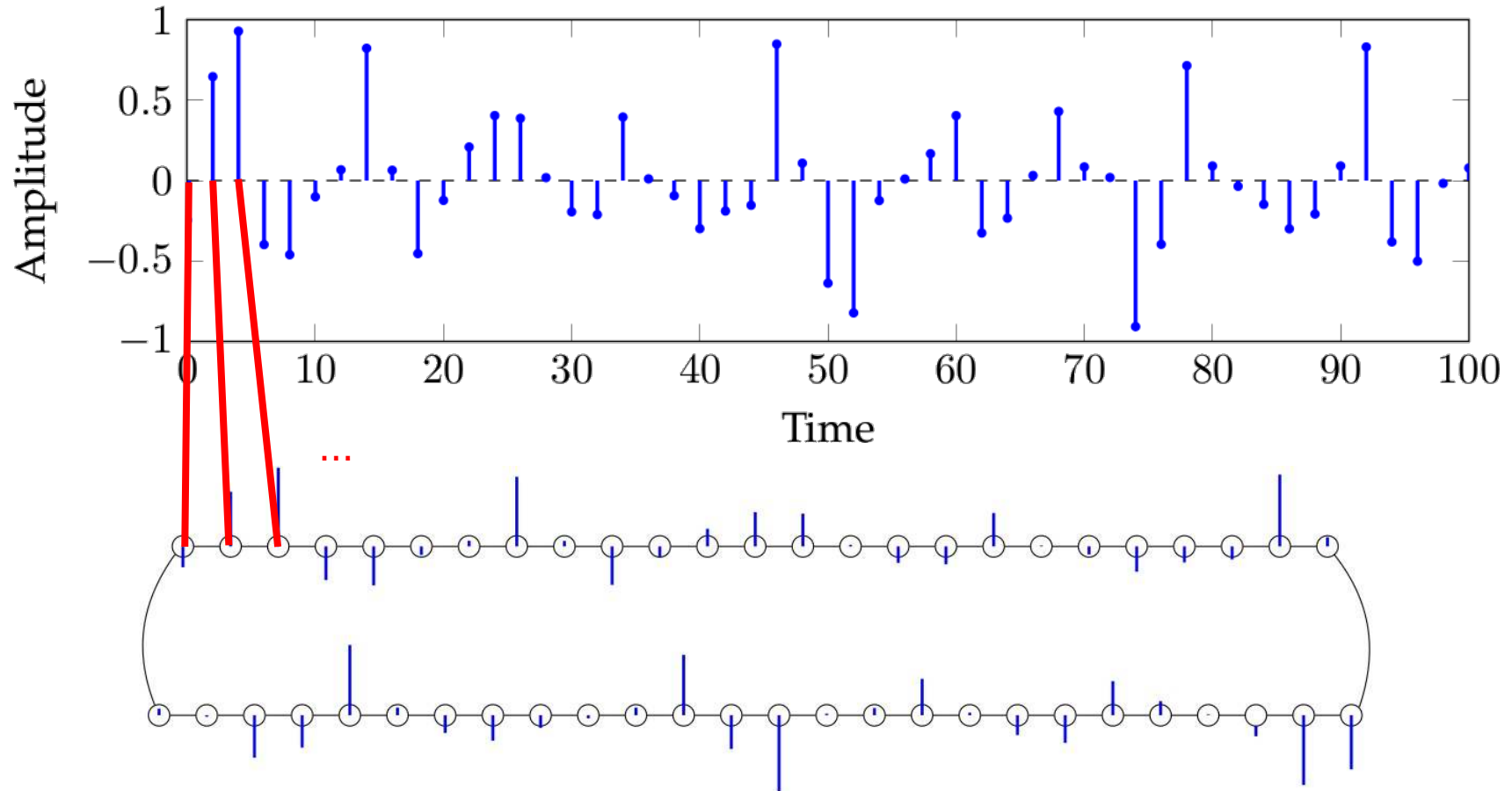
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ From signals to signals on graphs

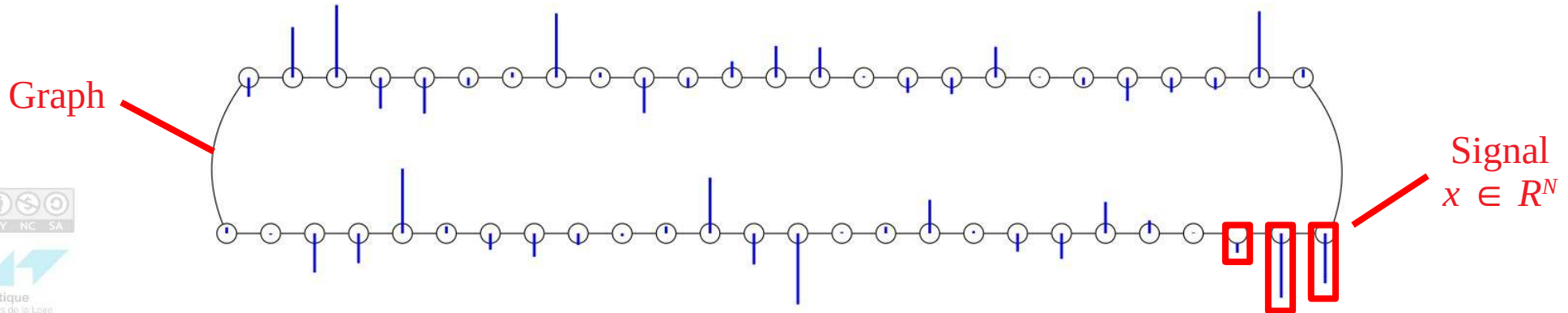
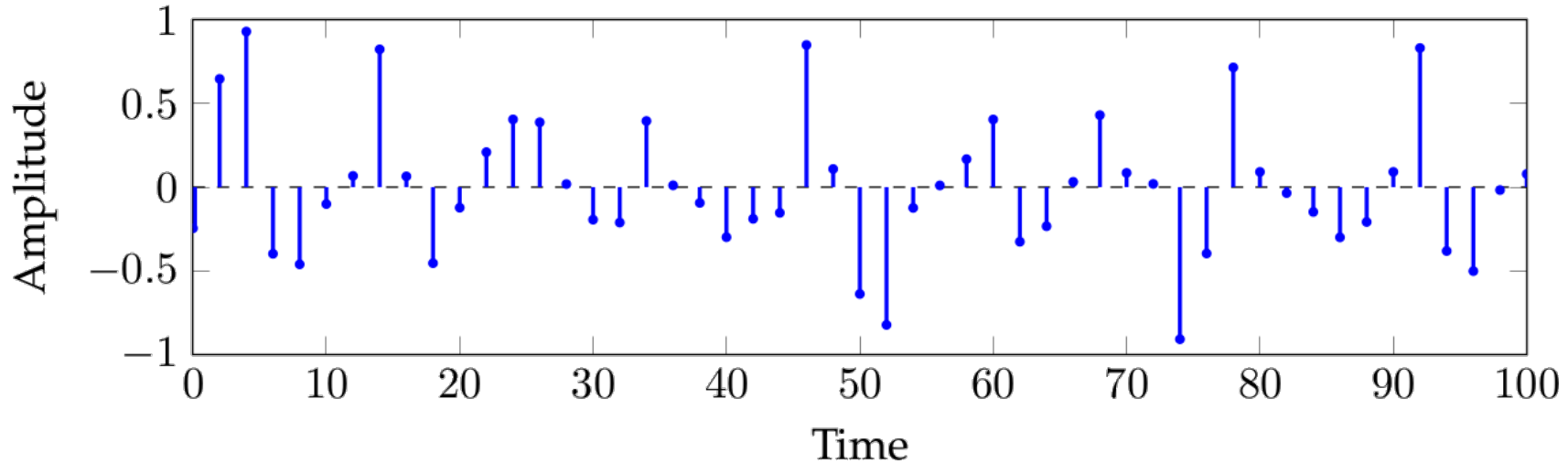
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ From signals to signals on graphs

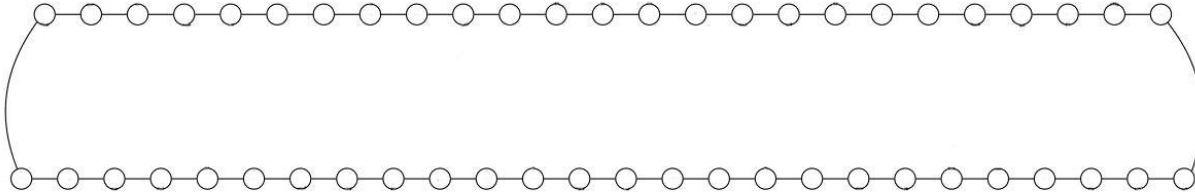
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Linking the graph with Fourier transform

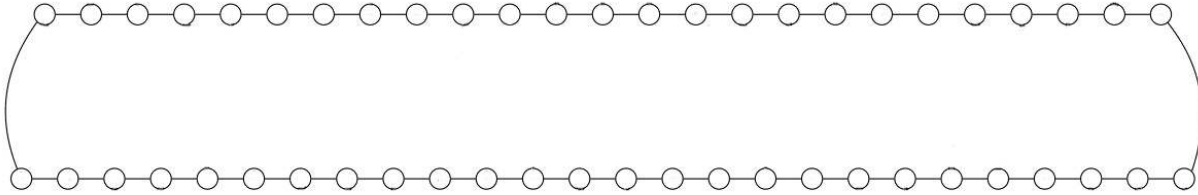
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Linking the graph with Fourier transform

<https://tel.archives-ouvertes.fr/tel-01708824>



Compute Laplacian L



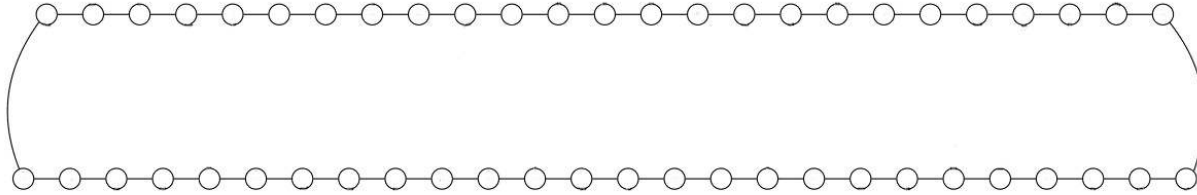
Diagonalize
 $L = X\Lambda X^T$



Graph signal processing

└ Linking the graph with Fourier transform

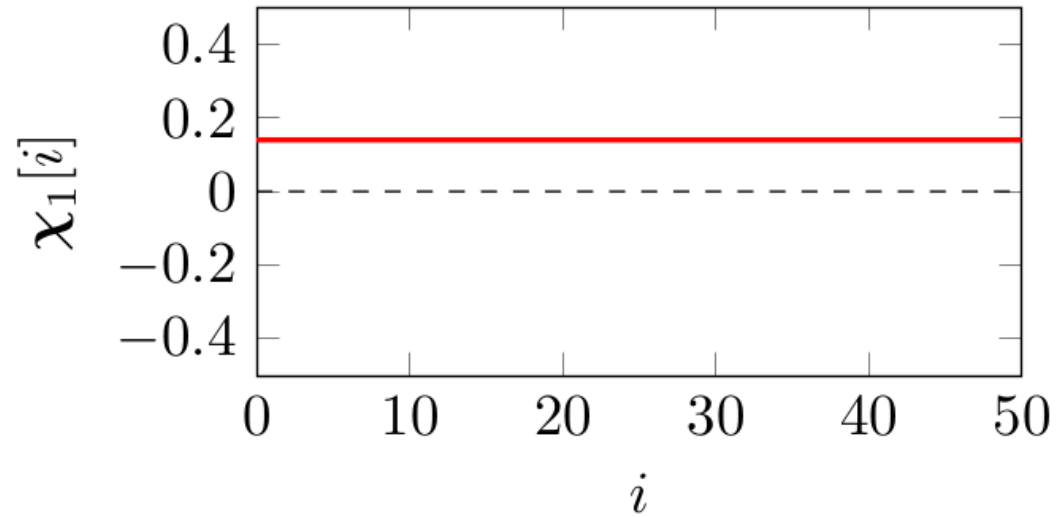
<https://tel.archives-ouvertes.fr/tel-01708824>



Compute Laplacian L



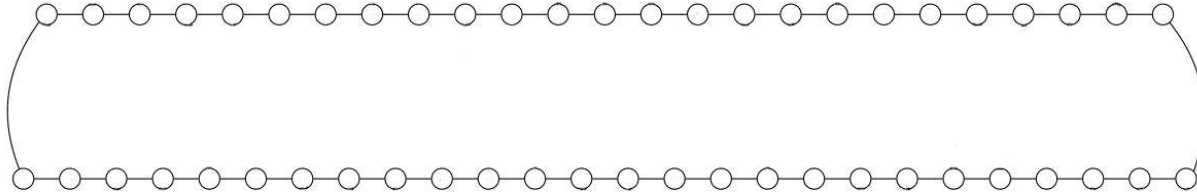
Diagonalize
 $L = X\Lambda X^T$



Graph signal processing

└ Linking the graph with Fourier transform

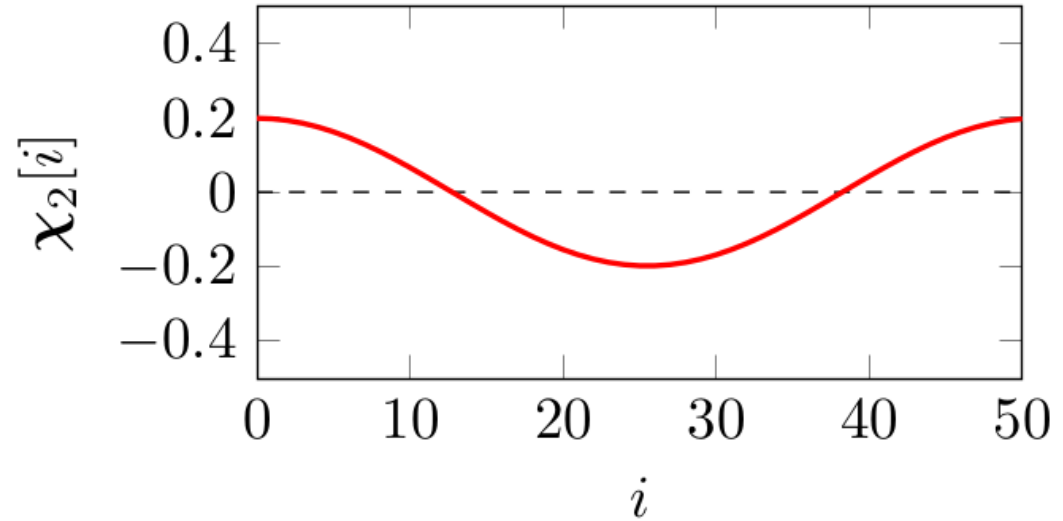
<https://tel.archives-ouvertes.fr/tel-01708824>



Compute Laplacian L



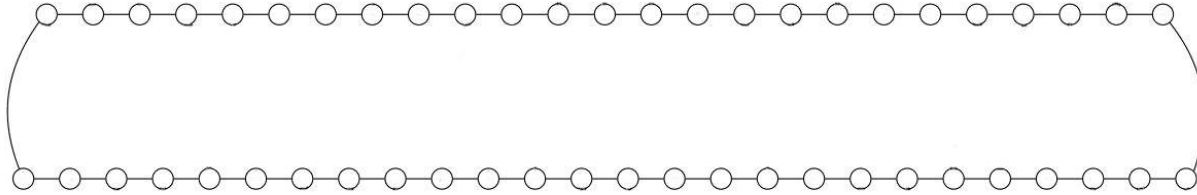
Diagonalize
 $L = X\Lambda X^T$



Graph signal processing

└ Linking the graph with Fourier transform

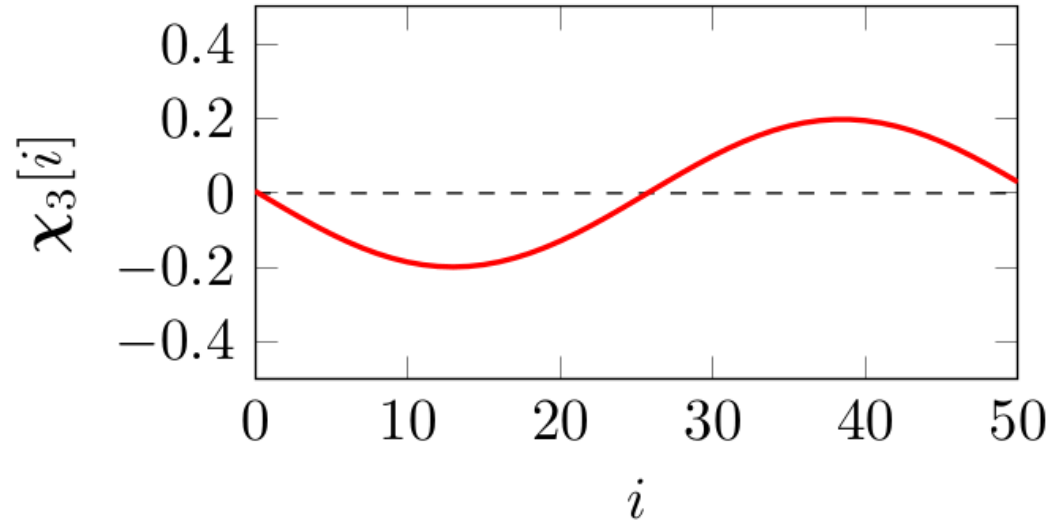
<https://tel.archives-ouvertes.fr/tel-01708824>



Compute Laplacian L



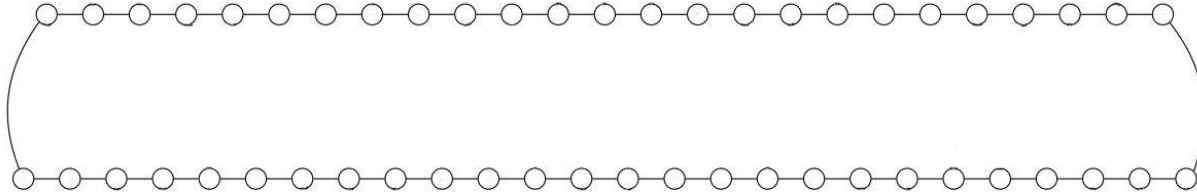
Diagonalize
 $L = X\Lambda X^T$



Graph signal processing

└ Linking the graph with Fourier transform

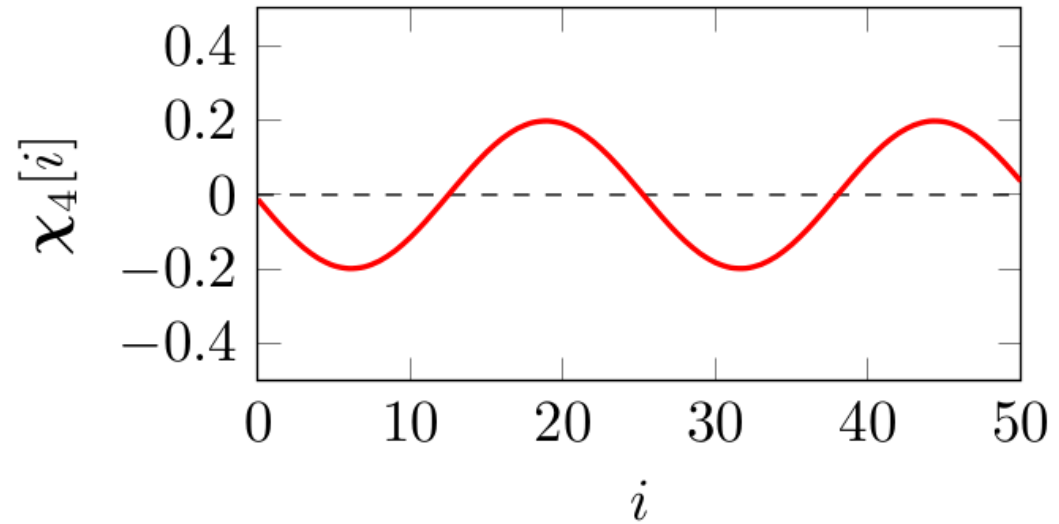
<https://tel.archives-ouvertes.fr/tel-01708824>



Compute Laplacian L



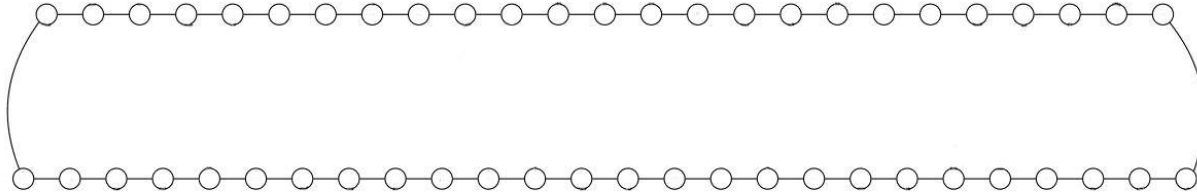
Diagonalize
 $L = X\Lambda X^T$



Graph signal processing

└ Linking the graph with Fourier transform

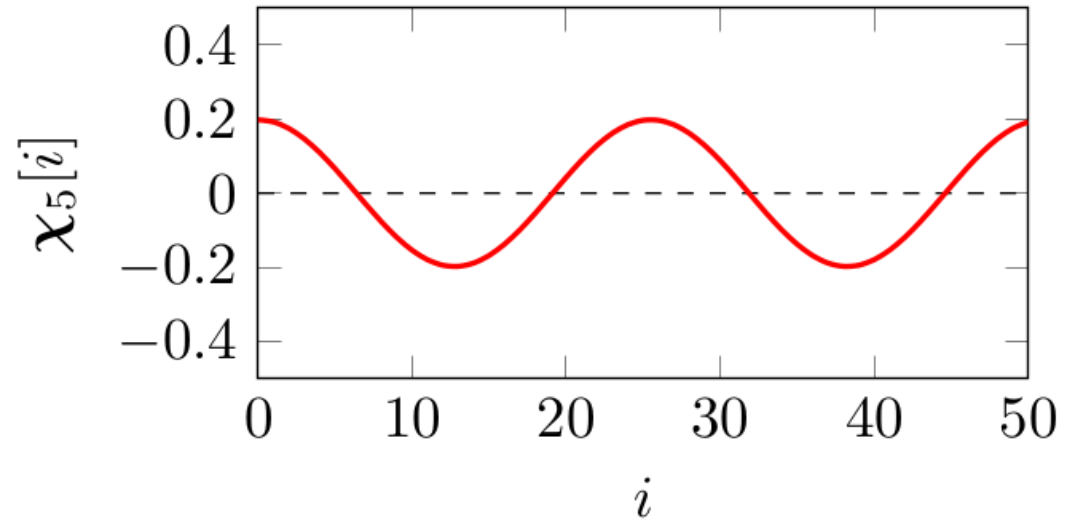
<https://tel.archives-ouvertes.fr/tel-01708824>



Compute Laplacian L



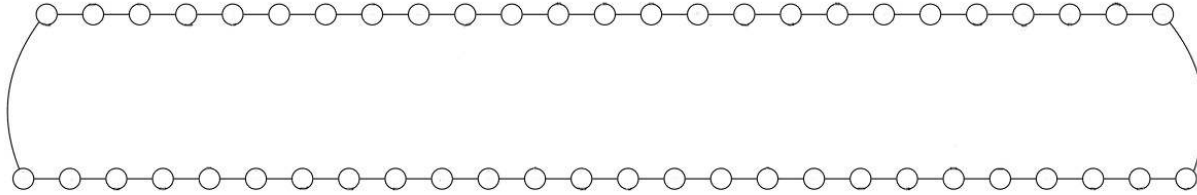
Diagonalize
 $L = X\Lambda X^T$



Graph signal processing

└ Linking the graph with Fourier transform

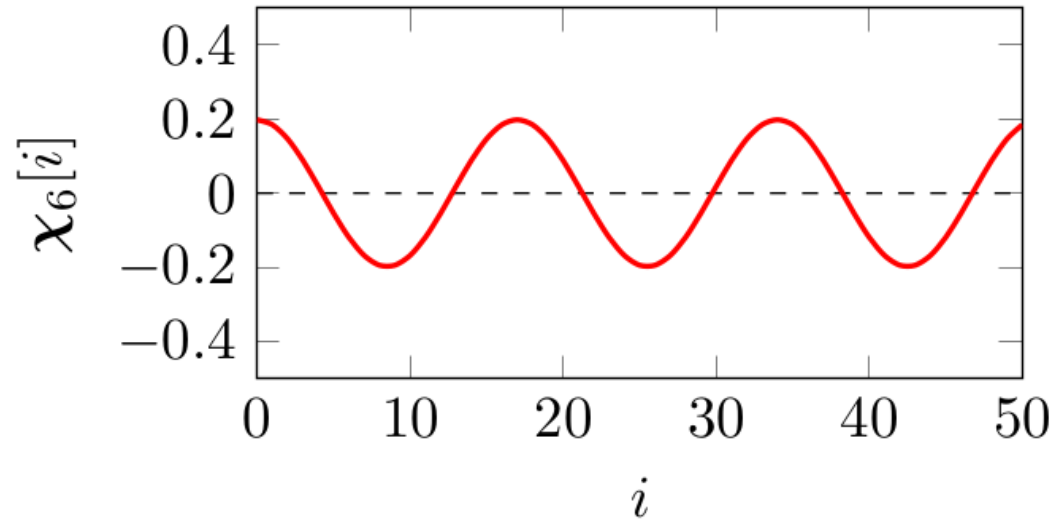
<https://tel.archives-ouvertes.fr/tel-01708824>



Compute Laplacian L



Diagonalize
 $L = X\Lambda X^T$



Graph signal processing

└ Linking the graph with Fourier transform

1708824

Definition 43: Graph Fourier transform

The *graph Fourier transform* $\hat{\mathbf{x}} = \text{GFT}(\mathbf{x})$ of a signal $\mathbf{x}$ on a graph is:

$$\hat{\mathbf{x}} = \mathcal{X}^\top \mathbf{x} . \quad (32)$$

Definition 44: Inverse graph Fourier transform

The *inverse graph Fourier transform* $\mathbf{x} = \text{GFT}^{-1}(\hat{\mathbf{x}})$ of the spectrum $\hat{\mathbf{x}}$ of a signal on a graph is:

$$\mathbf{x} = \mathcal{X} \hat{\mathbf{x}} . \quad (33)$$

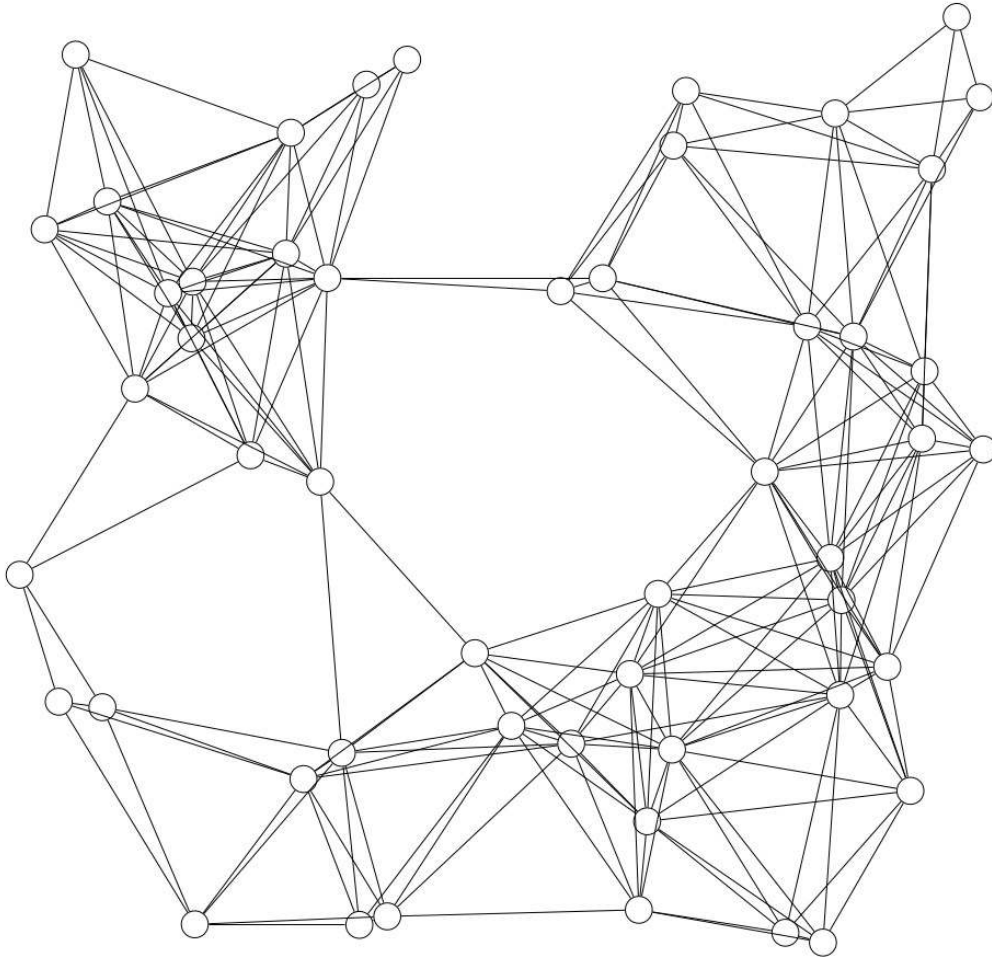
Note that orthonormality of the eigenvectors makes this function a valid inverse of the graph Fourier transform:

$$\text{GFT}^{-1}(\text{GFT}(\mathbf{x})) = \mathcal{X} \mathcal{X}^\top \mathbf{x} = \mathbf{x} . \quad (34)$$

Graph signal processing

└ Why is it interesting?

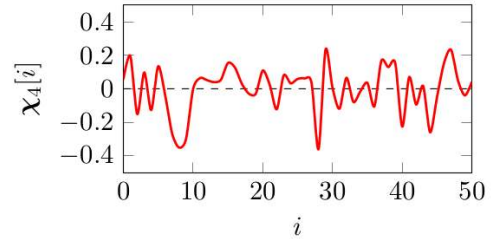
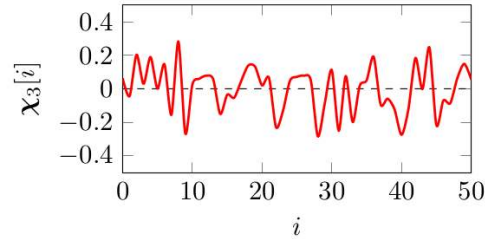
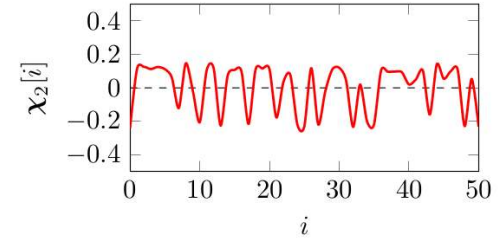
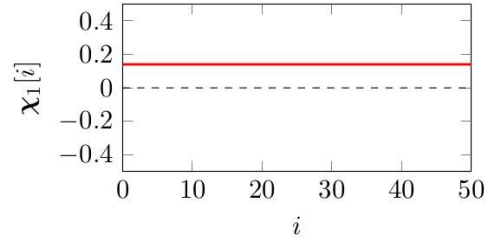
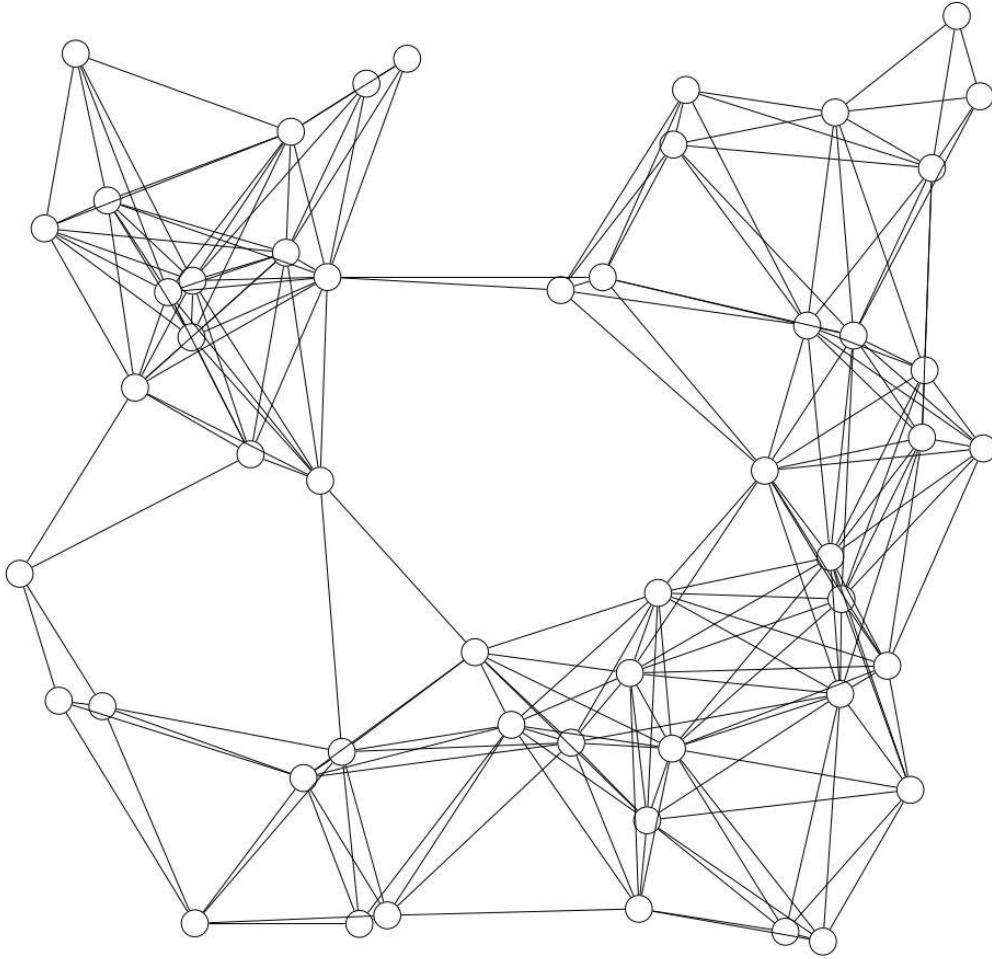
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Why is it interesting?

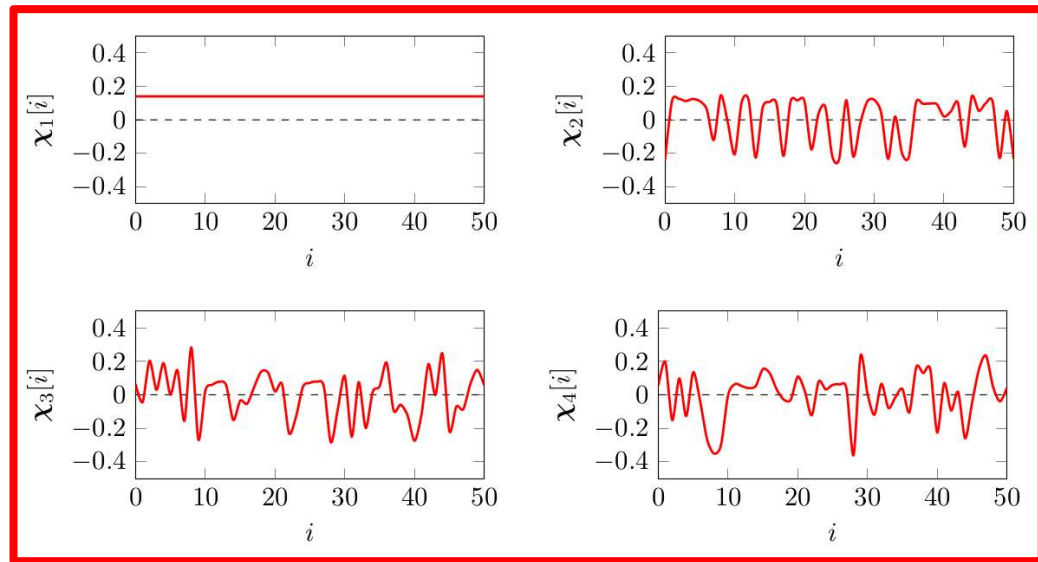
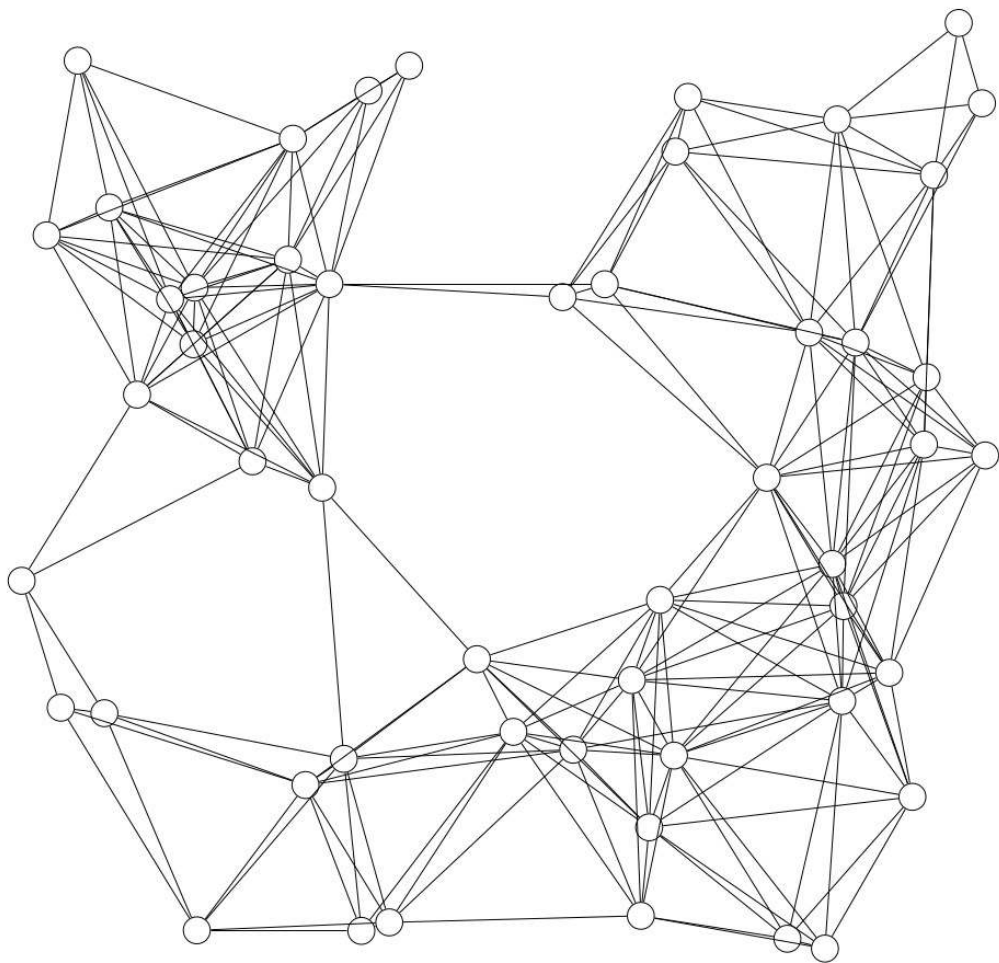
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Why is it interesting?

<https://tel.archives-ouvertes.fr/tel-01708824>

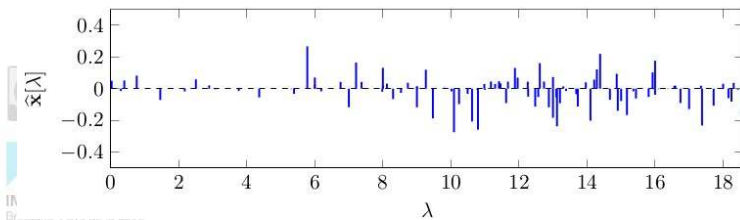
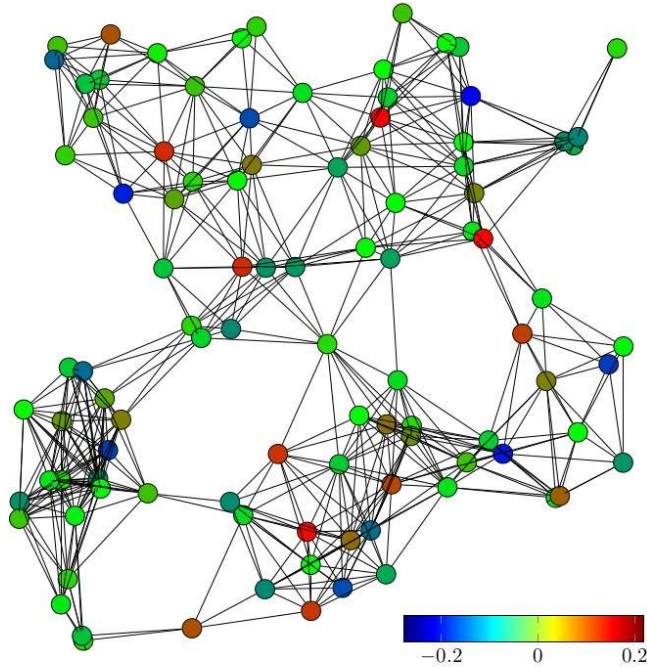


These define a Fourier transform
for this graph!

Graph signal processing

└ Convolution of signals on graphs

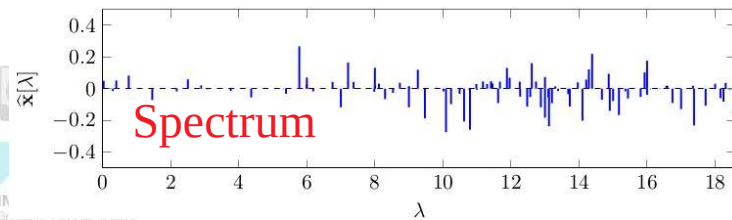
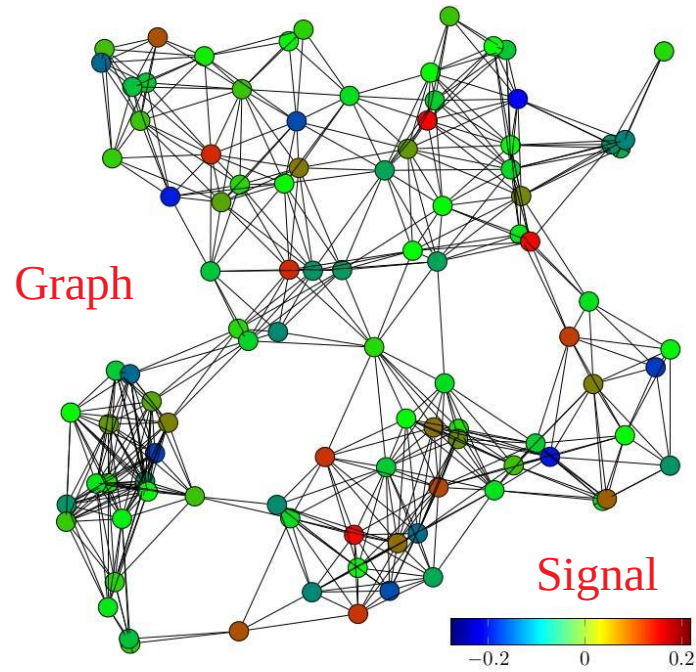
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Convolution of signals on graphs

<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Convolution of signals on graphs

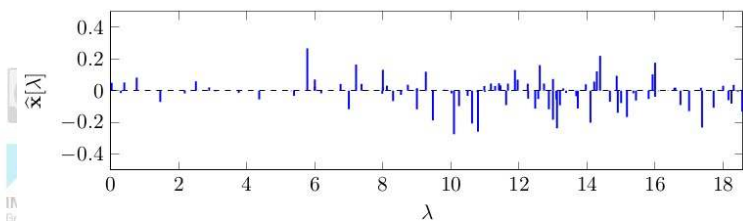
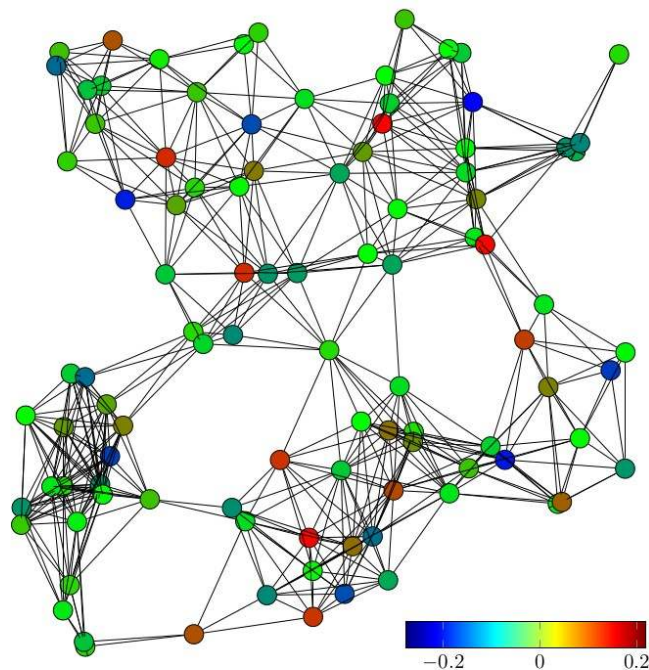
<https://tel.archives-ouvertes.fr/tel-01708824>

Definition 45: Convolution of signals on a graph

Convolution of two signals on a graph is performed in the spectral domain analogously to classical convolution. The corresponding signal in the time domain is then obtained by inverse graph Fourier transform:

$$\mathbf{x}_1 * \mathbf{x}_2 = \text{GFT}^{-1}(\text{GFT}(\mathbf{x}_1) \odot \text{GFT}(\mathbf{x}_2)) = \text{GFT}^{-1}(\widehat{\mathbf{x}}_1 \odot \widehat{\mathbf{x}}_2), \quad (35)$$

where $\odot$ is the *entrywise product* of vectors.



Graph signal processing

Convolution of signals on graphs

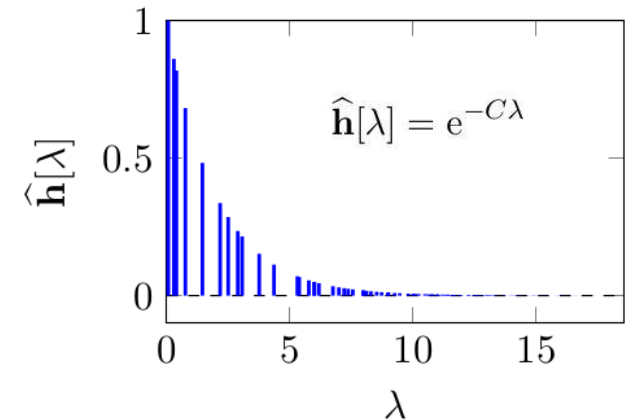
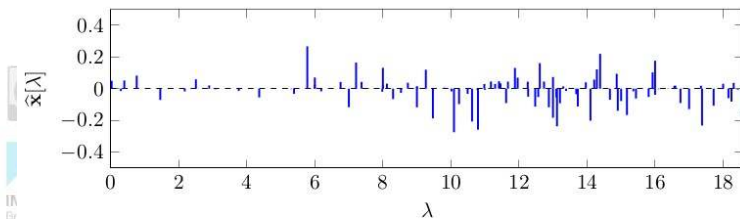
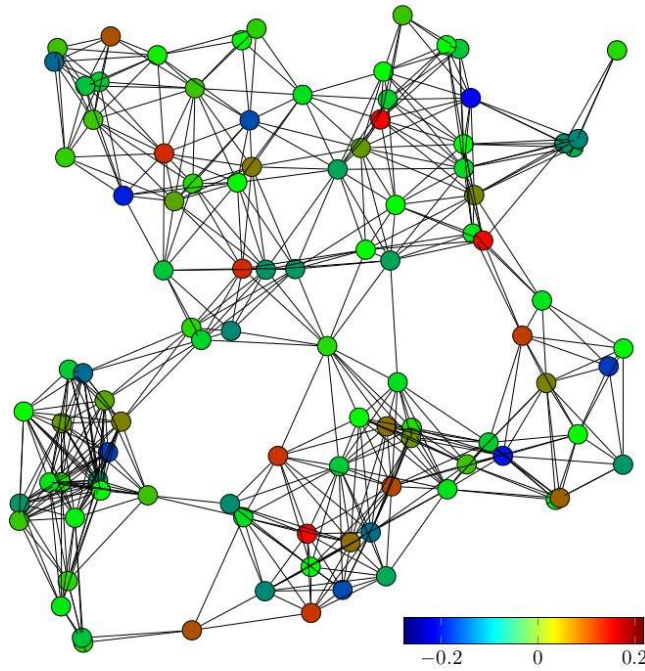
<https://tel.archives-ouvertes.fr/tel-01708824>

Definition 45: Convolution of signals on a graph

Convolution of two signals on a graph is performed in the spectral domain analogously to classical convolution. The corresponding signal in the time domain is then obtained by inverse graph Fourier transform:

$$\mathbf{x}_1 * \mathbf{x}_2 = \text{GFT}^{-1}(\text{GFT}(\mathbf{x}_1) \odot \text{GFT}(\mathbf{x}_2)) = \text{GFT}^{-1}(\widehat{\mathbf{x}}_1 \odot \widehat{\mathbf{x}}_2), \quad (35)$$

where $\odot$ is the *entrywise product* of vectors.



Graph signal processing

Convolution of signals on graphs

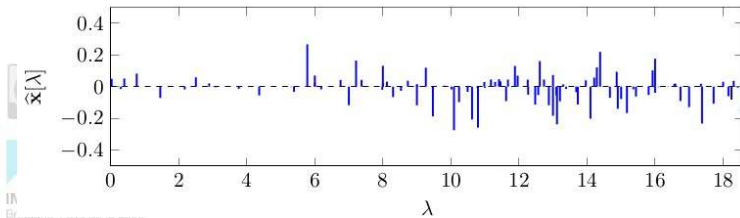
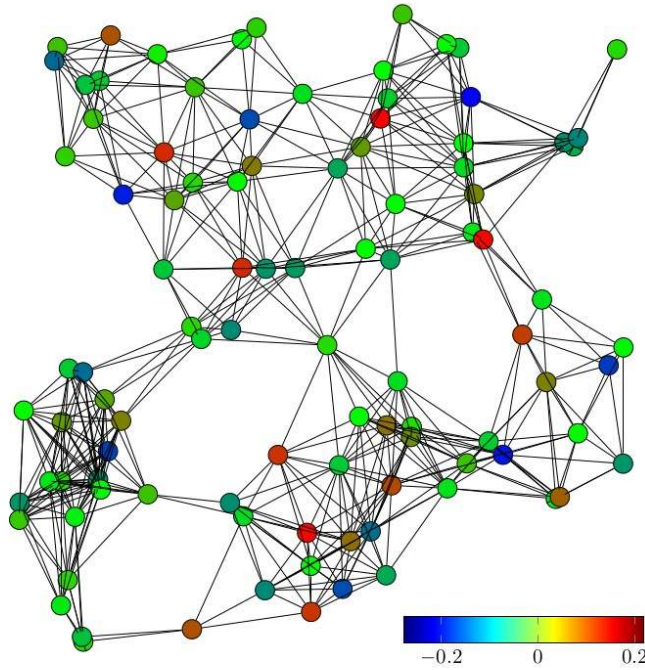
<https://tel.archives-ouvertes.fr/tel-01708824>

Definition 45: Convolution of signals on a graph

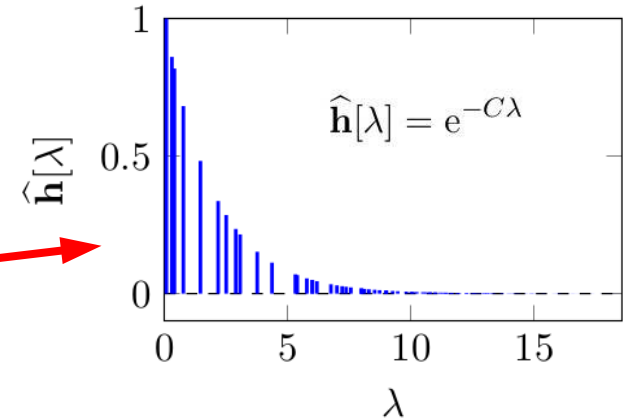
Convolution of two signals on a graph is performed in the spectral domain analogously to classical convolution. The corresponding signal in the time domain is then obtained by inverse graph Fourier transform:

$$\mathbf{x}_1 * \mathbf{x}_2 = \text{GFT}^{-1}(\text{GFT}(\mathbf{x}_1) \odot \text{GFT}(\mathbf{x}_2)) = \text{GFT}^{-1}(\widehat{\mathbf{x}}_1 \odot \widehat{\mathbf{x}}_2), \quad (35)$$

where $\odot$ is the *entrywise product* of vectors.



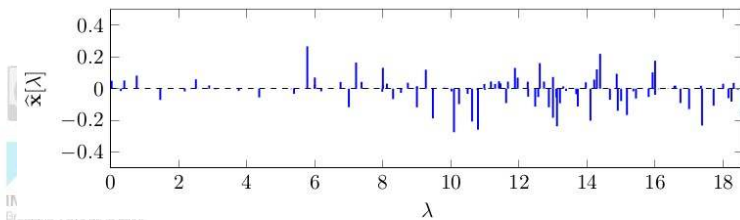
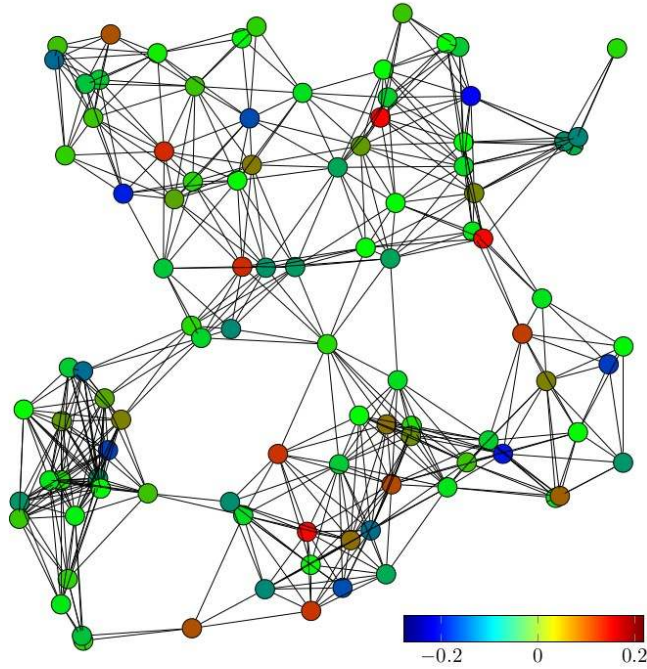
Guess what happens



Graph signal processing

└ Convolution of signals on graphs

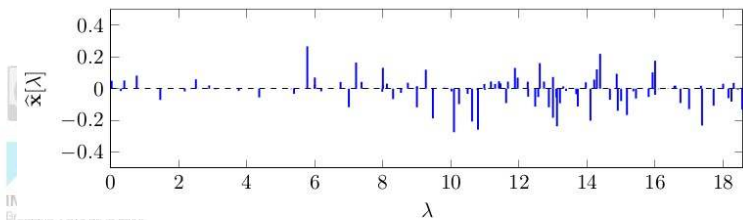
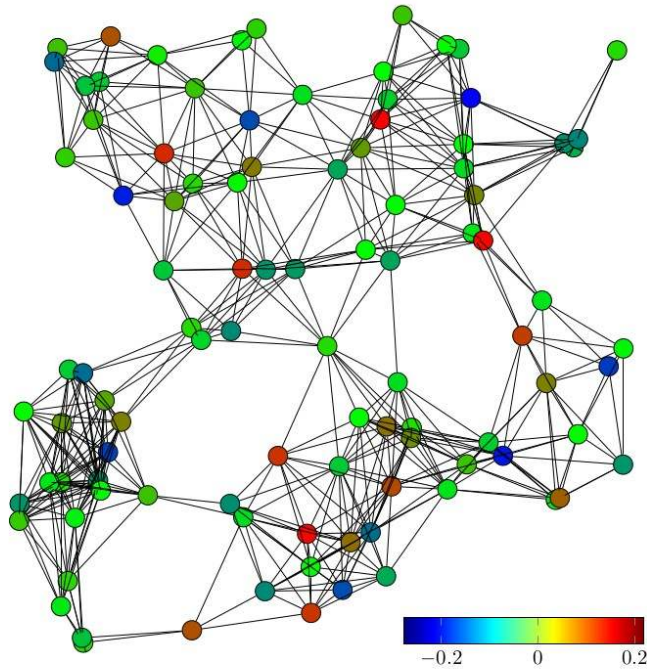
<https://tel.archives-ouvertes.fr/tel-01708824>



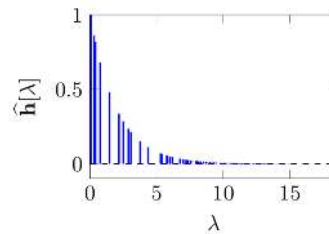
Graph signal processing

└ Convolution of signals on graphs

<https://tel.archives-ouvertes.fr/tel-01708824>

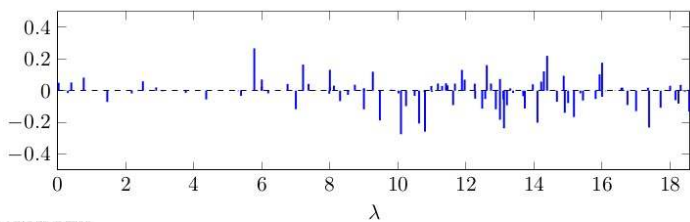
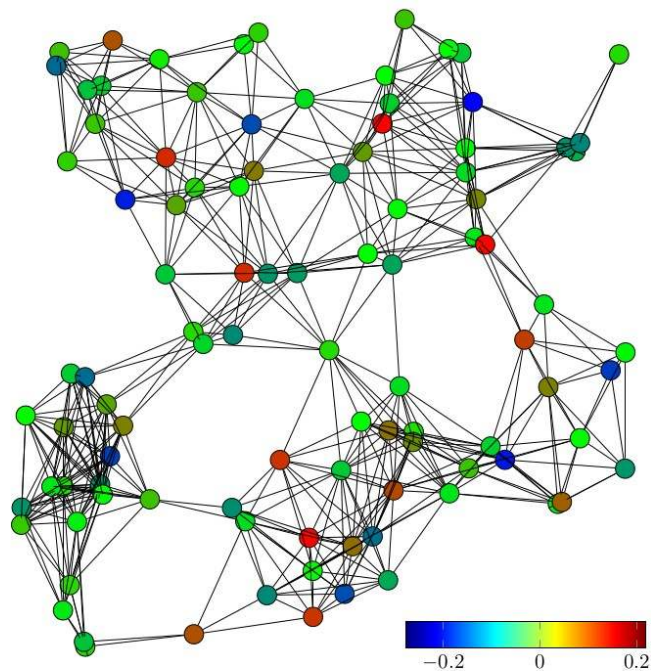


*

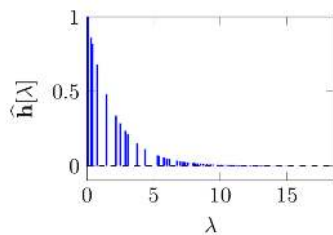


Graph signal processing

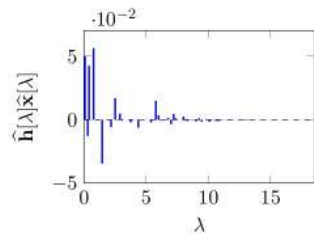
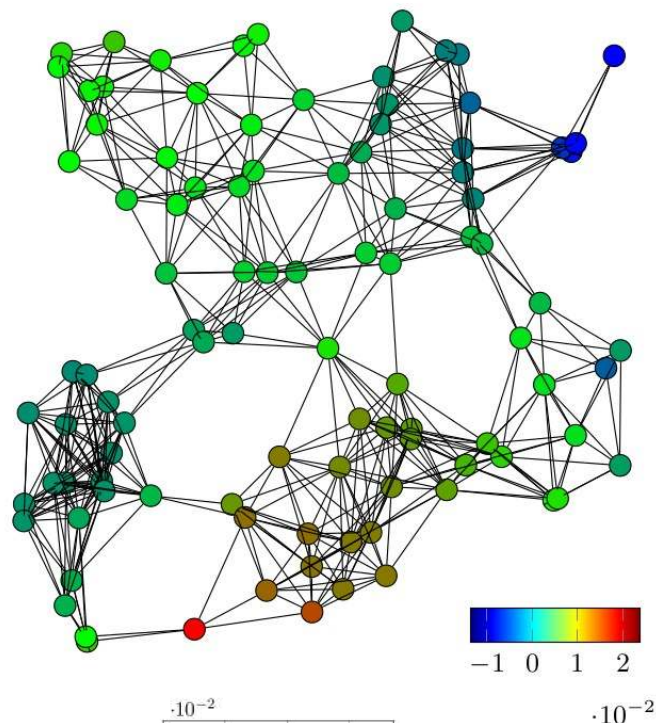
Convolution of signals on graphs



*

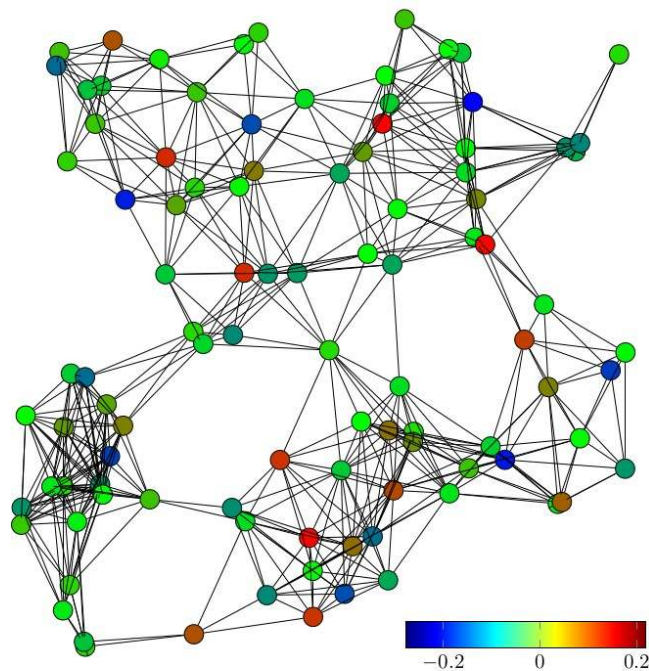


=

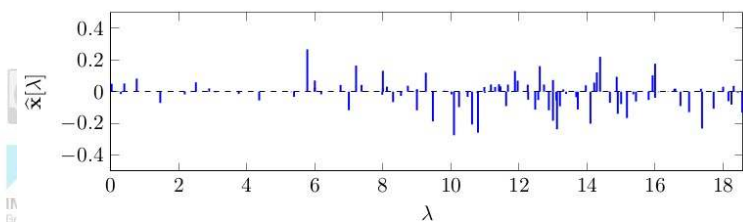
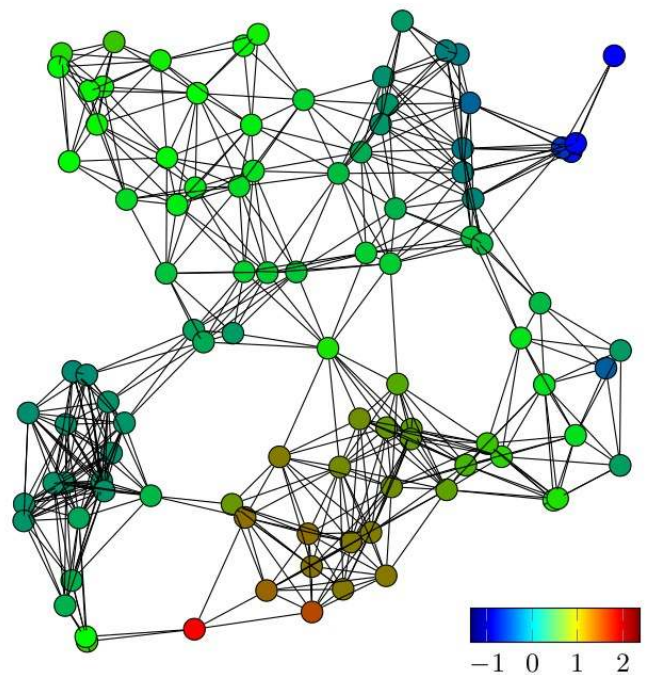


Graph signal processing

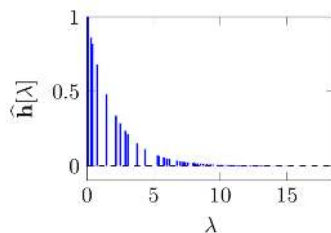
└ Convolution of signals on graphs



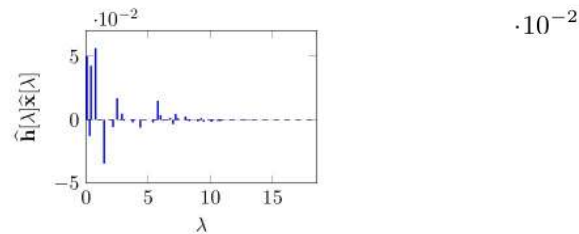
Smoothing!
(low-pass filter)



*



=



Graph signal processing

└ Translation of signals on graphs

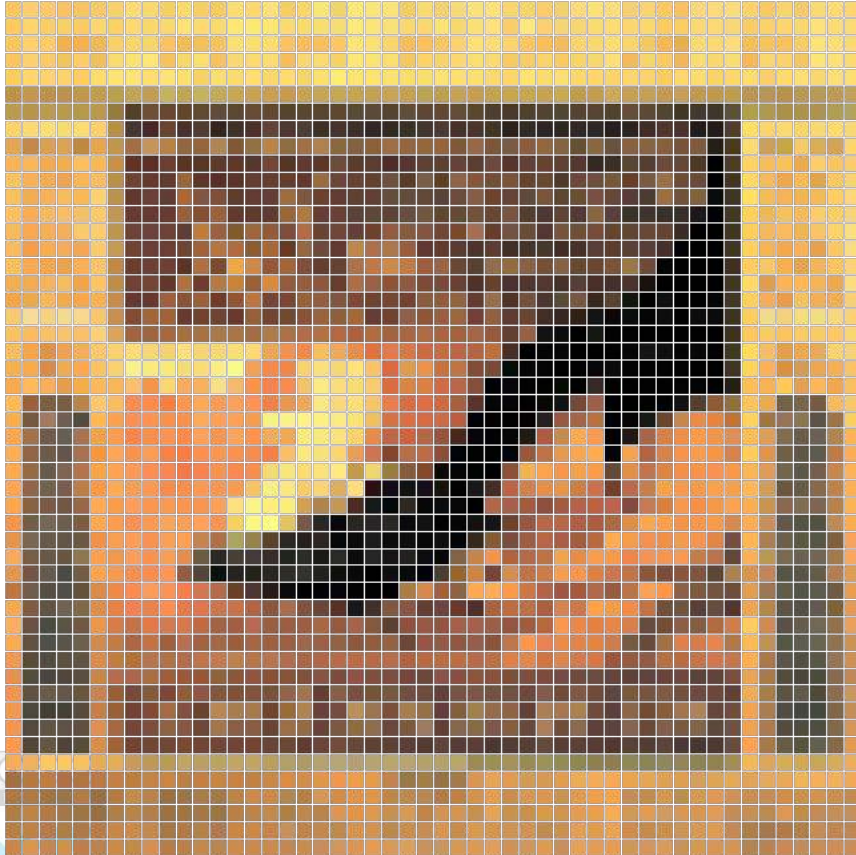
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Translation of signals on graphs

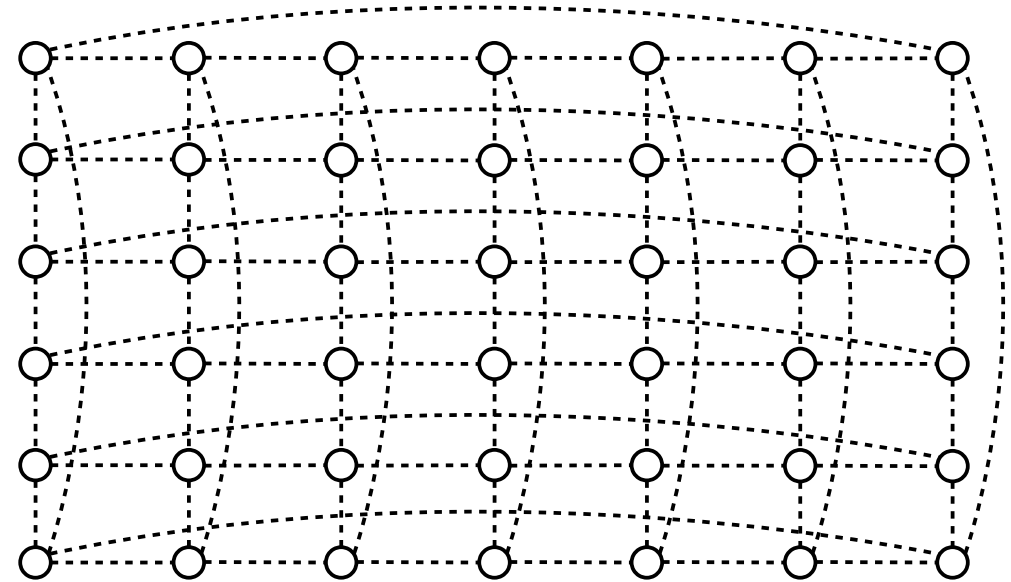
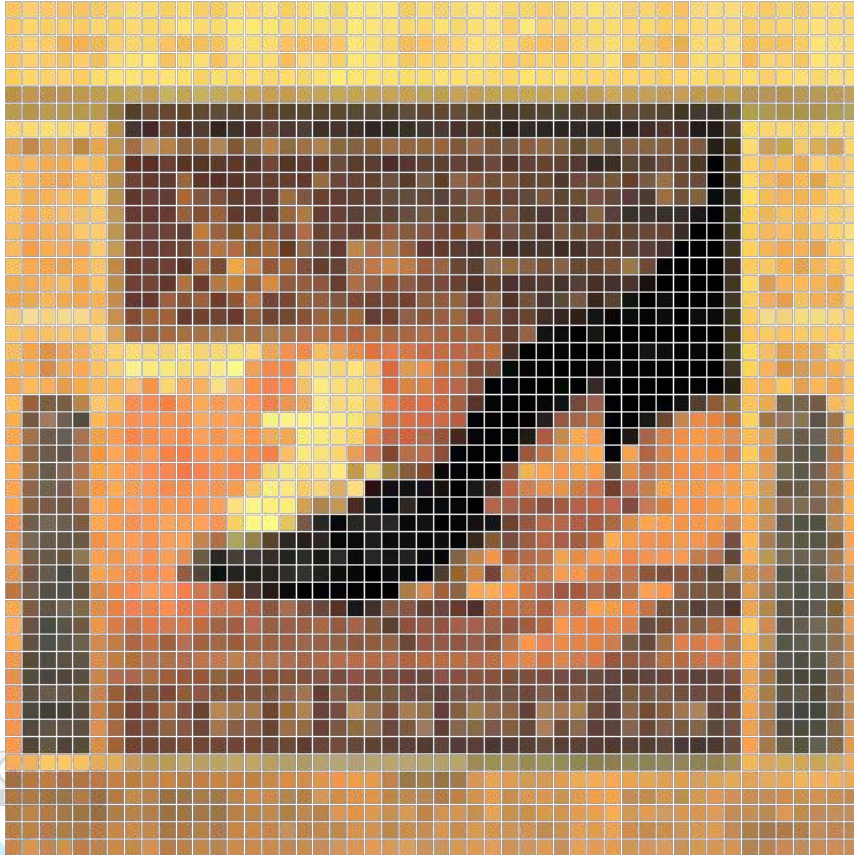
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

- Translation of signals on graphs

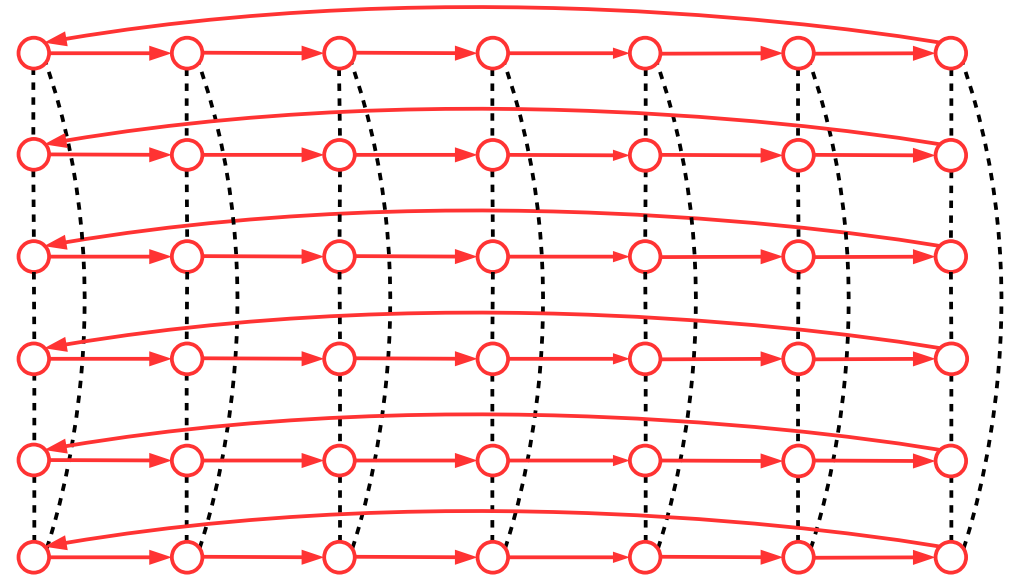
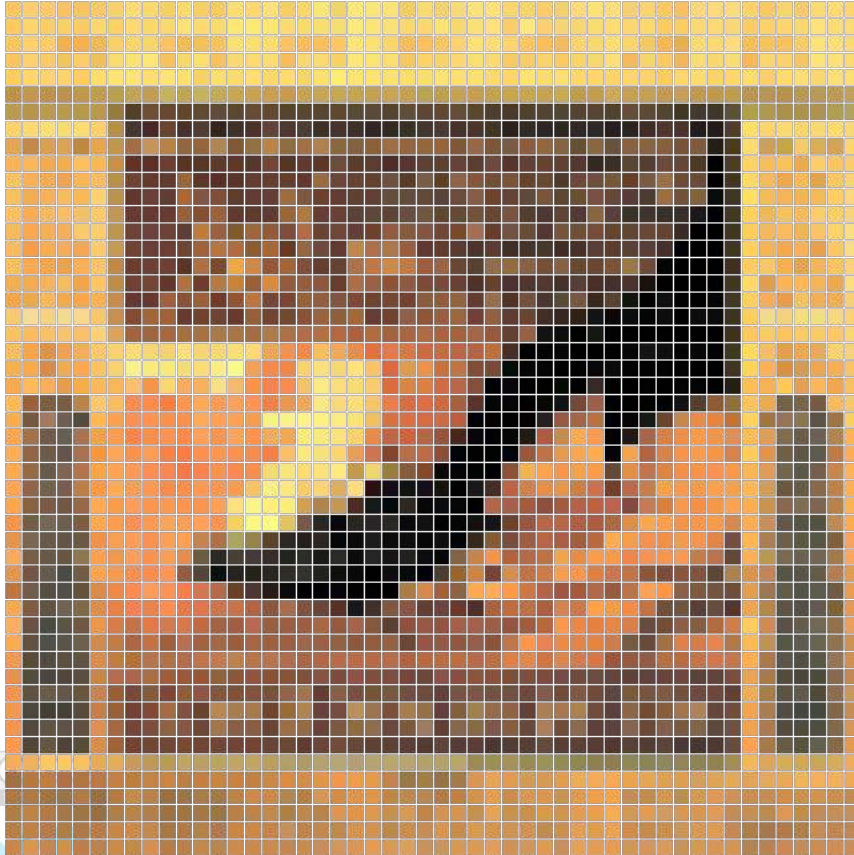
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Translation of signals on graphs

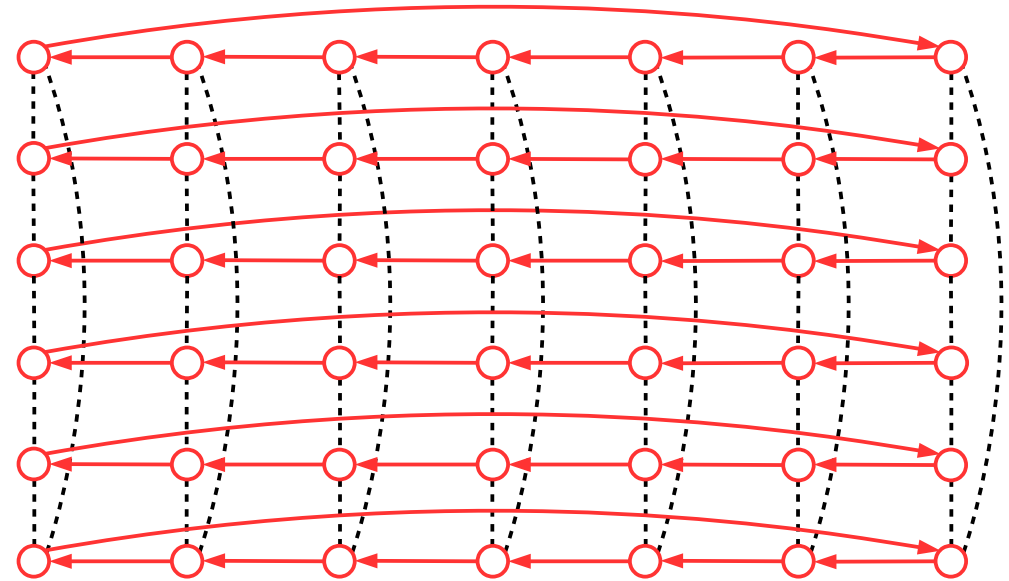
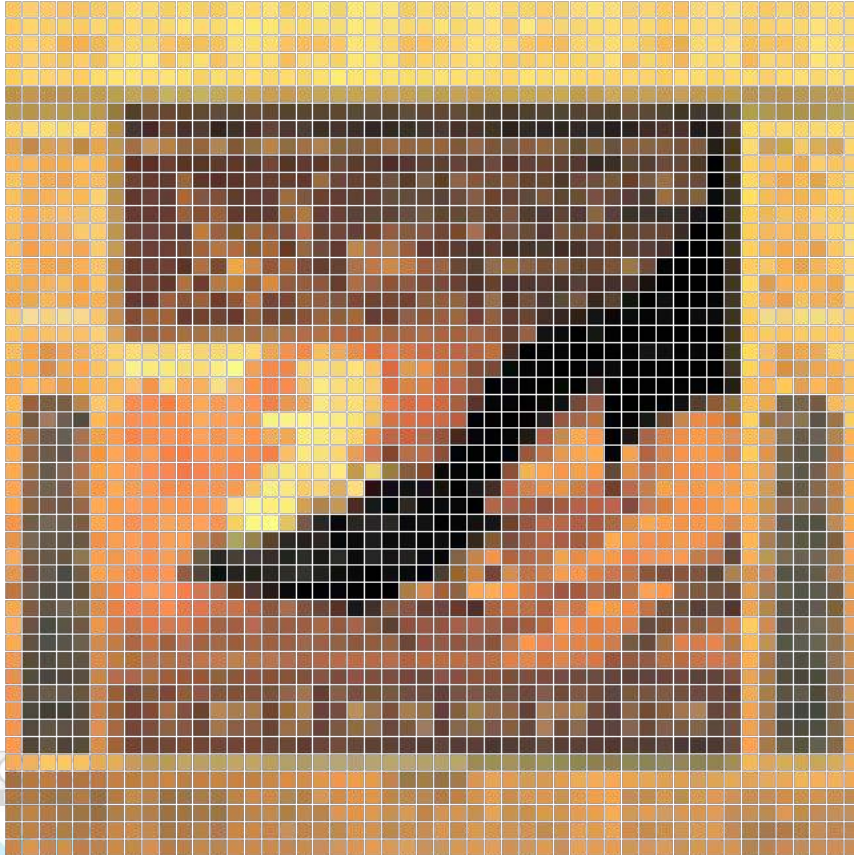
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

- Translation of signals on graphs

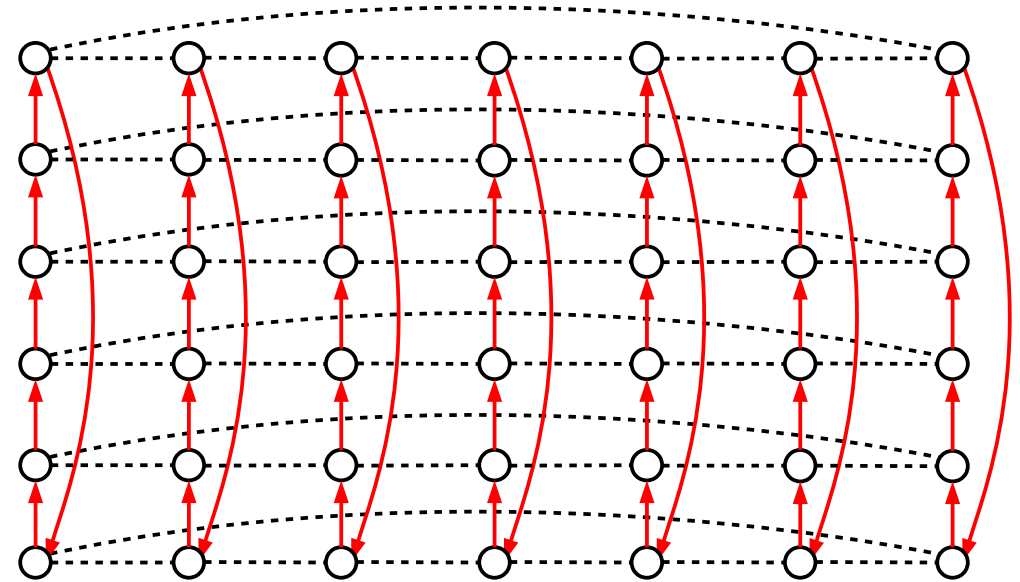
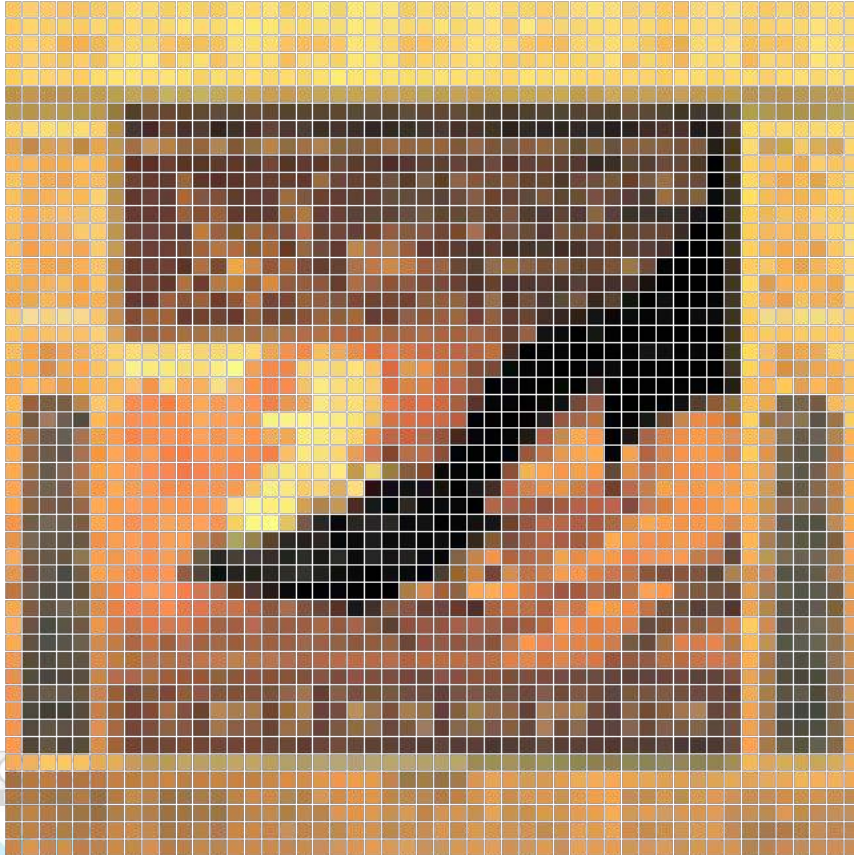
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

- Translation of signals on graphs

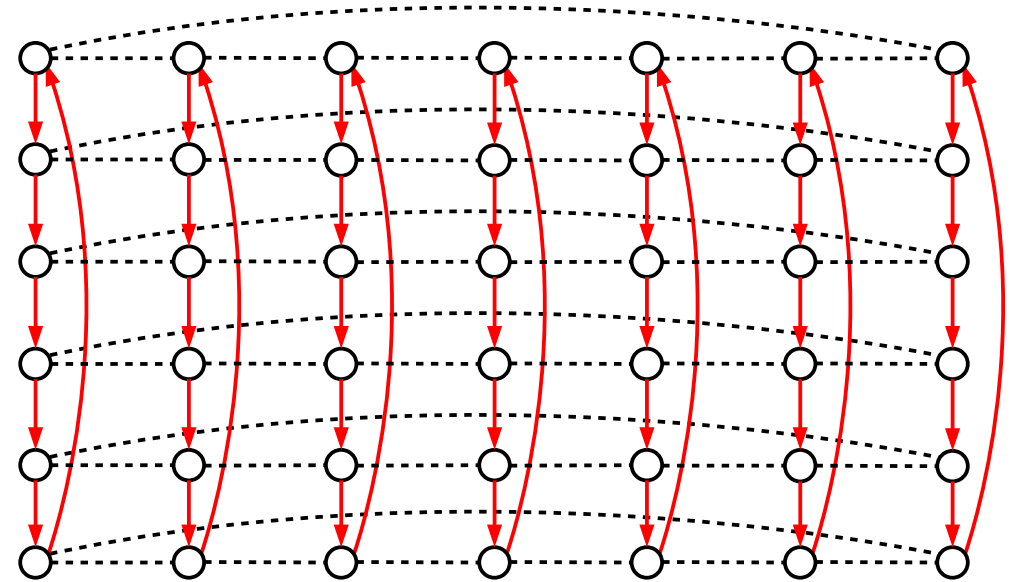
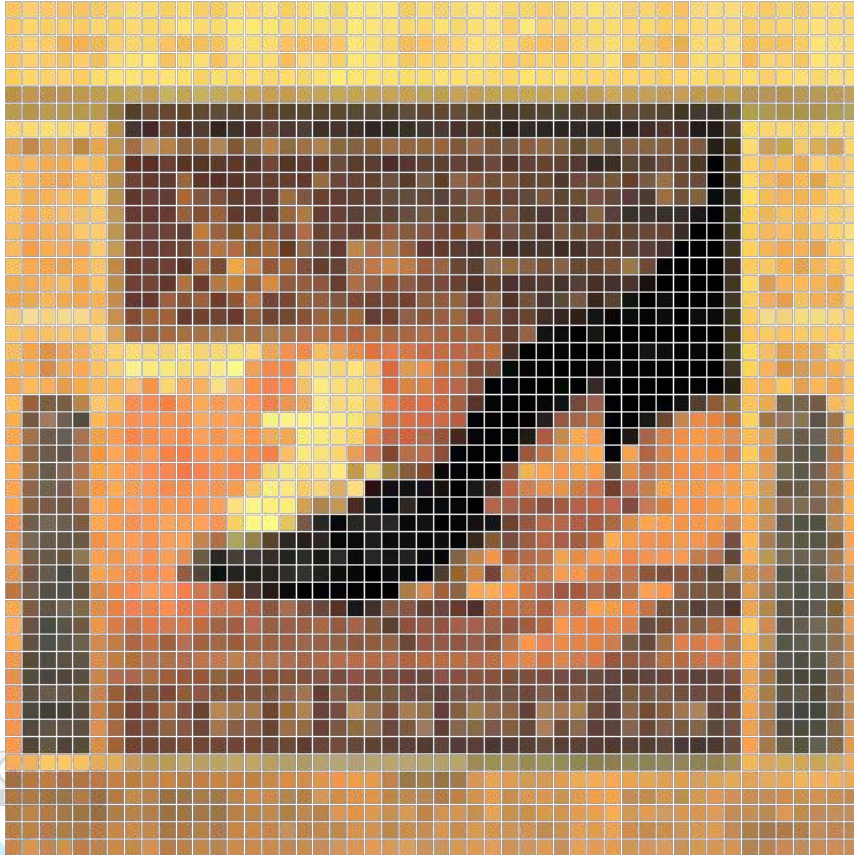
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

- Translation of signals on graphs

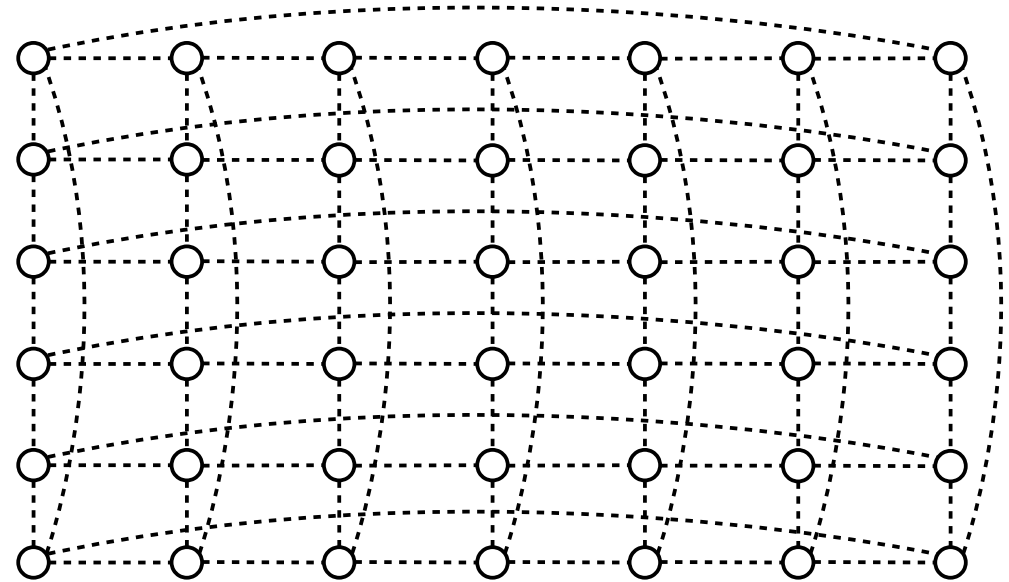
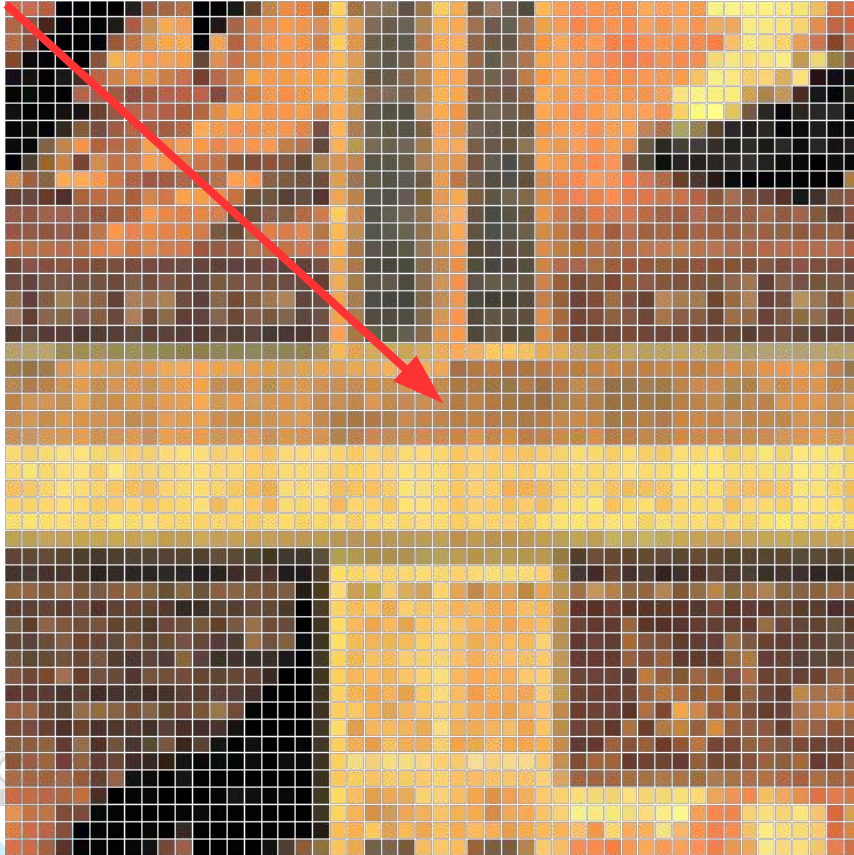
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Translation of signals on graphs

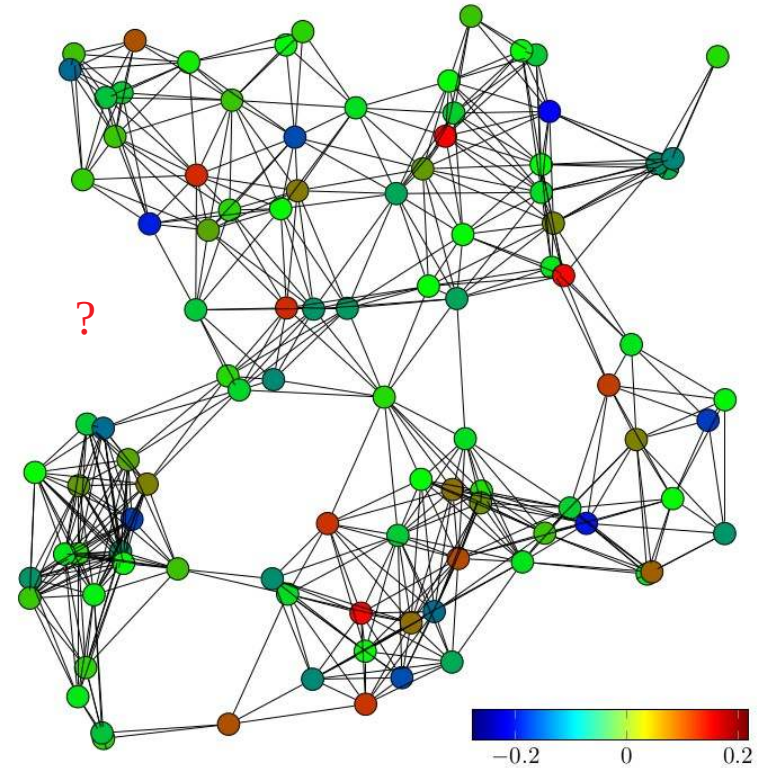
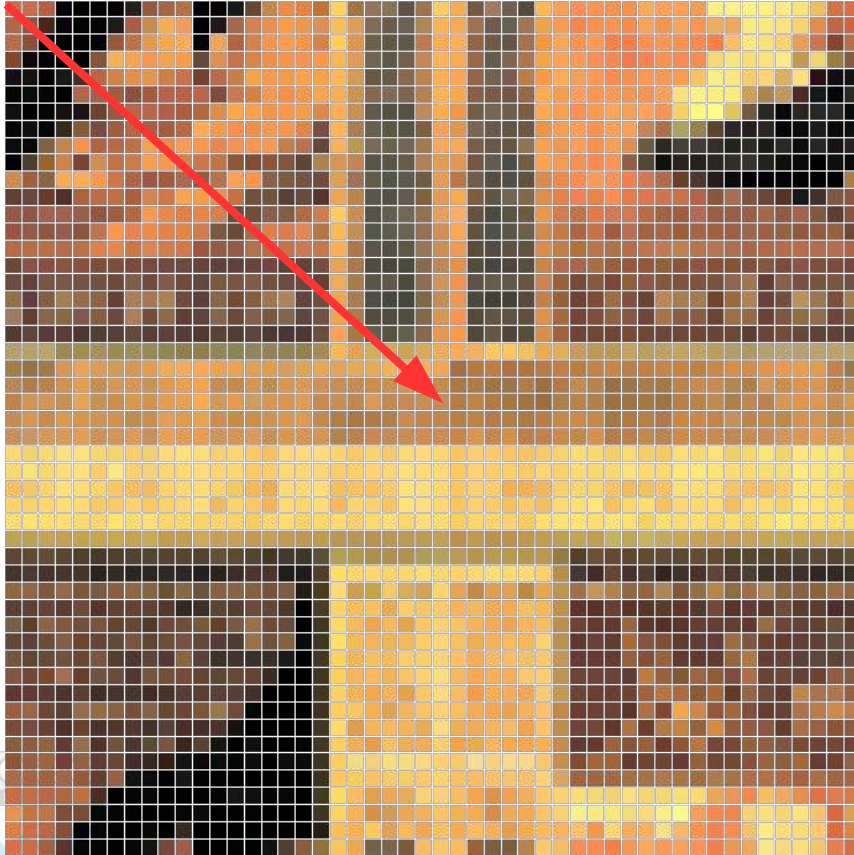
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Translation of signals on graphs

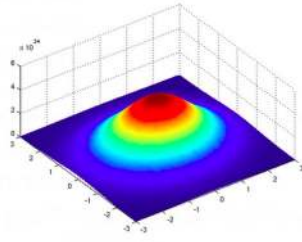
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing

└ Translation of signals on graphs

<https://tel.archives-ouvertes.fr/tel-01708824>
<https://en.wikipedia.org/wiki/Convolution>



In mathematics, **Schwartz space** $\mathcal{S}$ is the function space of all functions whose derivatives are rapidly decreasing. This space has the important property that the Fourier transform is an automorphism on this space. This property enables one, by duality, to define the Fourier transform for element



If f is a **Schwartz function**, then $\tau_x f$ is the convolution with a translated Dirac delta function $\tau_x f = f * \tau_x \delta$. So translation invariance of the convolution of Schwartz functions is a consequence of the associativity of convolution.

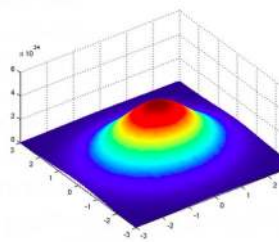


Graph signal processing

└ Translation of signals on graphs

<https://tel.archives-ouvertes.fr/tel-01708824>

<https://en.wikipedia.org/wiki/Convolution>



In mathematics, **Schwartz space** $\mathcal{S}$ is the function space of all functions whose derivatives are rapidly decreasing. This space has the important property that the Fourier transform is an automorphism on this space. This property enables one, by duality, to define the Fourier transform for elements

Definition 46: Dirac delta signal on a graph

A **Dirac delta signal** on a graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$ of N vertices at vertex $v_1 \in \mathcal{V}$ is a signal $\delta_{v_1} \in \mathbb{R}^N$ such that:

$$\delta_{v_1}[v_2] = \begin{cases} 1 & \text{if } v_2 = v_1 \\ 0 & \text{otherwise} \end{cases} . \quad (37)$$

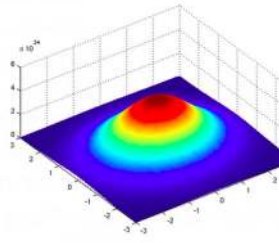
If f is a **Schwartz function**, then $\tau_x f$ is the convolution with a translated Dirac delta function $\tau_x f = f * \tau_x \delta$. So translation invariance of the convolution of Schwartz functions is a consequence of the associativity of convolution.



Graph signal processing

Translation of signals on graphs

<https://tel.archives-ouvertes.fr/tel-01708824>
<https://en.wikipedia.org/wiki/Convolution>



In mathematics, **Schwartz space** $\mathcal{S}$ is the function space of all functions whose derivatives are rapidly decreasing. This space has the important property that the Fourier transform is an automorphism on this space. This property enables one, by duality, to define the Fourier transform for elements

If f is a **Schwartz function**, then $\tau_x f$ is the convolution with a translation. Convolution of Schwartz functions is a consequence of the associativity of convolution.

Definition 46: Dirac delta signal on a graph

A **Dirac delta signal** on a graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$ of N vertices at vertex $v_1 \in \mathcal{V}$ is a signal $\delta_{v_1} \in \mathbb{R}^N$ such that:

$$\delta_{v_1}[v_2] = \begin{cases} 1 & \text{if } v_2 = v_1 \\ 0 & \text{otherwise} \end{cases} \quad (37)$$

Definition 47: Translation of a signal on a graph

Let us consider a graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$ of order N . **Translation** of a signal $\mathbf{x}$ to a particular vertex $v_1 \in \mathcal{V}$, as introduced by Shuman *et al.* in [Shu+13], is defined as follows:

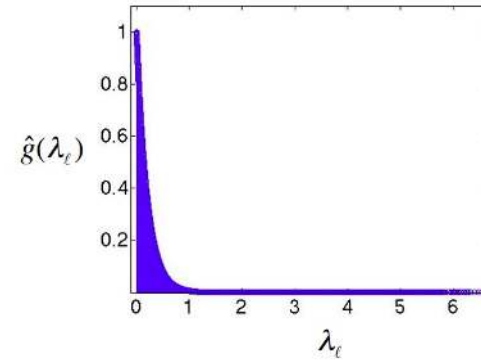
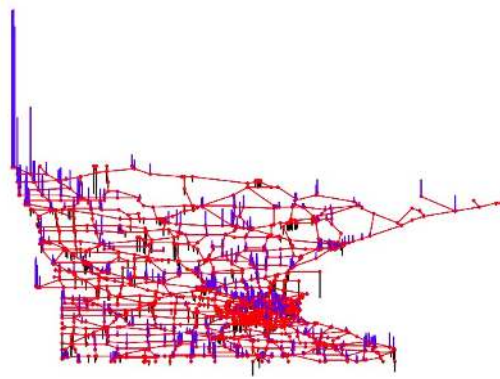
$$T_{v_1} \mathbf{x}[v_2] = \sqrt{N}(\mathbf{x} * \delta_{v_1})[v_2] \quad (38)$$

In this equation, $\sqrt{N}$ ensures preservation of the mean of the signal.

Graph signal processing

└ Translation of signals on graphs

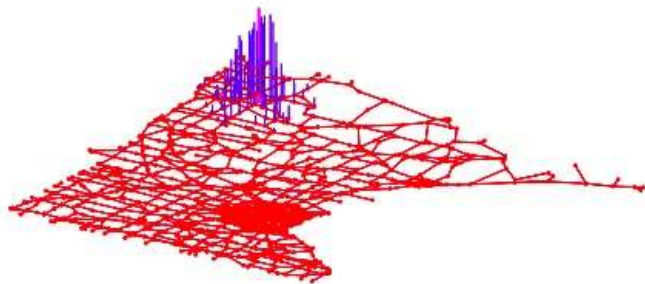
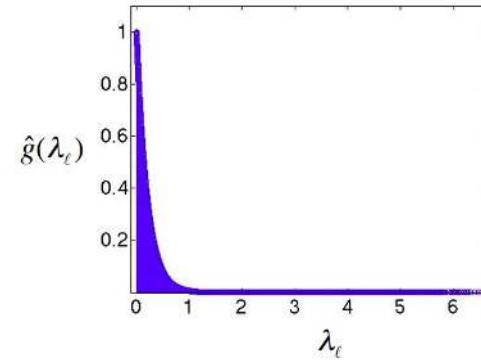
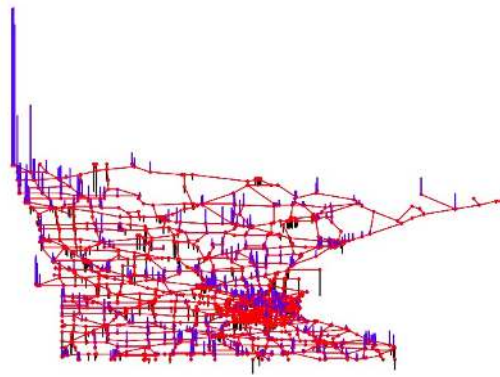
<https://arxiv.org/pdf/1211.0053.pdf>



Graph signal processing

└ Translation of signals on graphs

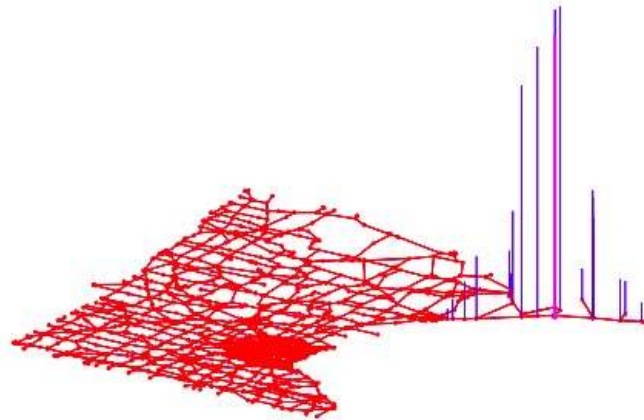
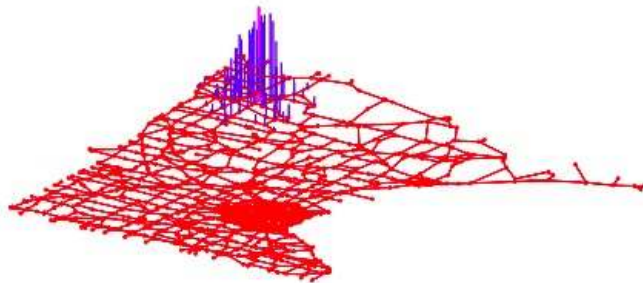
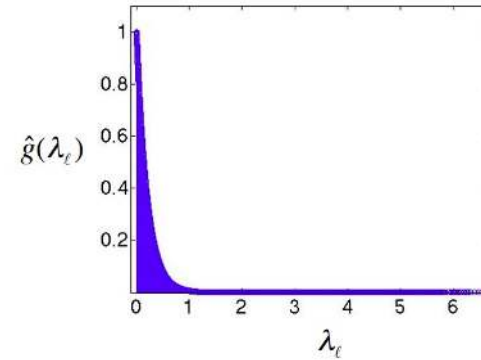
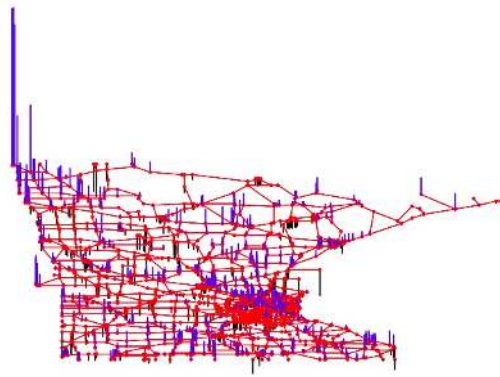
<https://arxiv.org/pdf/1211.0053.pdf>



Graph signal processing

└ Translation of signals on graphs

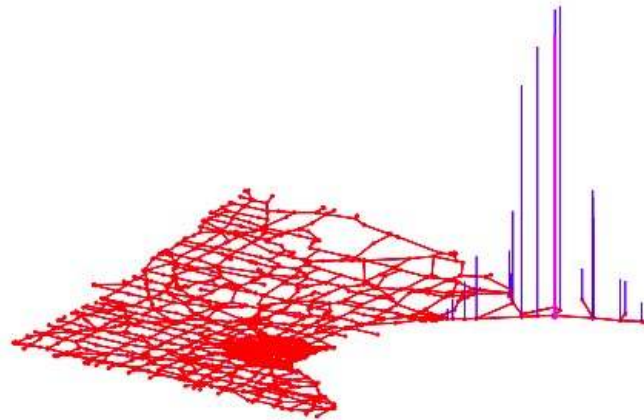
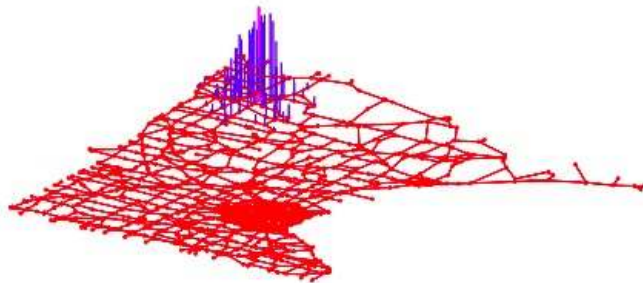
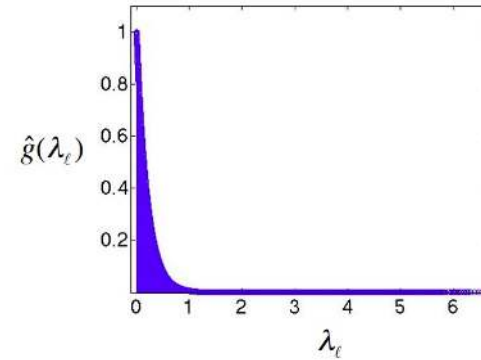
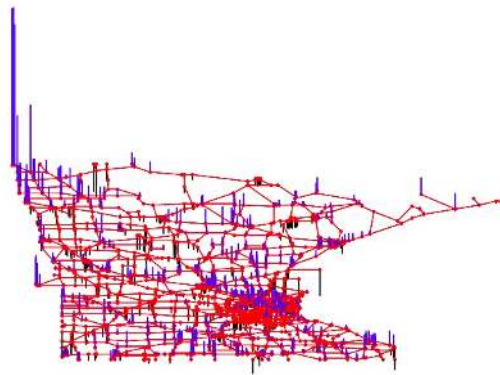
<https://arxiv.org/pdf/1211.0053.pdf>



Graph signal processing

└ Translation of signals on graphs

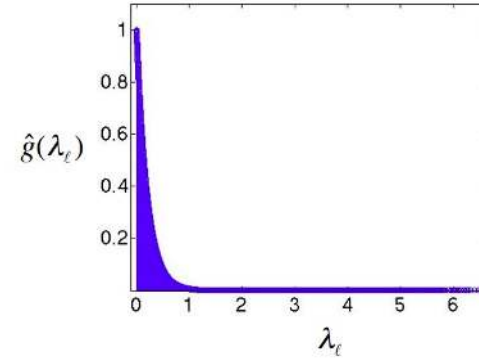
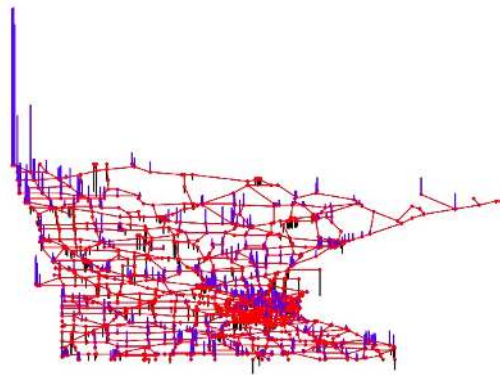
<https://arxiv.org/pdf/1211.0053.pdf>



Graph signal processing

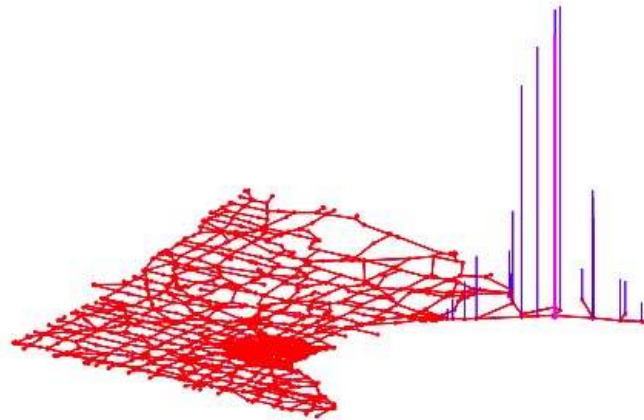
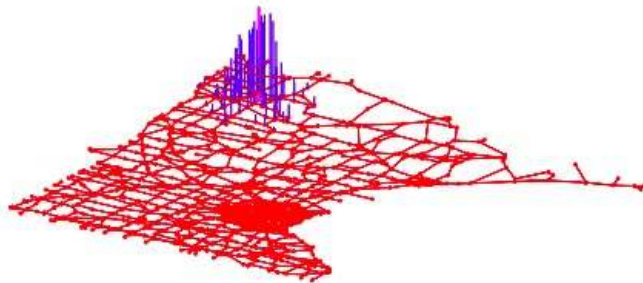
└ Translation of signals on graphs

<https://arxiv.org/pdf/1211.0053.pdf>



Remark #1

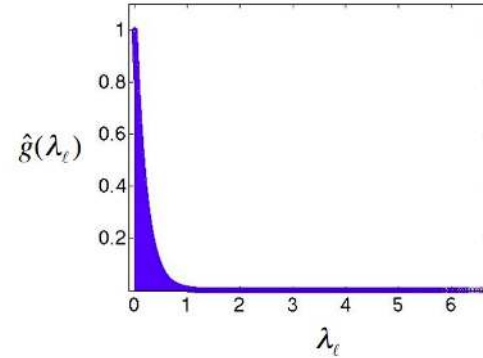
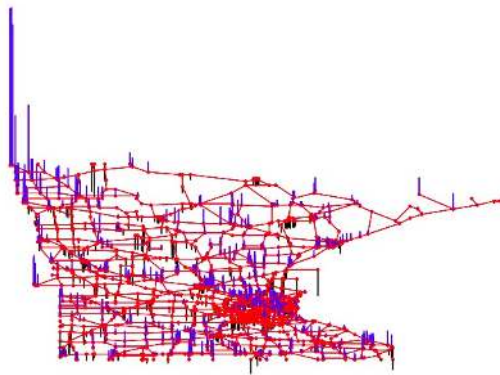
Rather a *relocalization*
than a *translation*



Graph signal processing

└ Translation of signals on graphs

<https://arxiv.org/pdf/1211.0053.pdf>

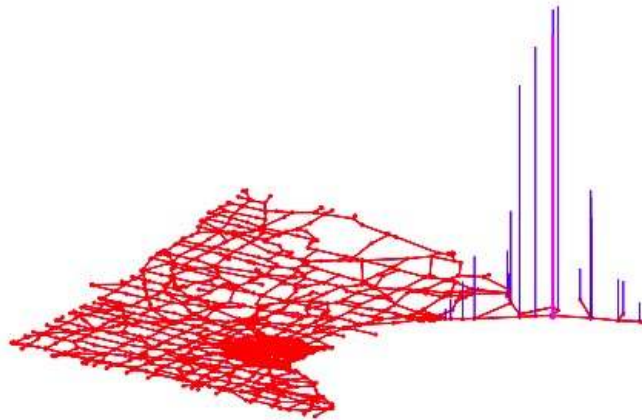
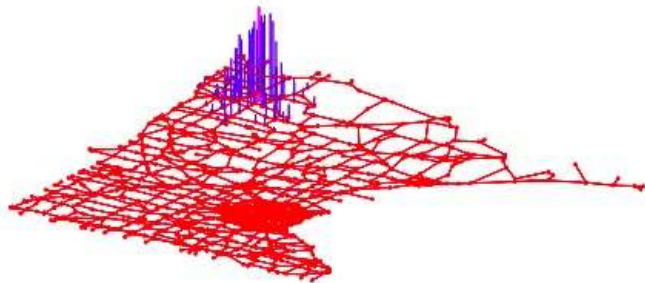


Remark #1

Rather a *relocalization*
than a *translation*

Remark #2

Don't translate an
already translated signal



Graph signal processing

└ Two awesome introductory papers

<https://arxiv.org/pdf/1211.0053.pdf>
<https://arxiv.org/pdf/1712.00468.pdf>

The Emerging Field of Signal Processing on Graphs

Extending High-Dimensional Data Analysis to Networks and Other Irregular Domains

David I Shuman[†], Sunil K. Narang[‡], Pascal Frossard[†], Antonio Ortega[‡] and Pierre Vandergheynst[†]

[†]Ecole Polytechnique Fédérale de Lausanne (EPFL), Signal Processing Laboratory (LTS2 and LTS4)

[‡]University of Southern California (USC), Signal and Image Processing Institute

{david.shuman, pascal.frossard, pierre.vandergheynst}@epfl.ch, kumarsun@usc.edu, antonio.ortega@sipi.usc.edu

Graph Signal Processing: Overview, Challenges and Applications

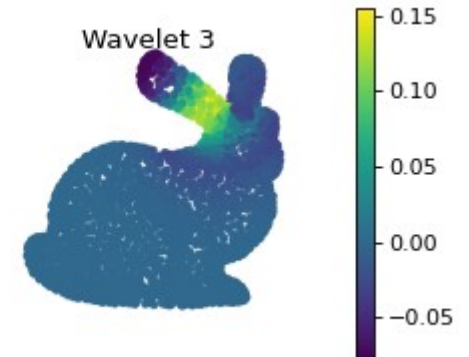
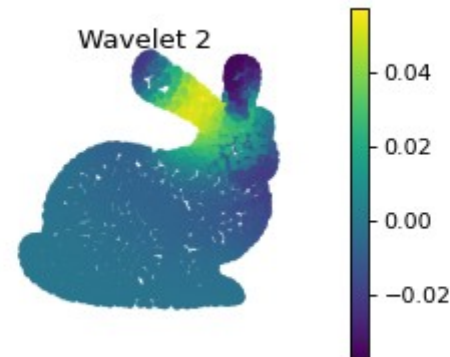
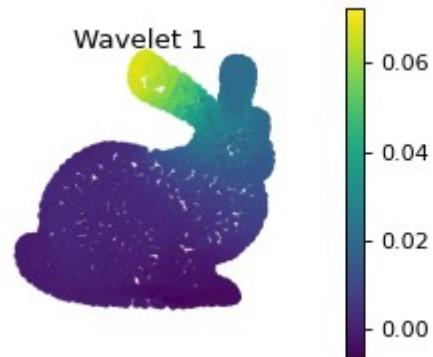
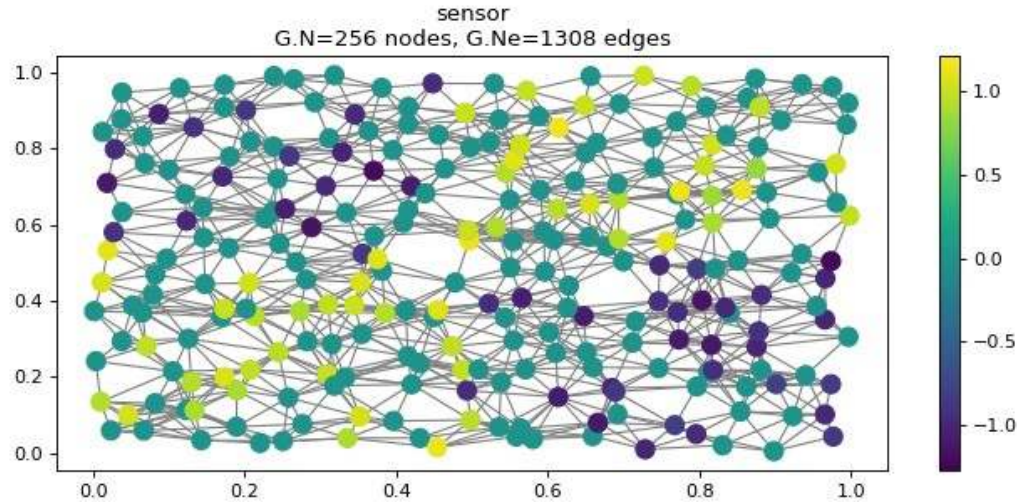
Antonio Ortega, *Fellow, IEEE*, Pascal Frossard, *Fellow, IEEE*, Jelena Kovačević, *Fellow, IEEE*, José M. F. Moura, *Fellow, IEEE*, and Pierre Vandergheynst



Graph signal processing

└ Some libraries exist – PyGSP / GSP Toolbox

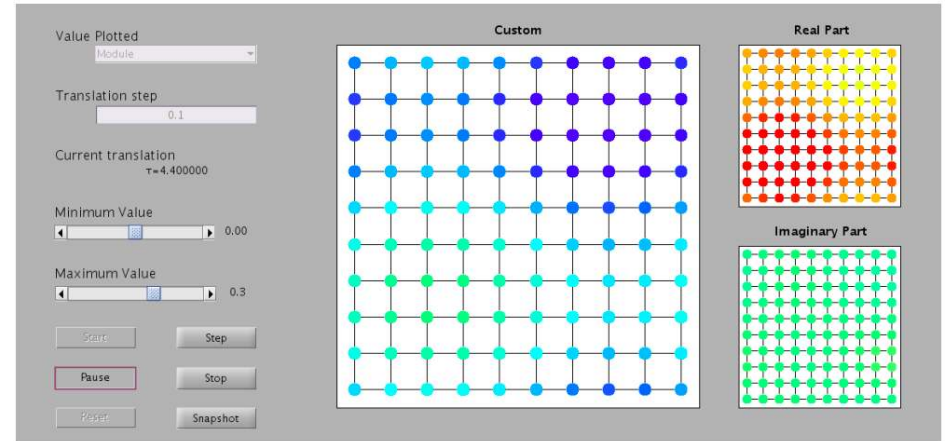
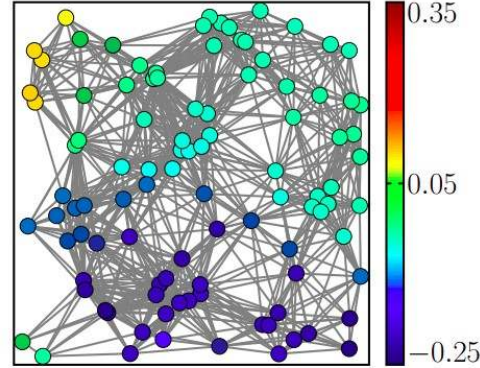
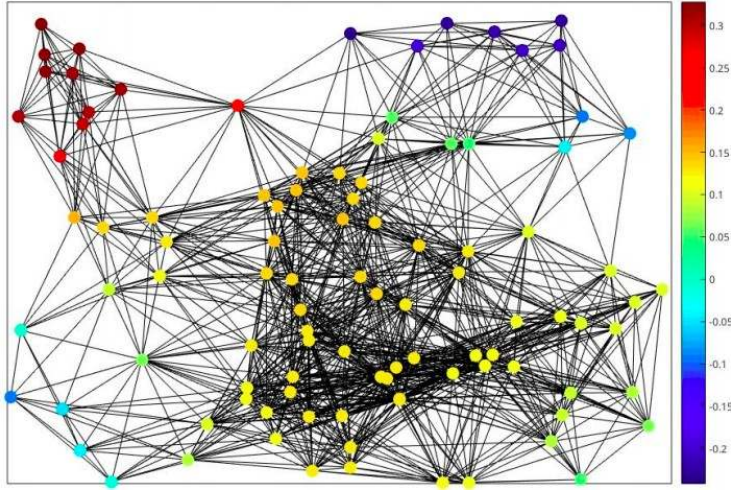
<https://pygsp.readthedocs.io/en/stable/>



Graph signal processing

└ Some libraries exist – GraSP

<https://github.com/GraSP-toolbox/GraSP>



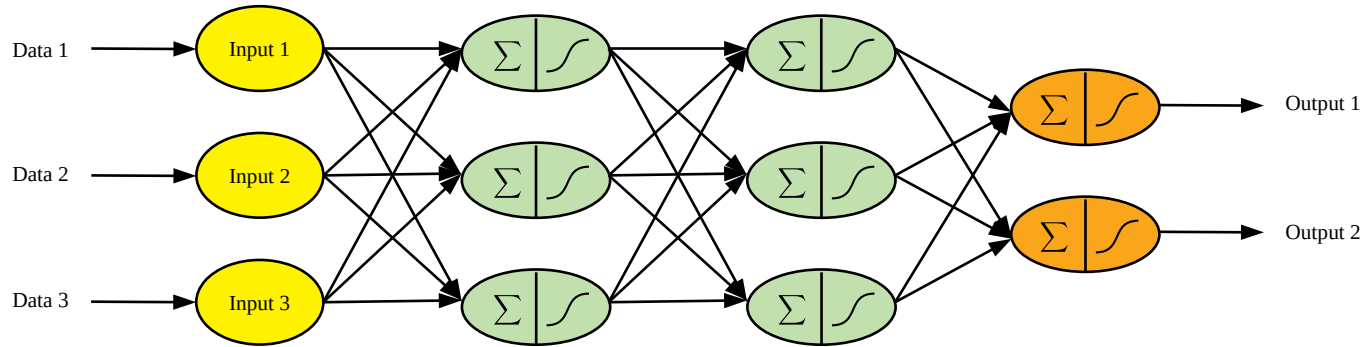
OUTLINE

- ▶ Signal processing
- ▶ Graph theory
- ▶ Graph signal processing
- ▶ Focus :
Graph convolutional networks



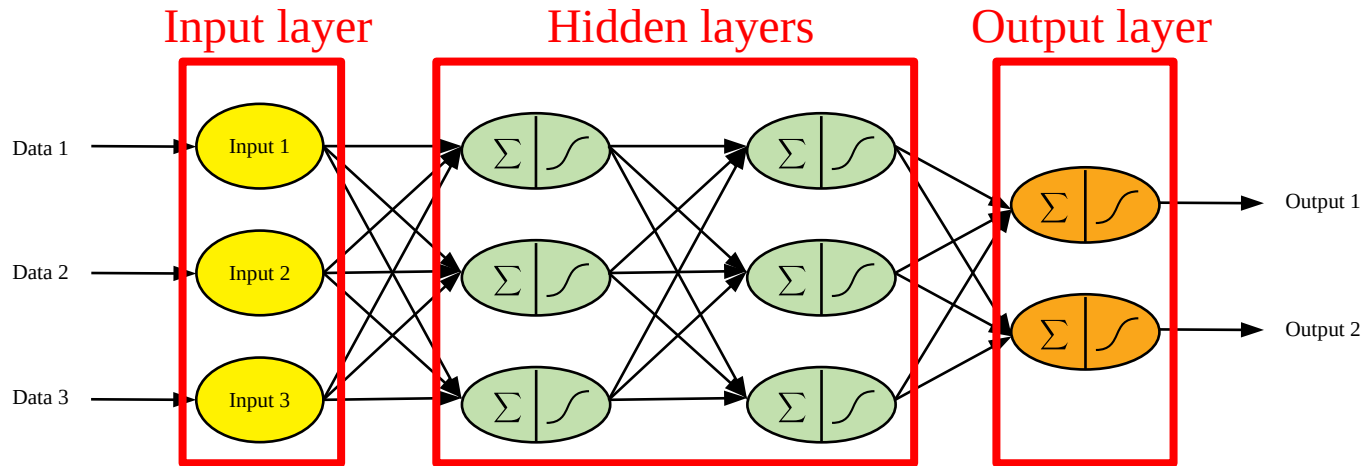
Graph convolutional networks

└ Reminders on neural networks



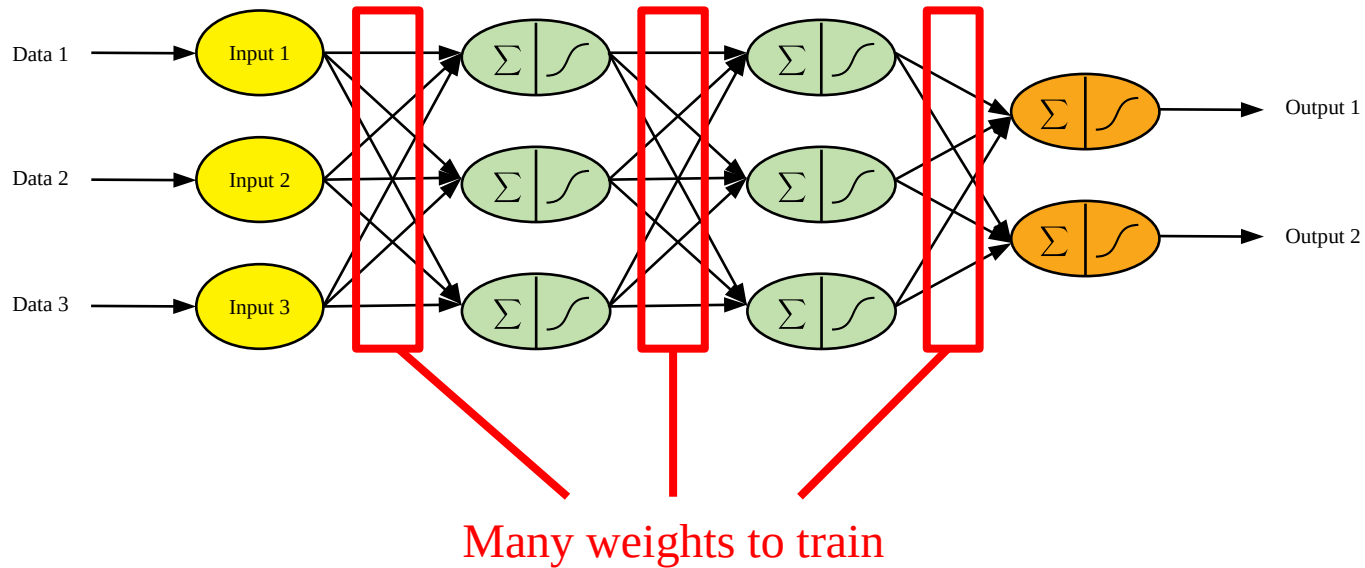
Graph convolutional networks

└ Reminders on neural networks



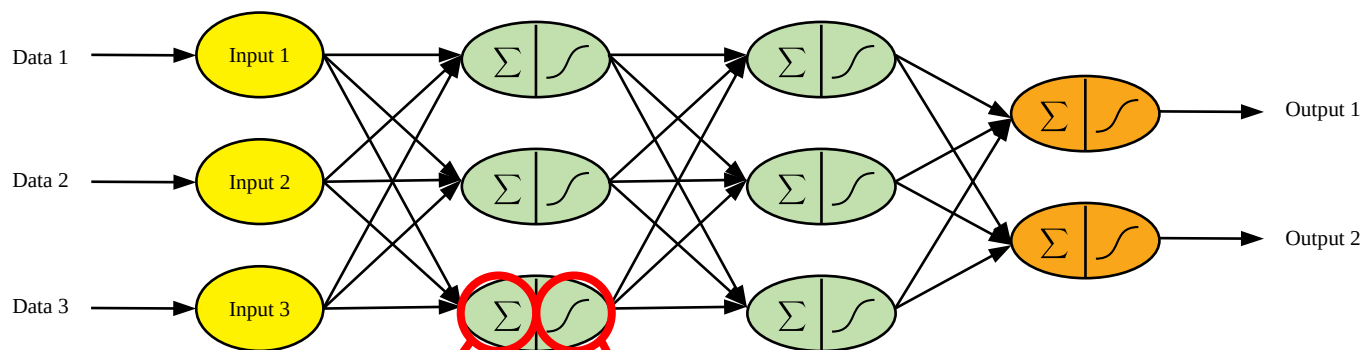
Graph convolutional networks

└ Reminders on neural networks



Graph convolutional networks

└ Reminders on neural networks



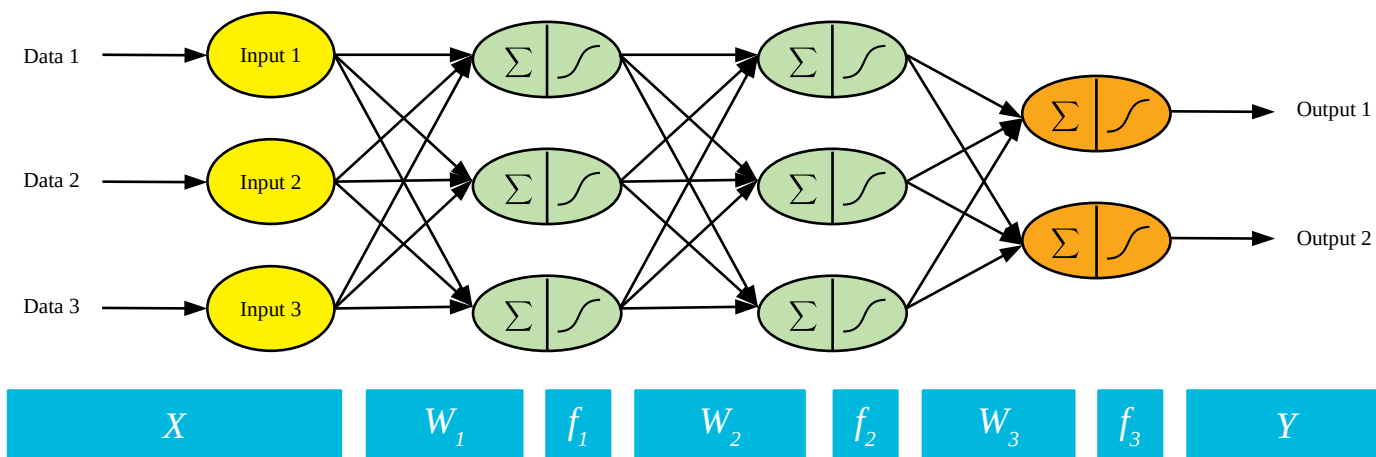
Weighted sum

Non-linearity:

- * Sigmoid
- * Hyperbolic tangent
- * Softmax
- * Rectified linear unit (ReLU)
- * ...

Graph convolutional networks

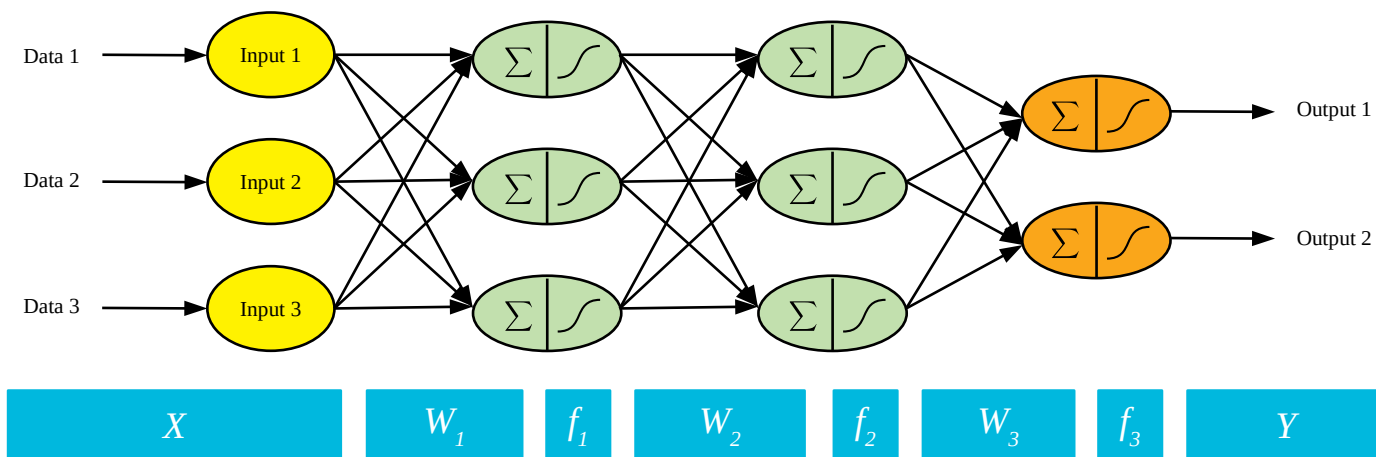
└ Reminders on neural networks



$$Y = f_3(W_3 \cdot f_2(W_2 \cdot f_1(W_1 \cdot X)))$$

Graph convolutional networks

└ Reminders on neural networks



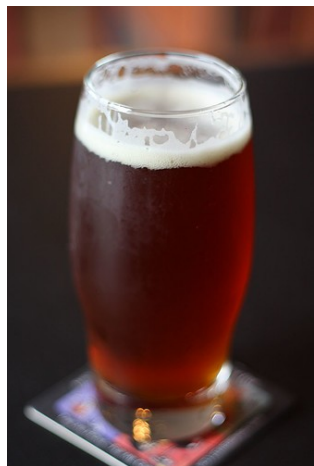
$$Y = f_3(W_3 \cdot f_2(W_2 \cdot f_1(W_1 \cdot X)))$$

Derivable → Backpropagation

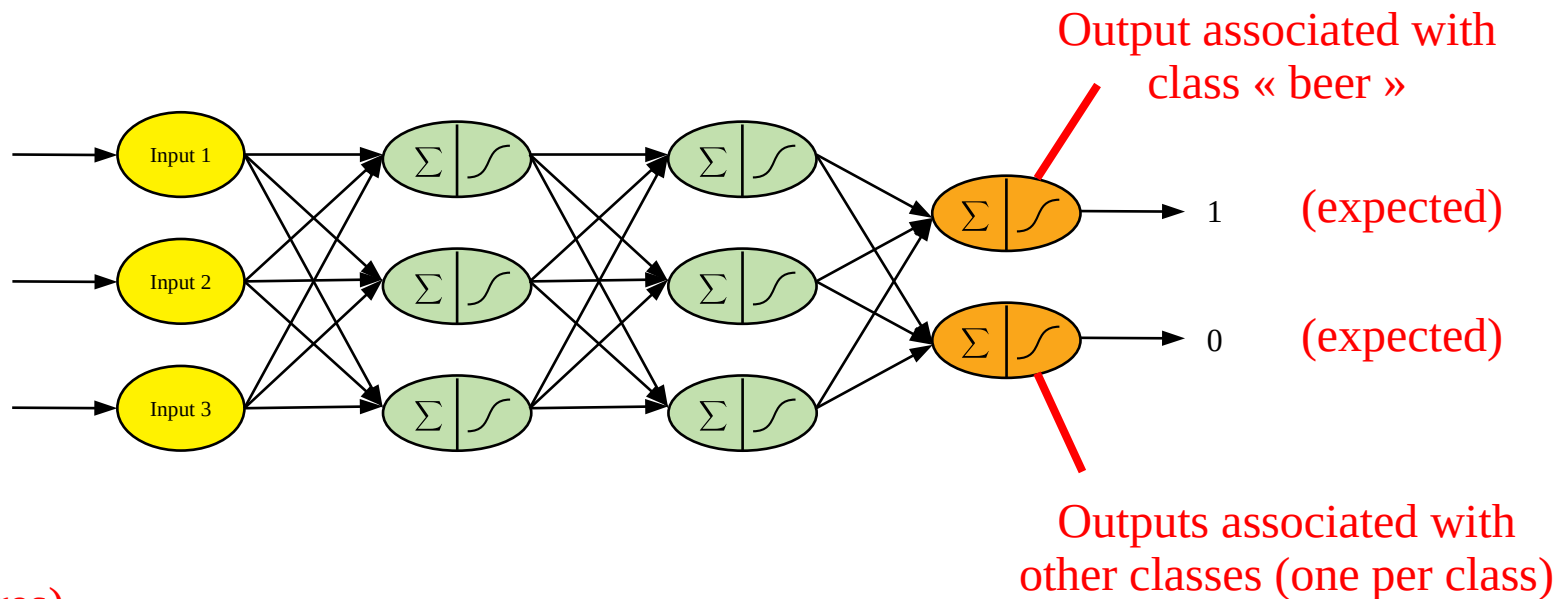
Graph convolutional networks

└ Reminders on neural networks

<http://www.image-net.org/>



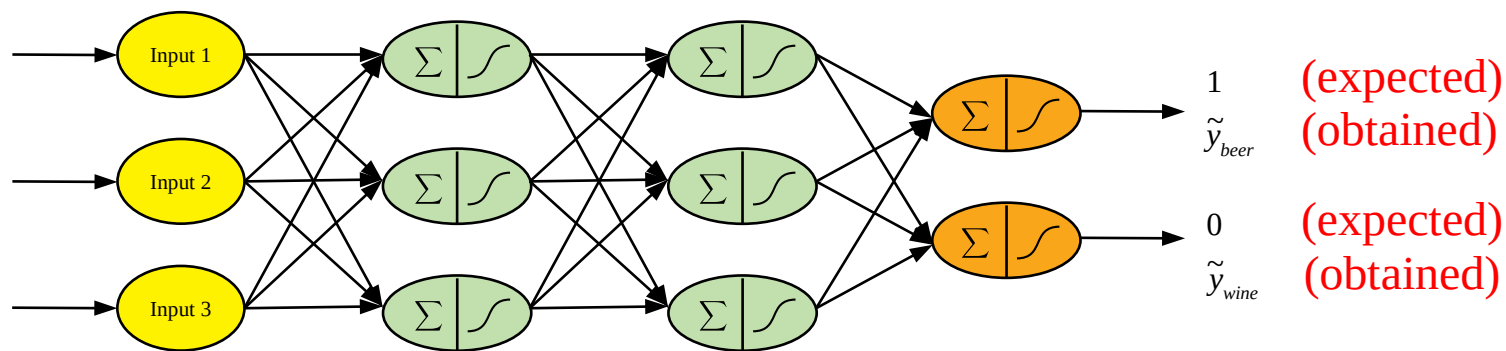
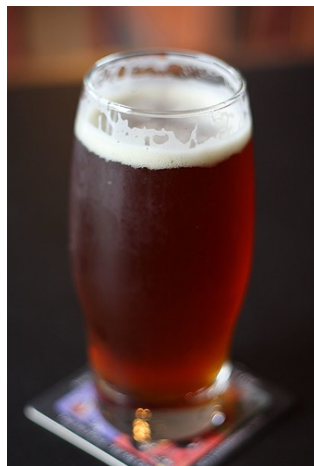
Raw input
(or sometimes features)



Graph convolutional networks

└ Reminders on neural networks

<http://www.image-net.org/>

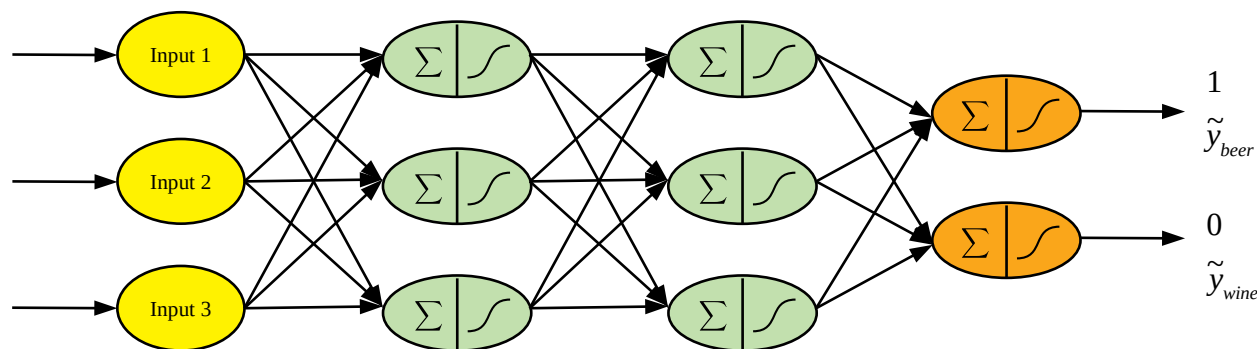
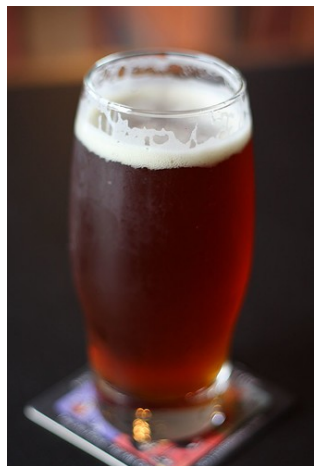


(1) Forward pass

Graph convolutional networks

└ Reminders on neural networks

<http://www.image-net.org/>



$$\epsilon = \text{loss}(\mathbf{Y}, \tilde{\mathbf{Y}})$$

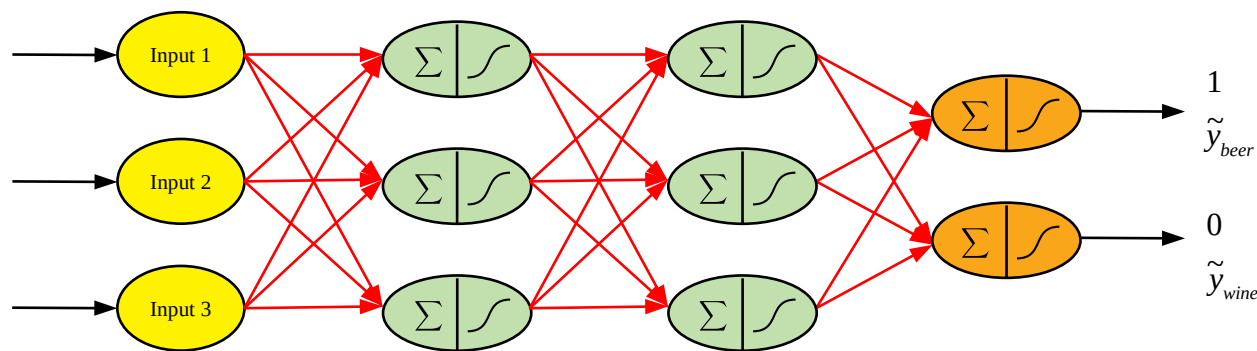
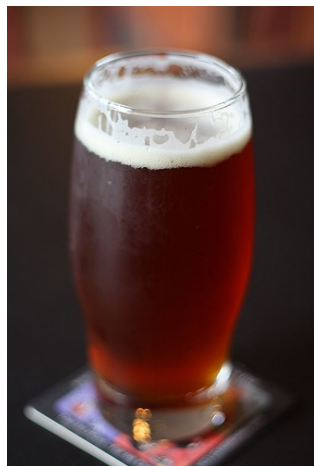
(2) Compute error
using *loss function*

e.g., cross-entropy: $-\log(\tilde{\mathbf{Y}}^T \mathbf{Y})$

Graph convolutional networks

└ Reminders on neural networks

<http://www.image-net.org/>



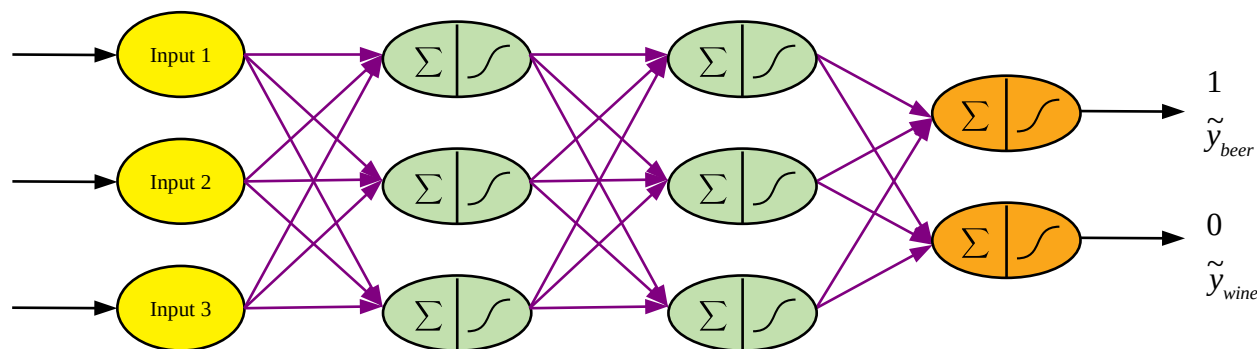
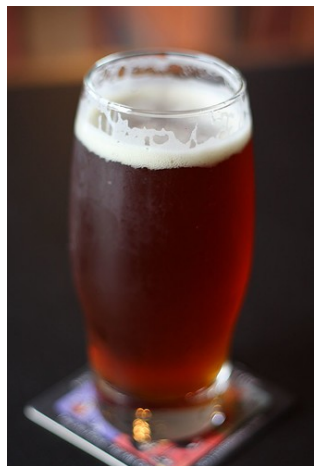
$$\varepsilon = \text{loss}(Y, \tilde{Y})$$

(3) Compute gradient with respect to individual weights

Graph convolutional networks

└ Reminders on neural networks

<http://www.image-net.org/>



$$\varepsilon = \text{loss}(Y, \tilde{Y})$$

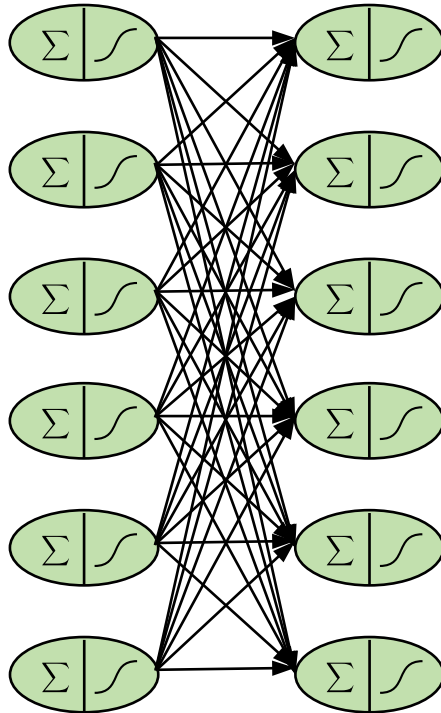
(4) Update weights:

$$w_{ij} := w_{ij} - \lambda \frac{\partial \varepsilon}{\partial w_{ij}}$$

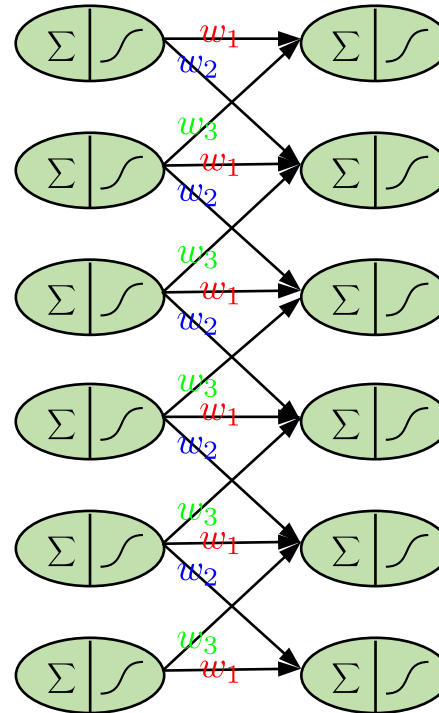
Graph convolutional networks

└ Reminders on neural networks

Dense



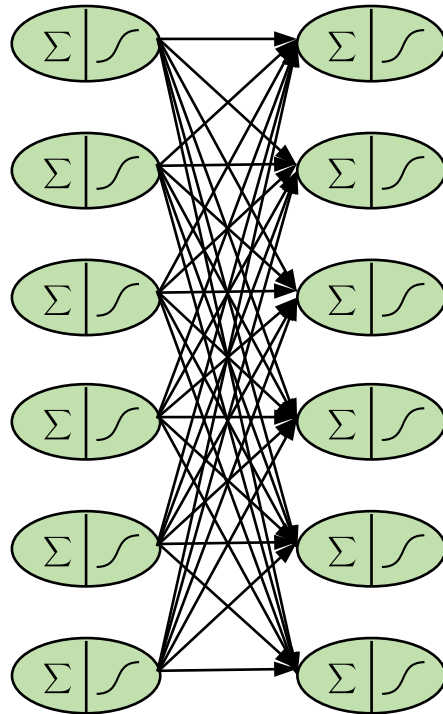
Convolutional



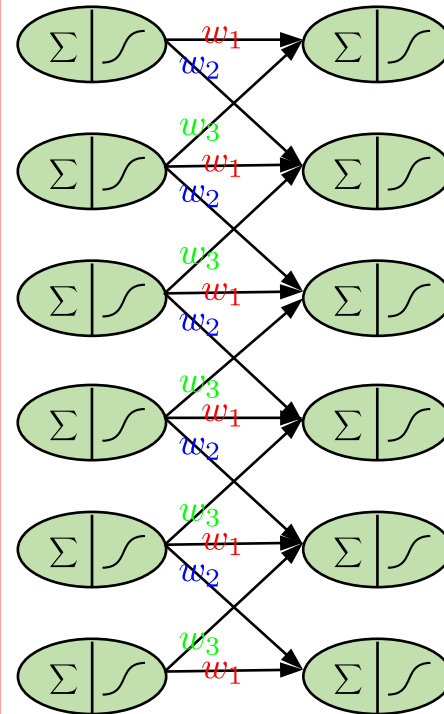
Graph **convolutional** networks

└ Reminders on neural networks

Dense

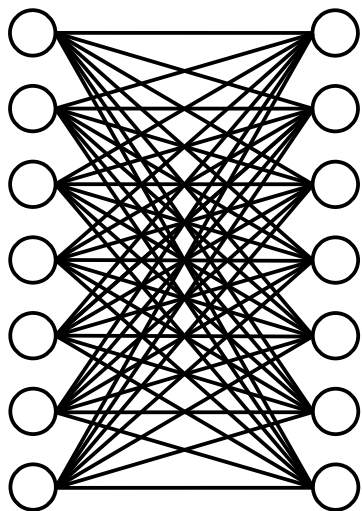


Convolutional



Graph convolutional networks

└ Convolutional neural networks

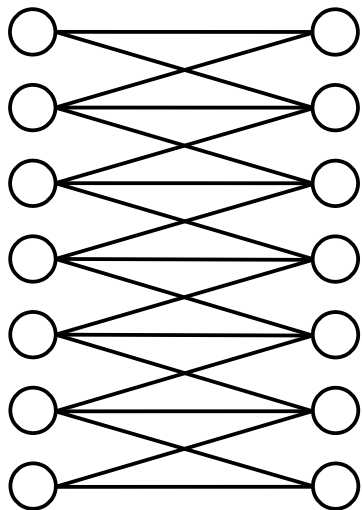


Dense layer

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} & w_{16} & w_{17} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} & w_{26} & w_{27} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} & w_{36} & w_{37} \\ w_{41} & w_{42} & w_{43} & w_{44} & w_{45} & w_{46} & w_{47} \\ w_{51} & w_{52} & w_{53} & w_{54} & w_{55} & w_{56} & w_{57} \\ w_{61} & w_{62} & w_{63} & w_{64} & w_{65} & w_{66} & w_{67} \\ w_{71} & w_{72} & w_{73} & w_{74} & w_{75} & w_{76} & w_{77} \end{bmatrix}$$

Graph convolutional networks

└ Convolutional neural networks



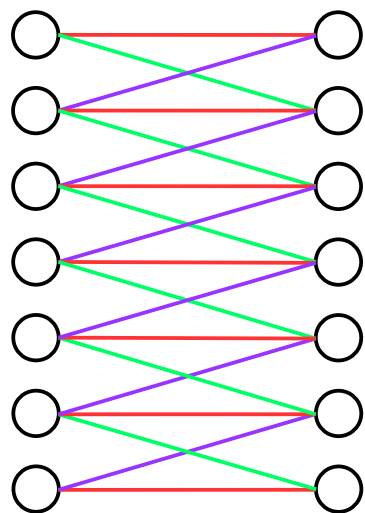
Convolutional
layer

A lot less weights to train

$$\begin{bmatrix} w_{11} & w_{12} & 0 & 0 & 0 & 0 & 0 \\ w_{21} & w_{22} & w_{23} & 0 & 0 & 0 & 0 \\ 0 & w_{32} & w_{33} & w_{34} & 0 & 0 & 0 \\ 0 & 0 & w_{43} & w_{44} & w_{45} & 0 & 0 \\ 0 & 0 & 0 & w_{54} & w_{55} & w_{56} & 0 \\ 0 & 0 & 0 & 0 & w_{65} & w_{66} & w_{67} \\ 0 & 0 & 0 & 0 & 0 & w_{76} & w_{77} \end{bmatrix}$$

Graph convolutional networks

└ Convolutional neural networks



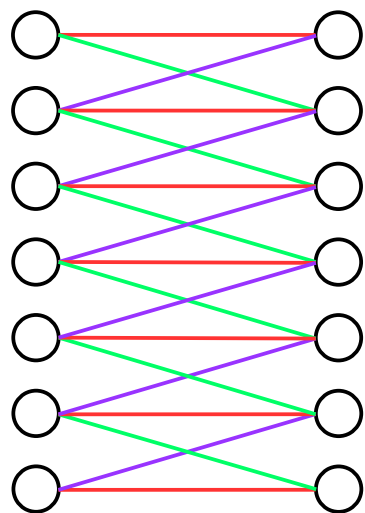
Convolutional
layer

Weights sharing

w_{11}	w_{12}	0	0	0	0	0
w_{21}	w_{22}	w_{23}	0	0	0	0
0	w_{32}	w_{33}	w_{34}	0	0	0
0	0	w_{43}	w_{44}	w_{45}	0	0
0	0	0	w_{54}	w_{55}	w_{56}	0
0	0	0	0	w_{65}	w_{66}	w_{67}
0	0	0	0	0	w_{76}	w_{77}

Graph convolutional networks

└ Convolutional neural networks



Convolutional
layer

Captures *locality* and
enforces *translation invariance*

$$\begin{bmatrix} w_{11} & w_{12} & 0 & 0 & 0 & 0 & 0 \\ w_{21} & w_{22} & w_{23} & 0 & 0 & 0 & 0 \\ 0 & w_{32} & w_{33} & w_{34} & 0 & 0 & 0 \\ 0 & 0 & w_{43} & w_{44} & w_{45} & 0 & 0 \\ 0 & 0 & 0 & w_{54} & w_{55} & w_{56} & 0 \\ 0 & 0 & 0 & 0 & w_{65} & w_{66} & w_{67} \\ 0 & 0 & 0 & 0 & 0 & w_{76} & w_{77} \end{bmatrix}$$

Graph convolutional networks

└ Convolutional neural networks

<https://inews.co.uk/culture/television/twin-peaks-episode-eight-david-lynch-75565>



Graph convolutional networks

└ Convolutional neural networks

<https://inews.co.uk/culture/television/twin-peaks-episode-eight-david-lynch-75565>



Initialize the convolutional layer
with one neuron per pixel



Graph convolutional networks

└ Convolutional neural networks

<https://inews.co.uk/culture/television/twin-peaks-episode-eight-david-lynch-75565>



Put a convolutional kernel
on your image (here 3x3)



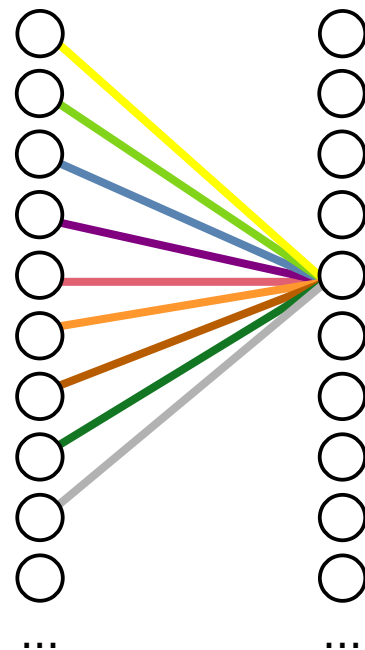
Graph convolutional networks

└ Convolutional neural networks

<https://inews.co.uk/culture/television/twin-peaks-episode-eight-david-lynch-75565>



Create connections where pixels are covered, with associated weights



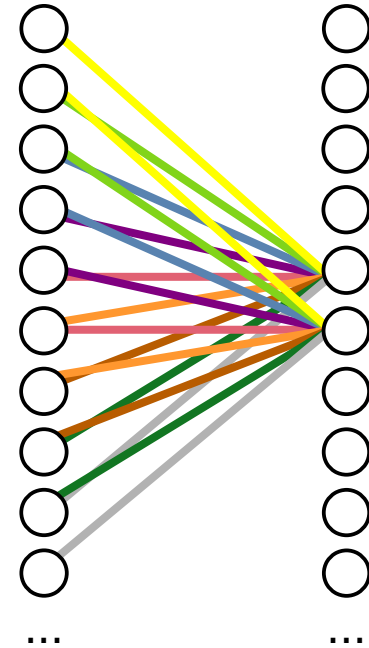
Graph convolutional networks

└ Convolutional neural networks

<https://inews.co.uk/culture/television/twin-peaks-episode-eight-david-lynch-75565>



Translate your kernel
and do it again



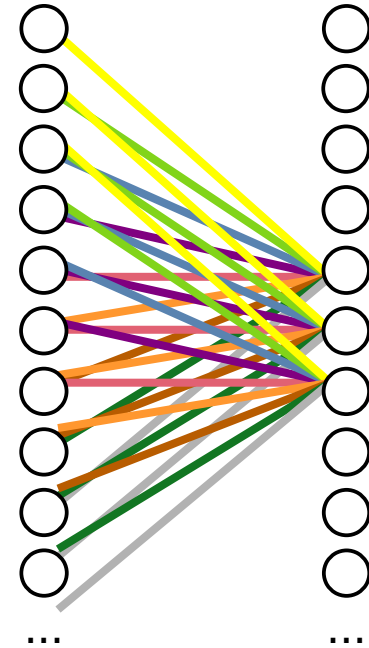
Graph convolutional networks

└ Convolutional neural networks

<https://inews.co.uk/culture/television/twin-peaks-episode-eight-david-lynch-75565>



... and again ...



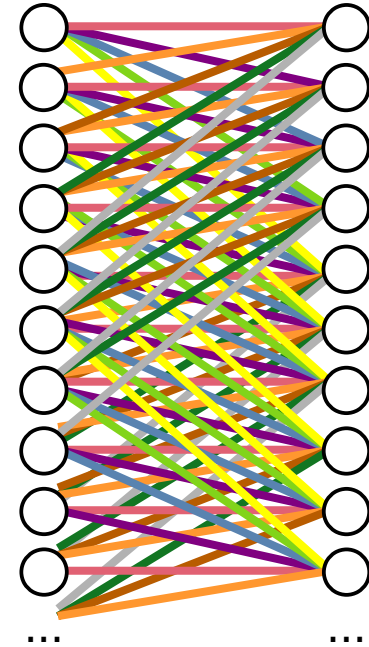
Graph convolutional networks

└ Convolutional neural networks

<https://news.co.uk/culture/television/twin-peaks-episode-eight-david-lynch-75565>



... for any possible
kernel location

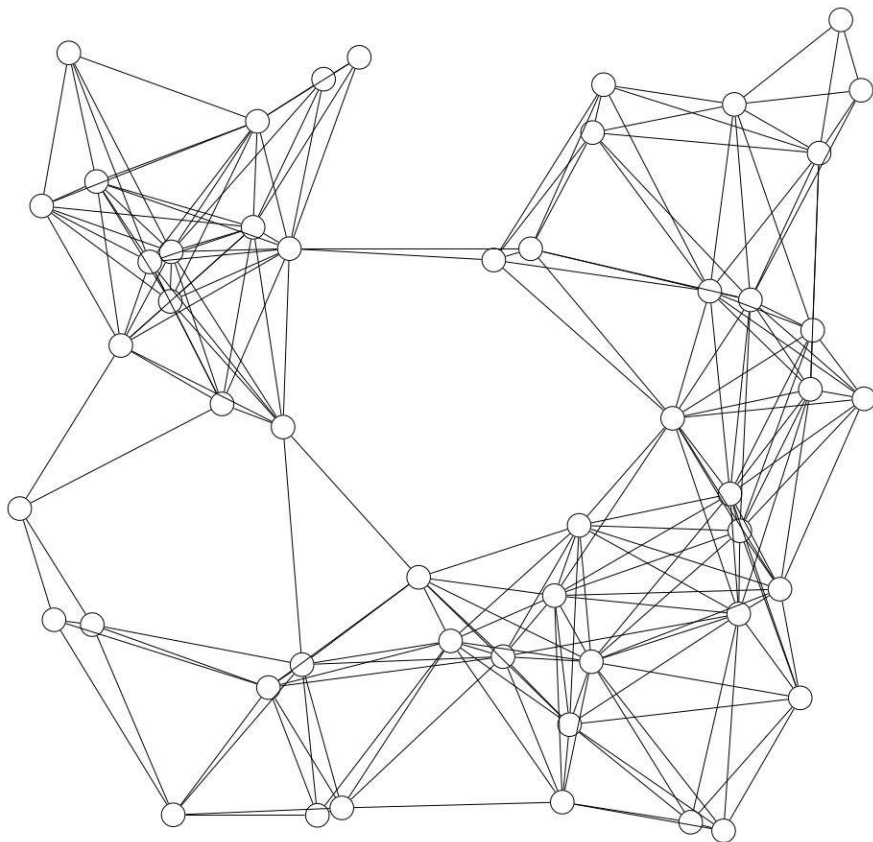


Graph convolutional networks

└ Extension to graphs

<https://tel.archives-ouvertes.fr/tel-01708824>

https://southpark.fandom.com/wiki/Underpants_Gnomes



Step 1: Initialize the graph convolutional layer with one neuron per vertex



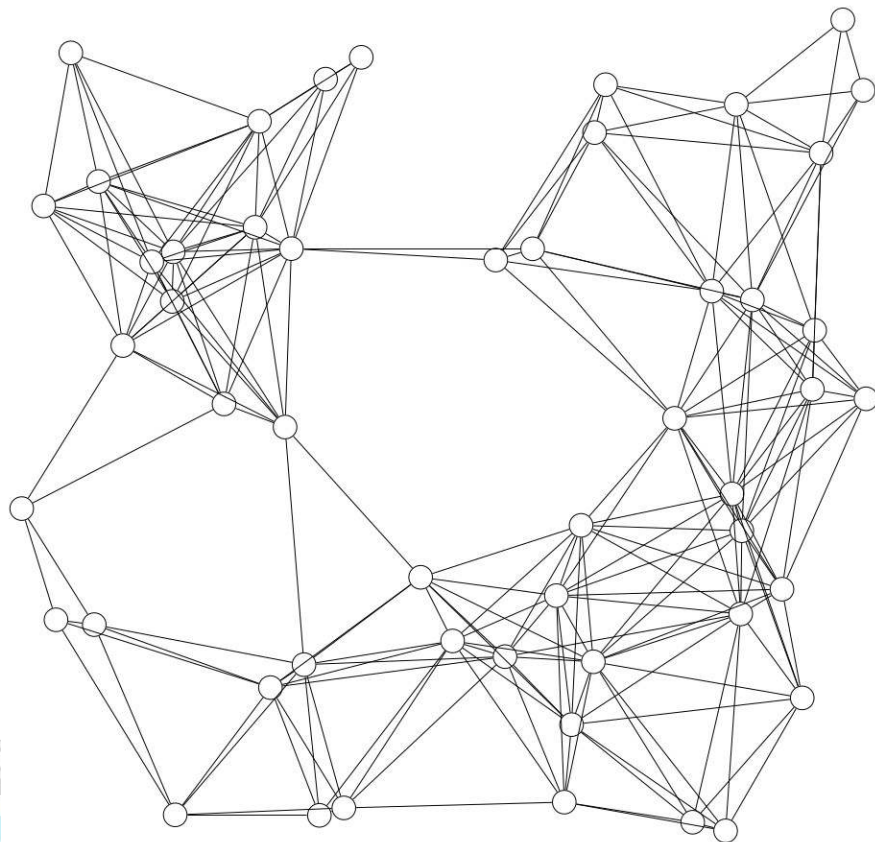
Graph convolutional networks

└ Extension to graphs

<https://tel.archives-ouvertes.fr/tel-01708824>

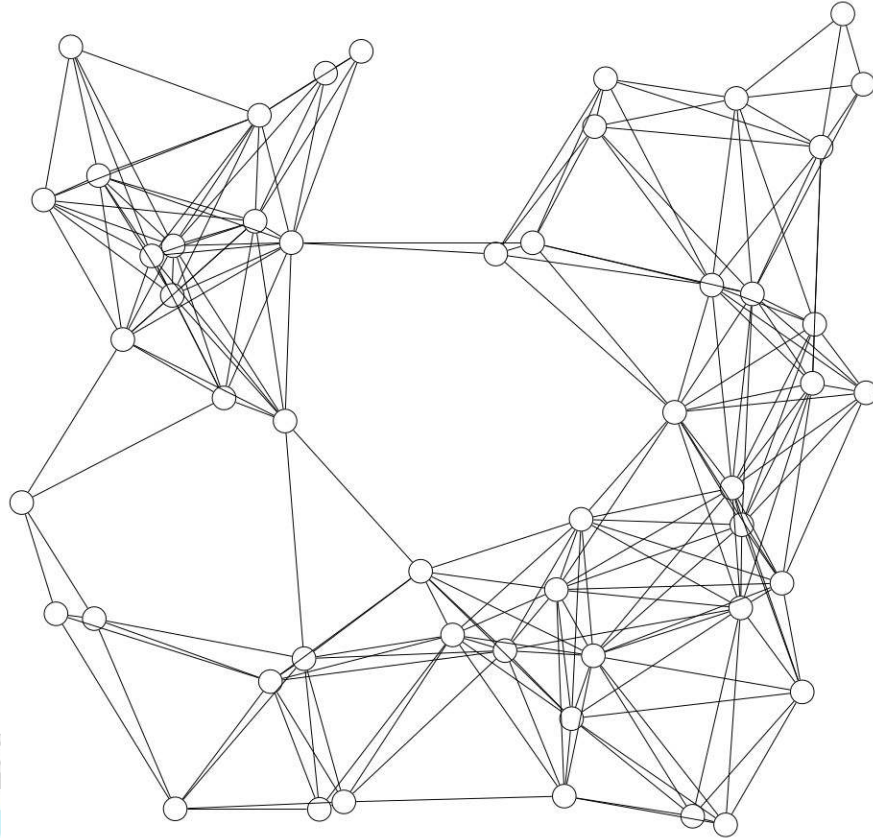
https://southpark.fandom.com/wiki/Underpants_Gnomes

Step 2: ...



Extension to graphs

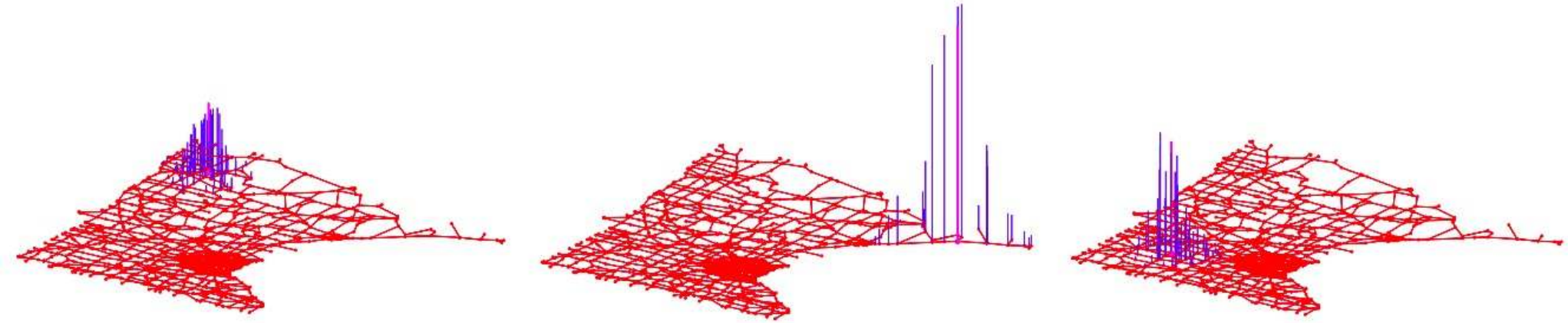
Step 3: PROFIT!!!



Graph convolutional networks

└ The problems with graphs

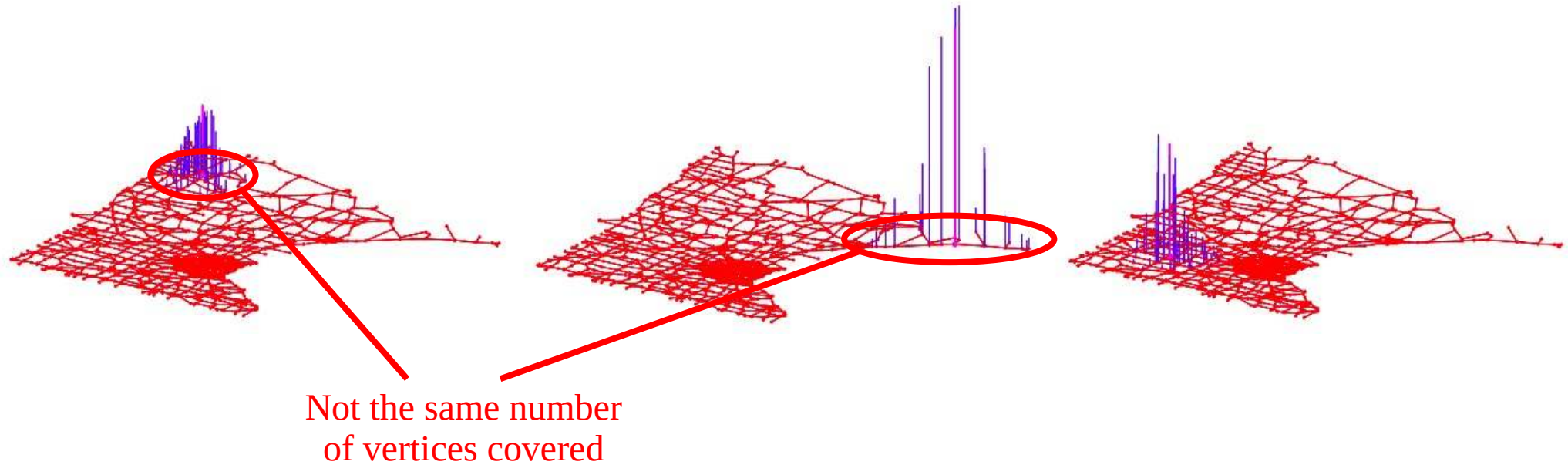
<https://arxiv.org/pdf/1211.0053.pdf>



Graph convolutional networks

└ The problems with graphs

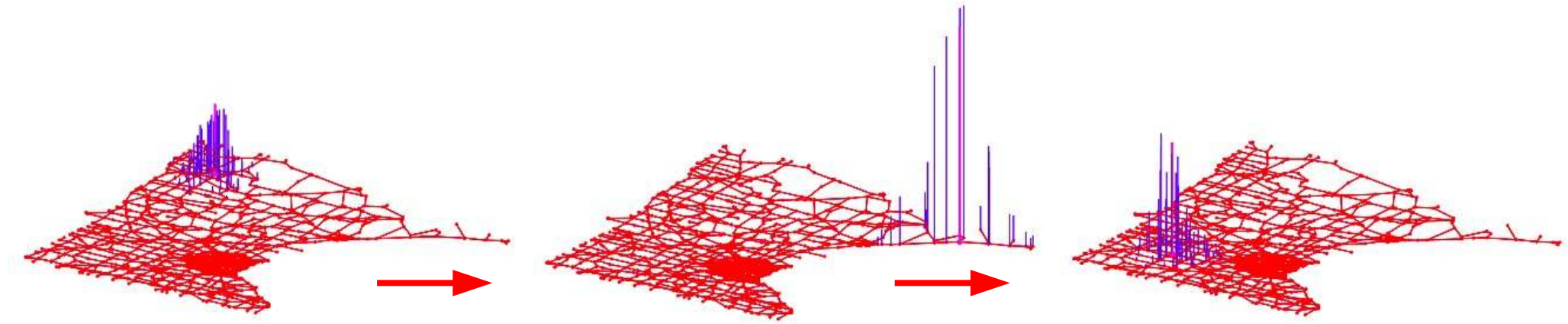
<https://arxiv.org/pdf/1211.0053.pdf>



Graph convolutional networks

└ The problems with graphs

<https://arxiv.org/pdf/1211.0053.pdf>

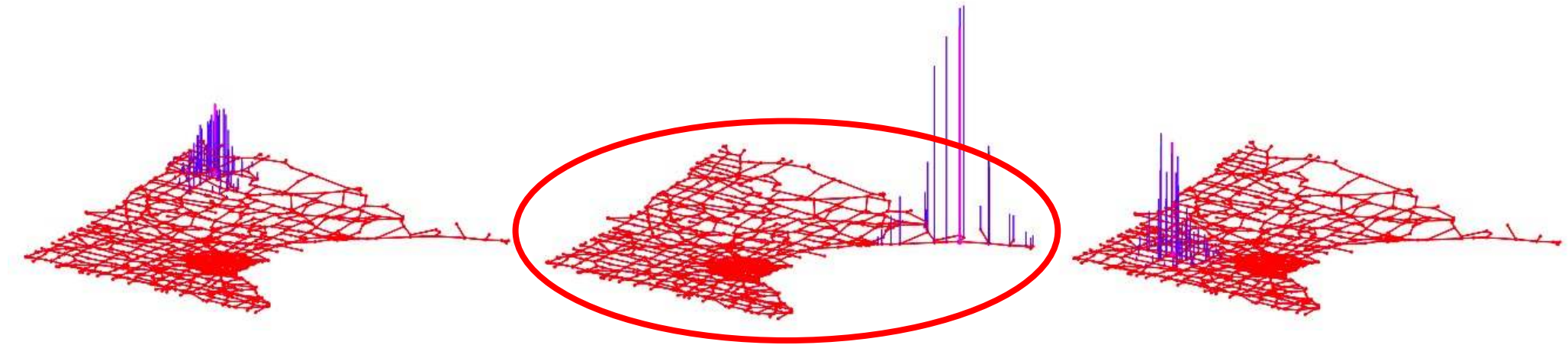


Cannot just translate kernel, need to relocalize from spectral definition
→ How do you accomplish weight sharing?

Graph convolutional networks

└ The problems with graphs

<https://arxiv.org/pdf/1211.0053.pdf>



Graphs can be quite big, which poses the question of required resources for diagonalization

Graph convolutional networks

└ Possible solutions (some)

- λ Bruna *et al.* – Spectral networks and locally connected networks on graphs (2013)
- λ Henaff *et al.* – Deep convolutional networks on graph-structured data (2015)
- $\mathcal{G}$ Atwood *et al.* – Search-convolutional neural Networks (2015)
- $\mathcal{G}$ Defferrard *et al.* – Convolutional neural networks on graphs with fast localized spectral filtering (2016)
- $\mathcal{G}$ Puy *et al.* – Unifying local and non-local signal processing with graph CNNs (2017)
- $\mathcal{G}$ Hechtlinger *et al.* – A Generalization of convolutional neural networks to graph-structured data (2017)
- $\mathcal{G}$ Vialatte *et al.* – Learning local receptive fields and their weight sharing scheme on graphs (2017)
- $\mathcal{G}$ Pasdeloup – Extending convolutional neural networks to irregular domains through graph inference (2017)



Graph convolutional networks

└ Possible solutions (some)

 **Google Scholar**

graph convolutional networks



 **Articles**

Environ 142 000 résultats (0,07 s)

Date indifférente

Depuis 2021

Depuis 2020

Depuis 2017

Période spécifique...

Trier par pertinence

Trier par date

Toutes les langues

Rechercher les pages en Français

☐ inclure les brevets

☐ inclure les citations

☒ Créer l'alerte

Simplifying graph convolutional networks

[F Wu, A Souza, T Zhang, C Fifty, T Yu...](#) - International ..., 2019 - proceedings.mlr.press

Abstract **Graph Convolutional Networks** (GCNs) and their variants have experienced significant attention and have become the de facto methods for learning **graph** representations. GCNs derive inspiration primarily from recent deep learning approaches ...

☆  Cité 391 fois [Autres articles](#) [Les 4 versions](#) 

[\[PDF\] mlr.press](#)

Large-scale learnable graph convolutional networks

[H Gao, Z Wang, S Ji](#) - Proceedings of the 24th ACM SIGKDD ..., 2018 - dl.acm.org

Convolutional neural **networks** (CNNs) have achieved great success on grid-like data such as images, but face tremendous challenges in learning from more generic data such as **graphs**. In CNNs, the trainable local filters enable the automatic extraction of high-level ...

☆  Cité 193 fois [Autres articles](#) [Les 11 versions](#) 

[\[PDF\] acm.org](#)

Semi-supervised classification with graph convolutional networks

[TN Kipf, M Welling](#) - arXiv preprint arXiv:1609.02907, 2016 - arxiv.org

We present a scalable approach for semi-supervised learning on **graph**-structured data that is based on an efficient variant of **convolutional** neural **networks** which operate directly on **graphs**. We motivate the choice of our **convolutional** architecture via a localized first-order ...

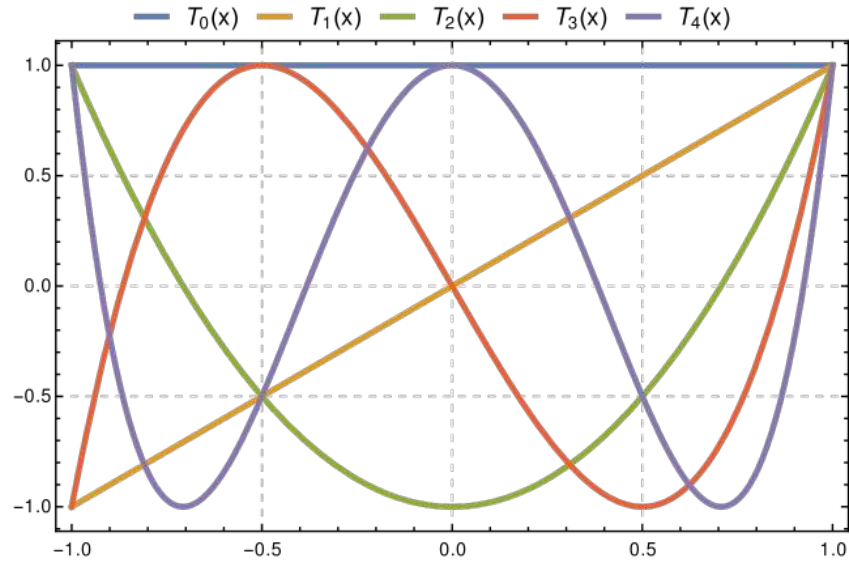
☆  Cité 7061 fois [Autres articles](#) [Les 15 versions](#) 

[\[PDF\] arxiv.org](#)



Graph convolutional networks

└ Chebyshev polynomials



<https://arxiv.org/pdf/1606.09375.pdf>
https://en.wikipedia.org/wiki/Chebyshev_polynomials

$$T_0(x) = 1$$

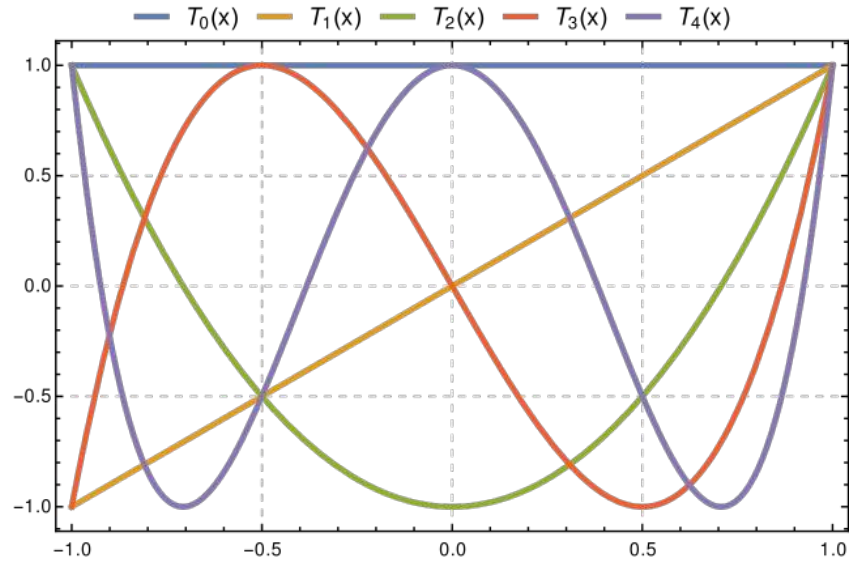
$$T_1(x) = x$$

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$



Graph convolutional networks

└ Chebyshev polynomials



$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

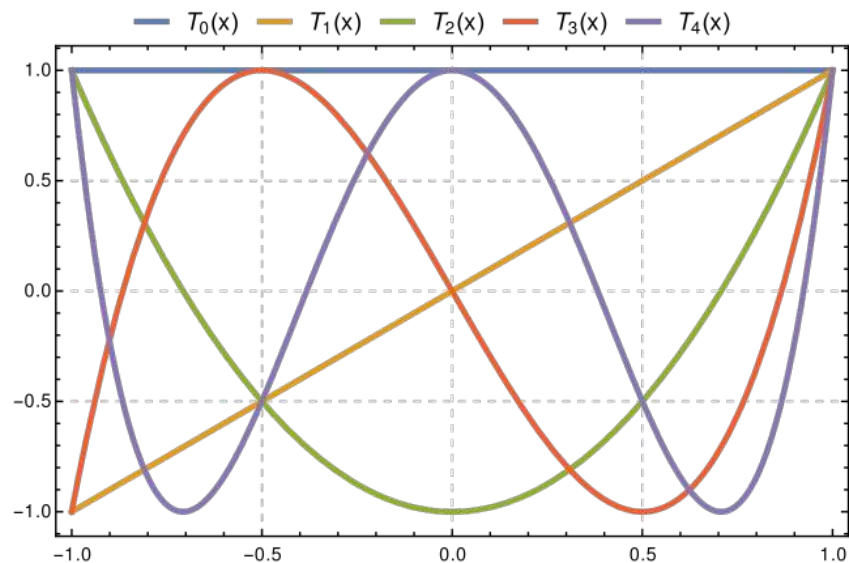
<https://arxiv.org/pdf/1606.09375.pdf>
https://en.wikipedia.org/wiki/Chebyshev_polynomials

$$g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{\Lambda})$$



Graph convolutional networks

Chebyshev polynomials



$$\begin{aligned}T_0(x) &= 1 \\T_1(x) &= x \\T_{n+1}(x) &= 2xT_n(x) - T_{n-1}(x)\end{aligned}$$

<https://arxiv.org/pdf/1606.09375.pdf>
https://en.wikipedia.org/wiki/Chebyshev_polynomials

Polynomial
order

Filter coefficients

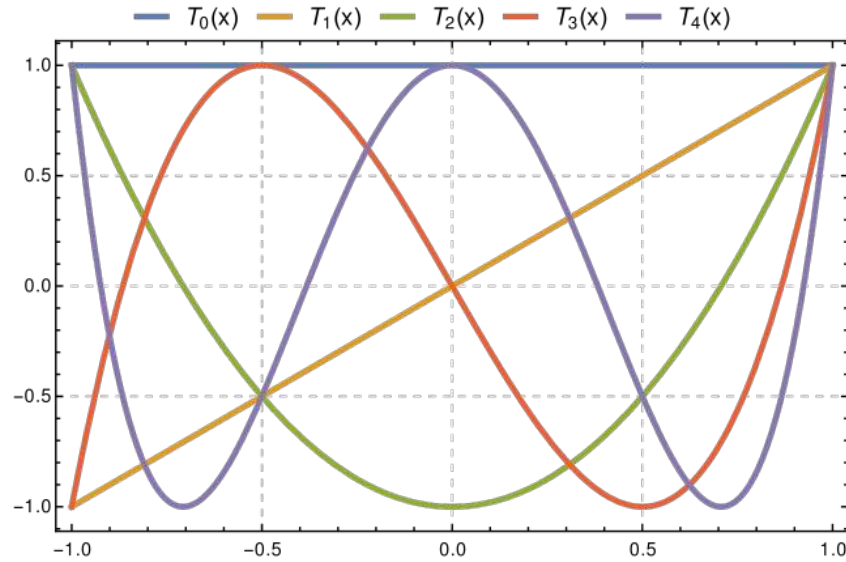
$$g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{\Lambda})$$

Filter

Eigenvalues

Graph convolutional networks

Chebyshev polynomials



$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

<https://arxiv.org/pdf/1606.09375.pdf>
https://en.wikipedia.org/wiki/Chebyshev_polynomials

Polynomial
order

Filter coefficients

$$g_{\theta}(\Lambda)$$

Filter

$$\sum_{k=0}^{K-1}$$

$$\theta_k$$

$$T_k(\tilde{\Lambda})$$

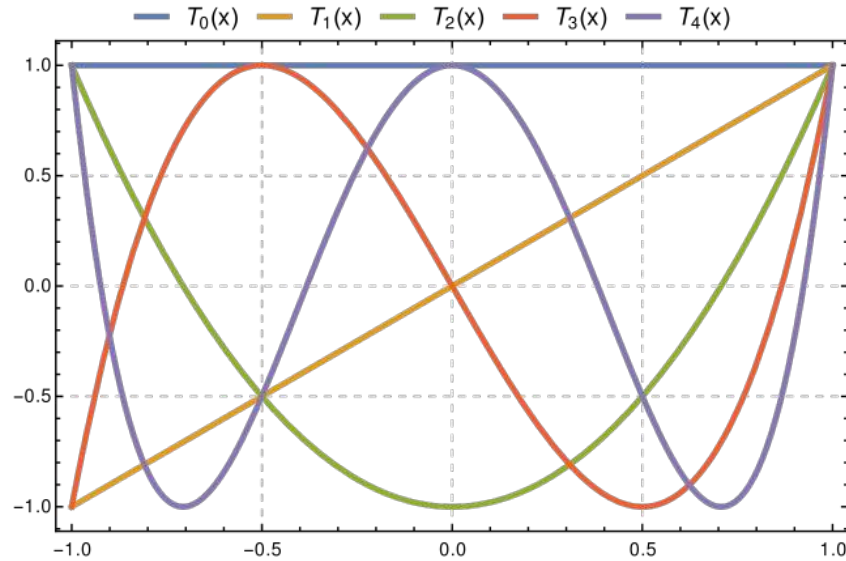
Eigenvalues

The filtering operation can then be written as $y = g_{\theta}(L)x = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{L})x$, where $T_k(\tilde{L}) \in \mathbb{R}^{n \times n}$ is the Chebyshev polynomial of order k evaluated at the scaled Laplacian $\tilde{L} = 2L/\lambda_{max} - I_n$.



Graph convolutional networks

Chebyshev polynomials



$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

<https://arxiv.org/pdf/1606.09375.pdf>
https://en.wikipedia.org/wiki/Chebyshev_polynomials

Polynomial
order

Filter coefficients

$$g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{\Lambda})$$

Filter

Eigenvalues

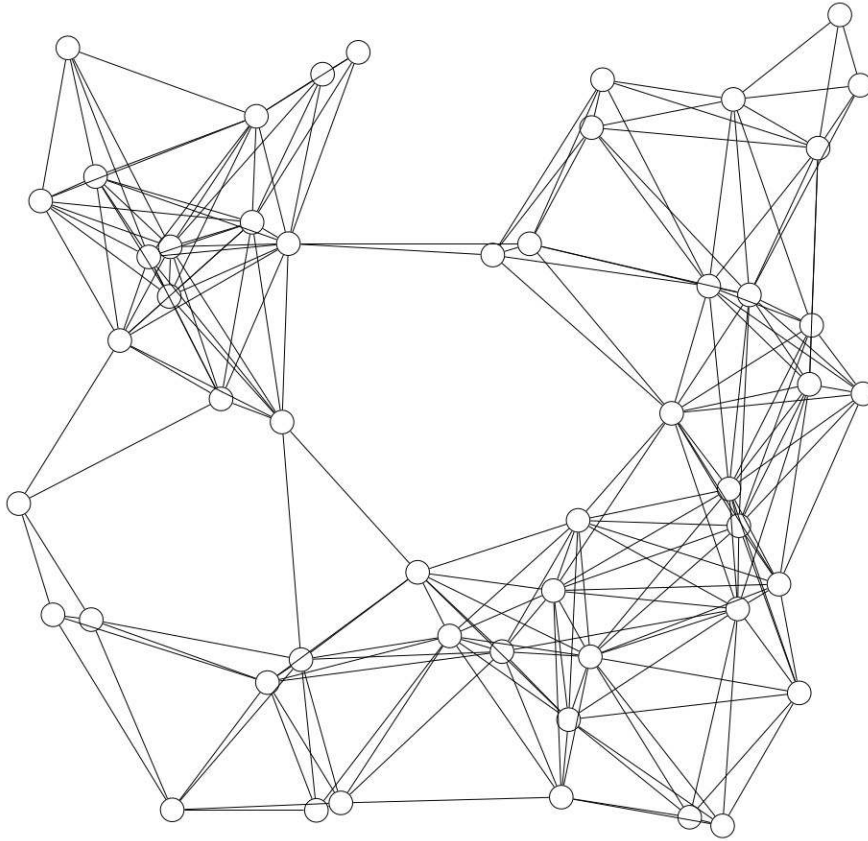
The filtering operation can then be written as $y = g_{\theta}(L)x = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{L})x$, where $T_k(\tilde{L}) \in \mathbb{R}^{n \times n}$ is the Chebyshev polynomial of order k evaluated at the scaled Laplacian $\tilde{L} = 2L/\lambda_{\max} - I_n$.

Remark: This is computed in the graph domain → No diagonalization
Remark #2: Does not work in practice

Graph convolutional networks

└ Graph convolutional layers with Chebyshev polynomials

<https://tel.archives-ouvertes.fr/tel-01708824>

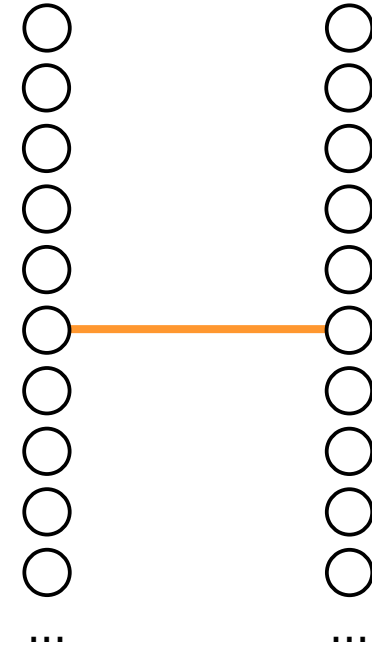
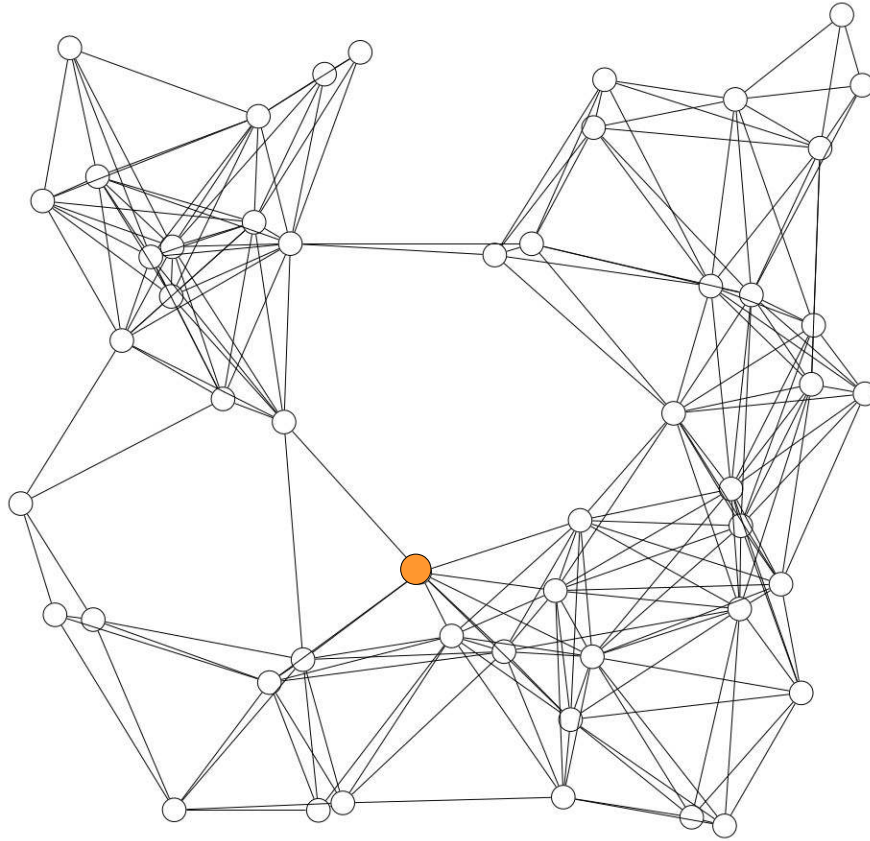


Graph convolutional networks

└ Graph convolutional layers with Chebyshev polynomials

<https://tel.archives-ouvertes.fr/tel-01708824>

$K = 0$

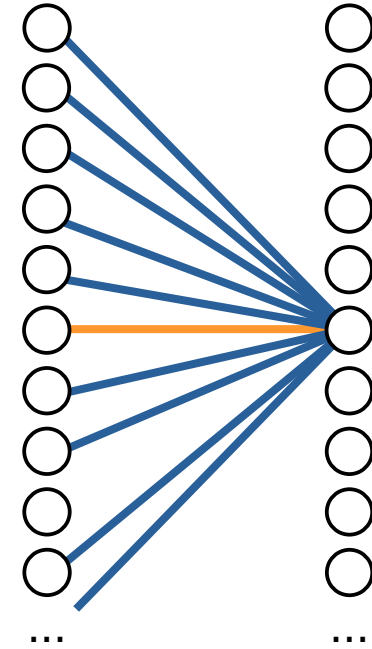
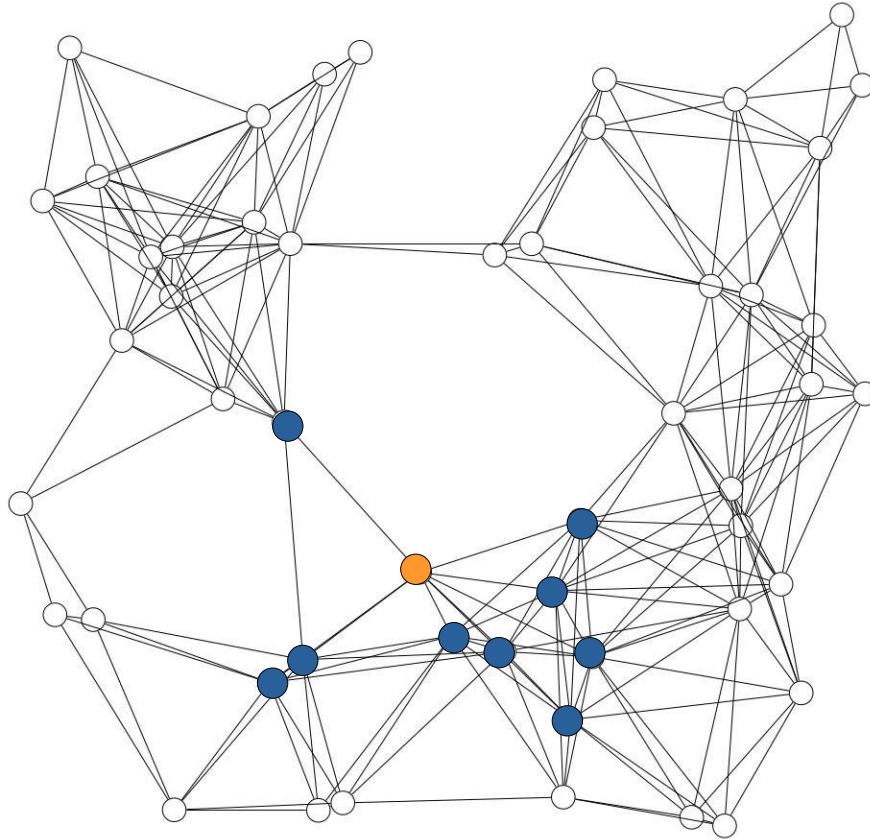


Graph convolutional networks

└ Graph convolutional layers with Chebyshev polynomials

<https://tel.archives-ouvertes.fr/tel-01708824>

$K = 1$

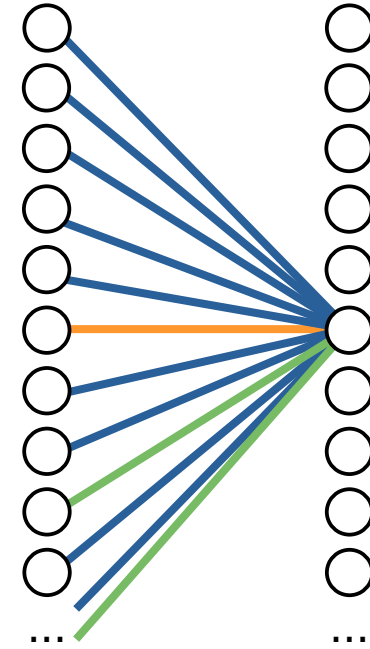
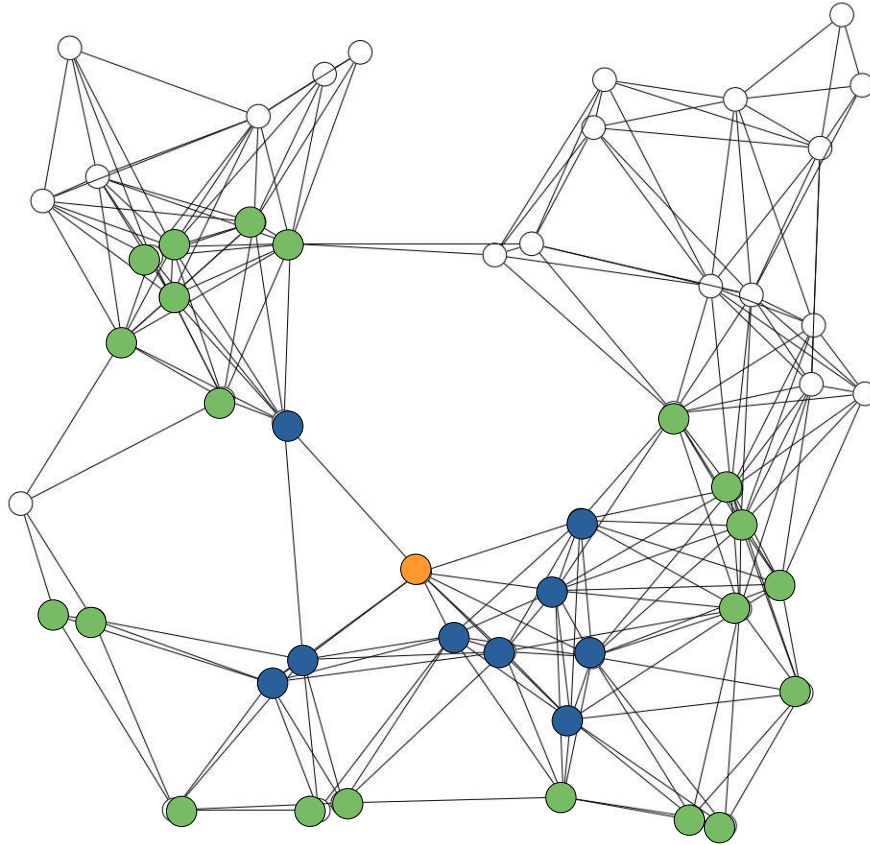


Graph convolutional networks

└ Graph convolutional layers with Chebyshev polynomials

$K = 2$

<https://tel.archives-ouvertes.fr/tel-01708824>

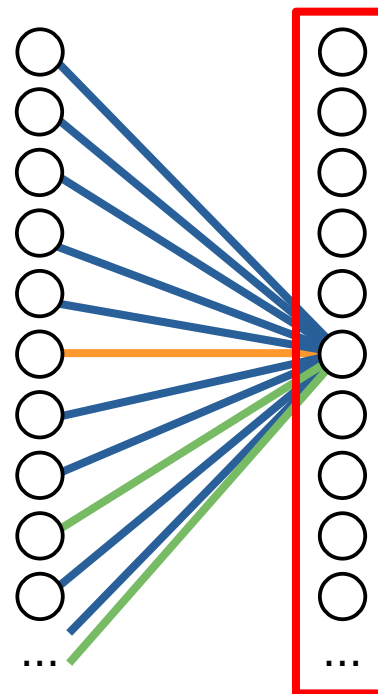
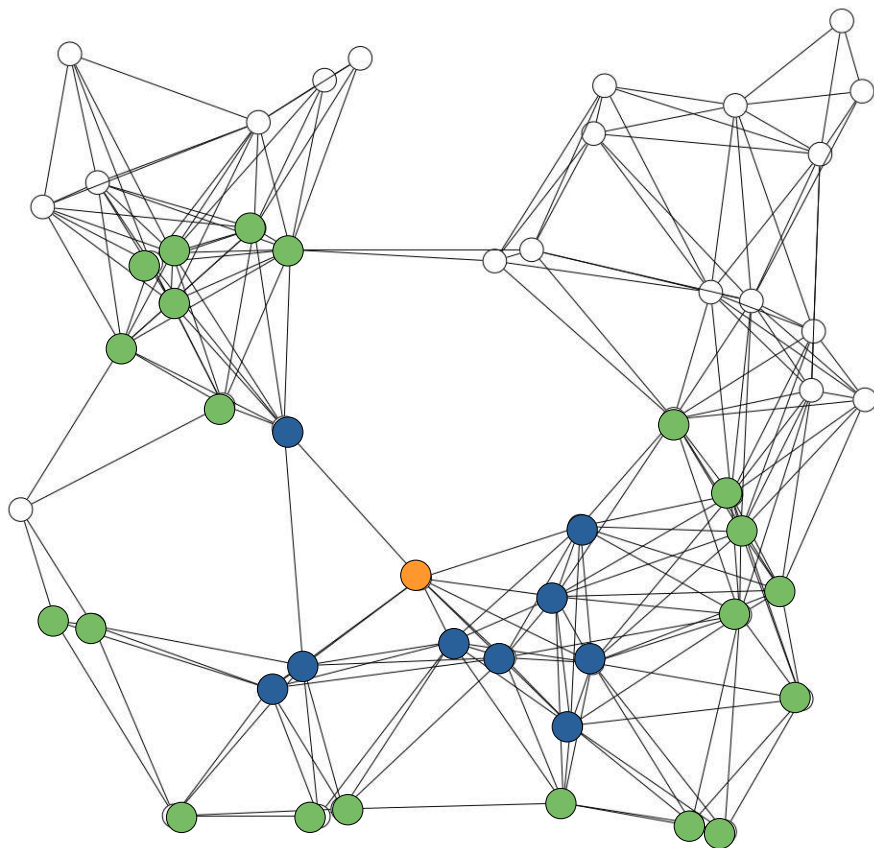


Graph convolutional networks

└ Graph convolutional layers with Chebyshev polynomials

<https://tel.archives-ouvertes.fr/tel-01708824>

$K = 2$

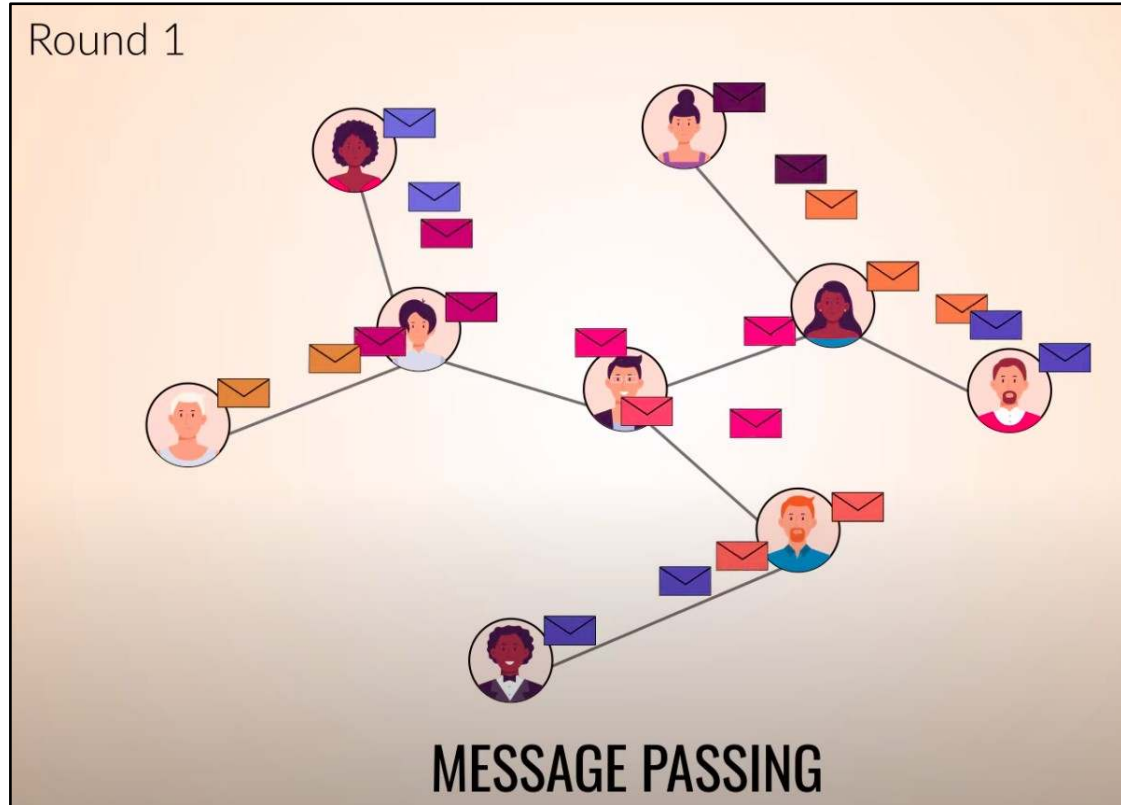


Do
again
for
any
center

Graph convolutional networks

└ The adopted version from the literature

<https://www.youtube.com/watch?v=GXhBEj1ZtE8>



Graph convolutional networks

└ The adopted version from the literature

<https://www.youtube.com/watch?v=GxhBEj1ZtE8>

<https://arxiv.org/pdf/1609.02907.pdf>

Round 1

In this section, we provide theoretical motivation for a specific graph-based neural network model $f(X, A)$ that we will use in the rest of this paper. We consider a multi-layer Graph Convolutional Network (GCN) with the following layer-wise propagation rule:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\right). \quad (2)$$

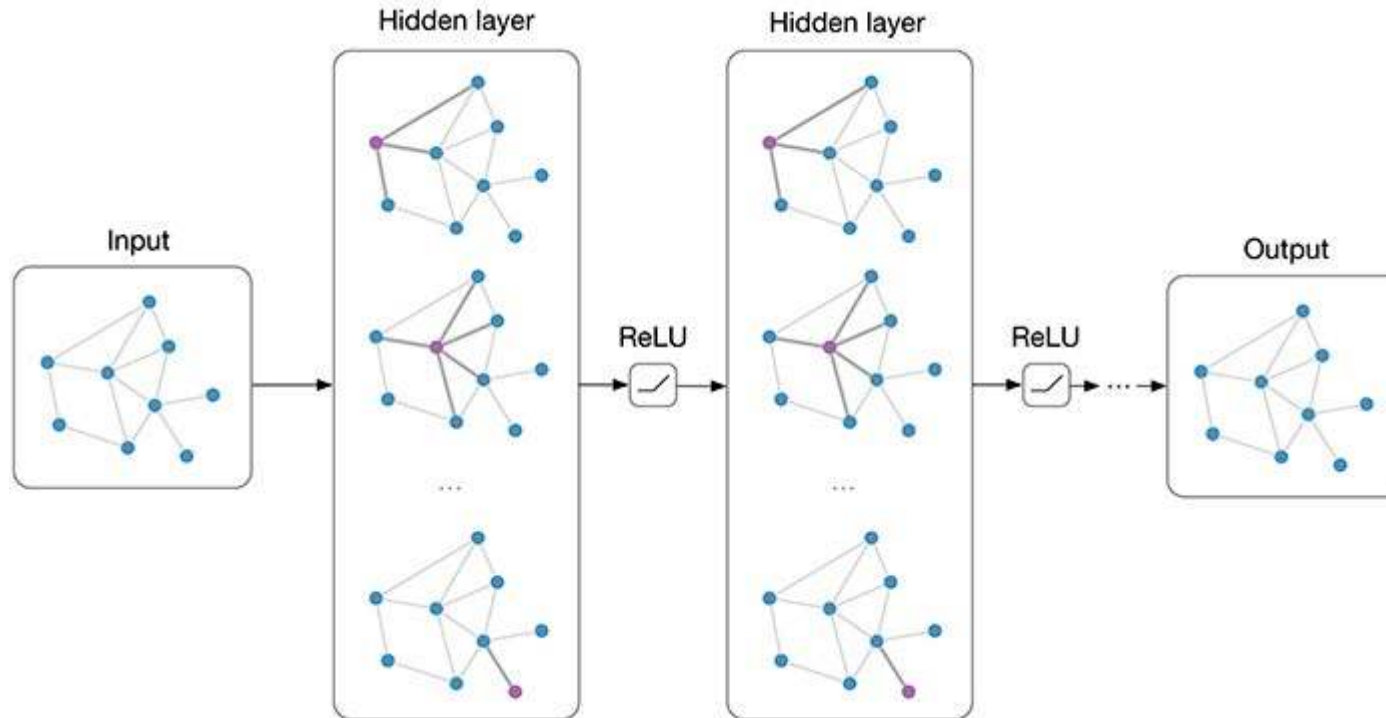
Here, $\tilde{A} = A + I_N$ is the adjacency matrix of the undirected graph $\mathcal{G}$ with added self-connections. I_N is the identity matrix, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ and $W^{(l)}$ is a layer-specific trainable weight matrix. $\sigma(\cdot)$ denotes an activation function, such as the $\text{ReLU}(\cdot) = \max(0, \cdot)$. $H^{(l)} \in \mathbb{R}^{N \times D}$ is the matrix of activations in the l^{th} layer; $H^{(0)} = X$. In the following, we show that the form of this propagation rule can be motivated¹ via a first-order approximation of localized spectral filters on graphs (Hammond et al., 2011; Defferrard et al., 2016).

MESSAGE PASSING

Graph convolutional networks

└ Assembling graph convolution layers

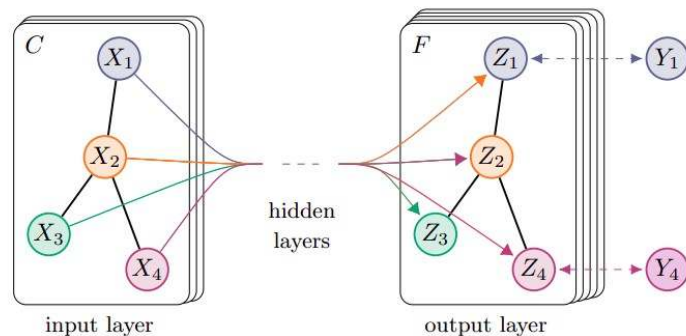
<http://tkipf.github.io/graph-convolutional-networks/>



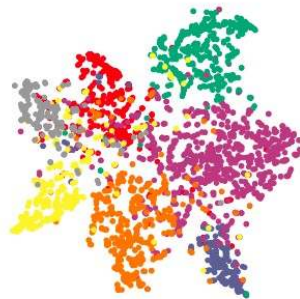
Graph convolutional networks

Graph convolutional networks

<https://arxiv.org/pdf/1609.02907.pdf>



(a) Graph Convolutional Network



(b) Hidden layer activations

Figure 1: *Left*: Schematic depiction of multi-layer Graph Convolutional Network (GCN) for semi-supervised learning with C input channels and F feature maps in the output layer. The graph structure (edges shown as black lines) is shared over layers, labels are denoted by Y_i . *Right*: t-SNE (Maaten & Hinton, 2008) visualization of hidden layer activations of a two-layer GCN trained on the Cora dataset (Sen et al., 2008) using 5% of labels. Colors denote document class.

Method	Citeseer	Cora	Pubmed	NELL
ManiReg [3]	60.1	59.5	70.7	21.8
SemiEmb [28]	59.6	59.0	71.1	26.7
LP [32]	45.3	68.0	63.0	26.5
DeepWalk [22]	43.2	67.2	65.3	58.1
ICA [18]	69.1	75.1	73.9	23.1
Planetoid* [29]	64.7 (26s)	75.7 (13s)	77.2 (25s)	61.9 (185s)
GCN (this paper)	70.3 (7s)	81.5 (4s)	79.0 (38s)	66.0 (48s)
GCN (rand. splits)	67.9 ± 0.5	80.1 ± 0.5	78.9 ± 0.7	58.4 ± 1.7

Graph convolutional networks

└ Perspectives

<https://arxiv.org/pdf/1901.00596.pdf>

Model depth The success of deep learning lies in deep neural architectures [174]. However, Li et al. show that the performance of a ConvGNN drops dramatically with an increase in the number of graph convolutional layers [53]. As graph convolutions push representations of adjacent nodes closer to each other, in theory, with an infinite number of graph convolutional layers, all nodes' representations will converge to a single point [53]. This raises the question of whether going deep is still a good strategy for learning graph data.

Scalability trade-off The scalability of GNNs is gained at the price of corrupting graph completeness. Whether using sampling or clustering, a model will lose part of the graph information. By sampling, a node may miss its influential neighbors. By clustering, a graph may be deprived of a distinct structural pattern. How to trade-off algorithm scalability and graph integrity could be a future research direction.

Heterogeneity The majority of current GNNs assume homogeneous graphs. It is difficult to directly apply current GNNs to heterogeneous graphs, which may contain different types of nodes and edges, or different forms of node and edge inputs, such as images and text. Therefore, new methods should be developed to handle heterogeneous graphs.

Dynamicity Graphs are in nature dynamic in a way that nodes or edges may appear or disappear, and that node/edge inputs may change time by time. New graph convolutions are needed to adapt to the dynamicity of graphs. Although the dynamicity of graphs can be partly addressed by STGNNs, few of them consider how to perform graph convolutions in the case of dynamic spatial relations.



OUTLINE

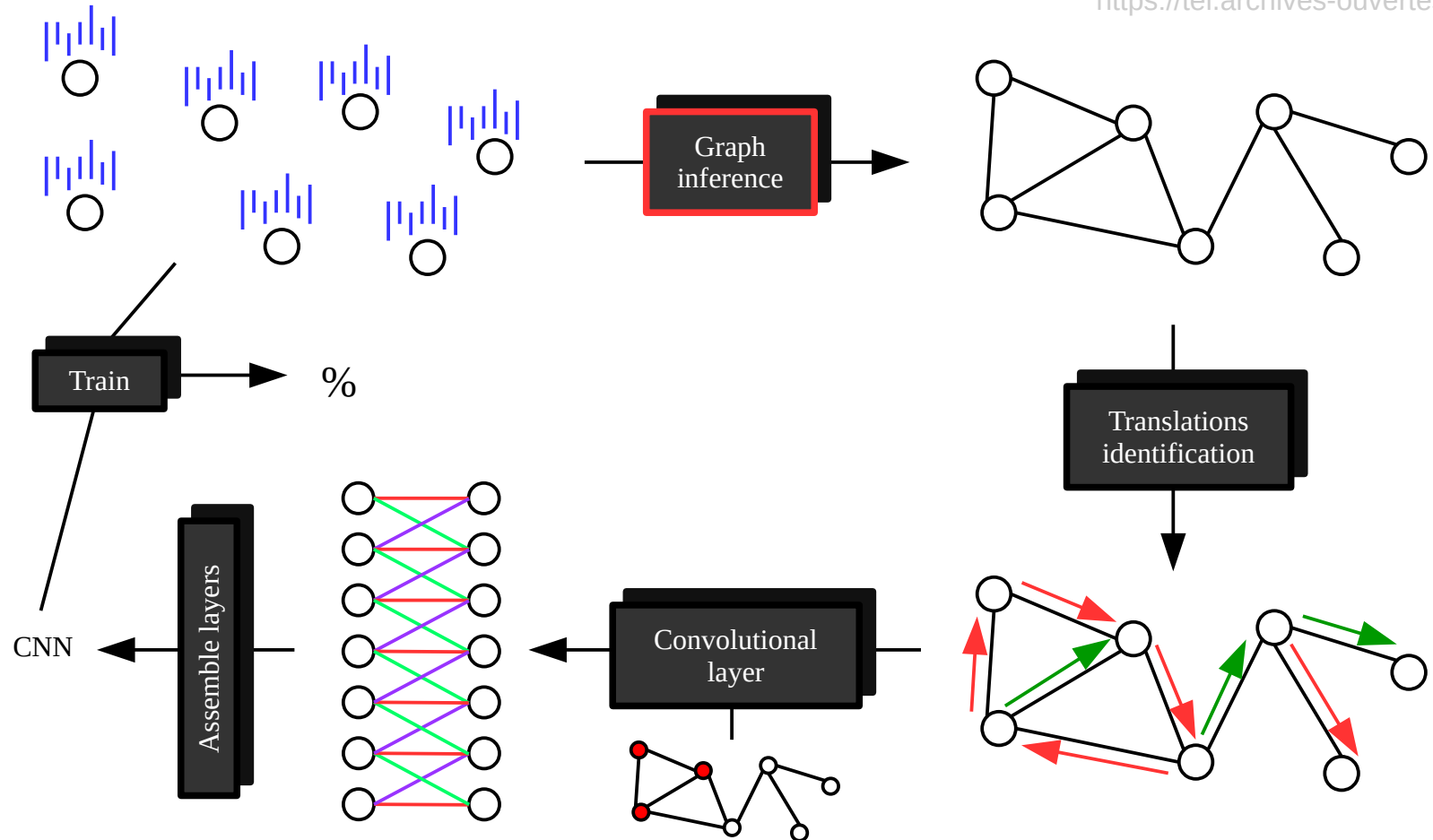
- ▶ Signal processing
- ▶ Graph theory
- ▶ Graph signal processing
- ▶ Focus :
Graph convolutional networks
- ▶ A lot of other things :D



Graph signal processing is still young

└ What happens if we don't have a graph?

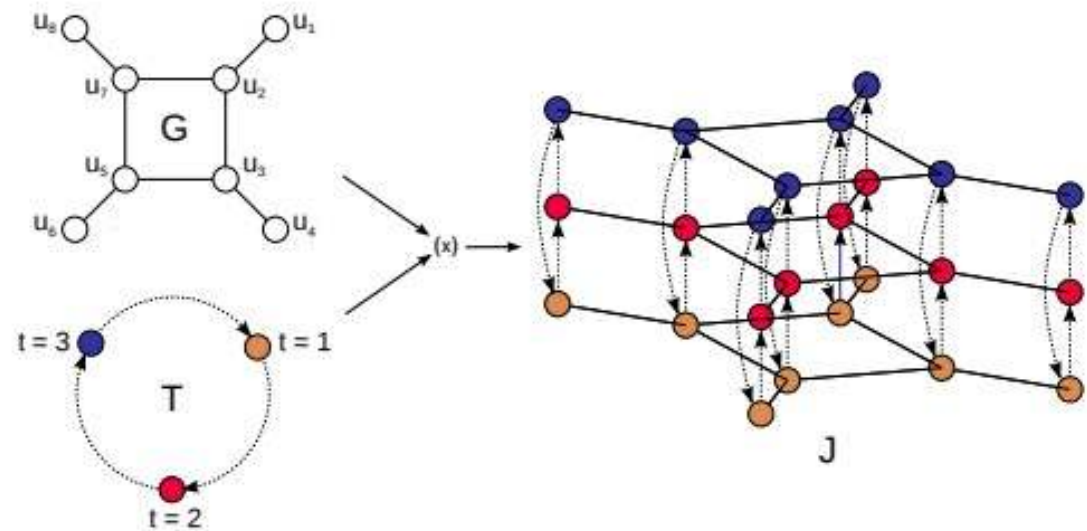
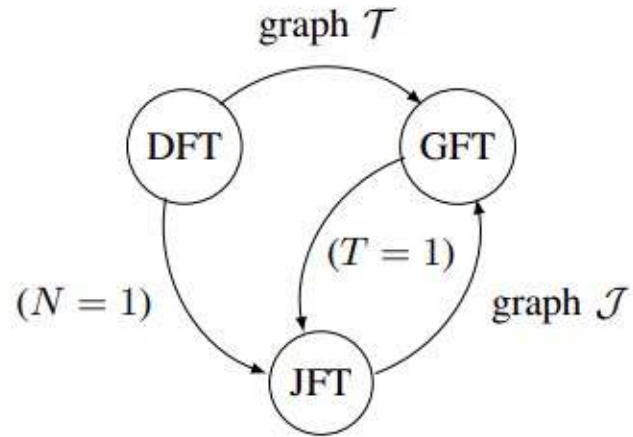
<https://tel.archives-ouvertes.fr/tel-01708824>



Graph signal processing is still young

└ What about spatio-temporal data?

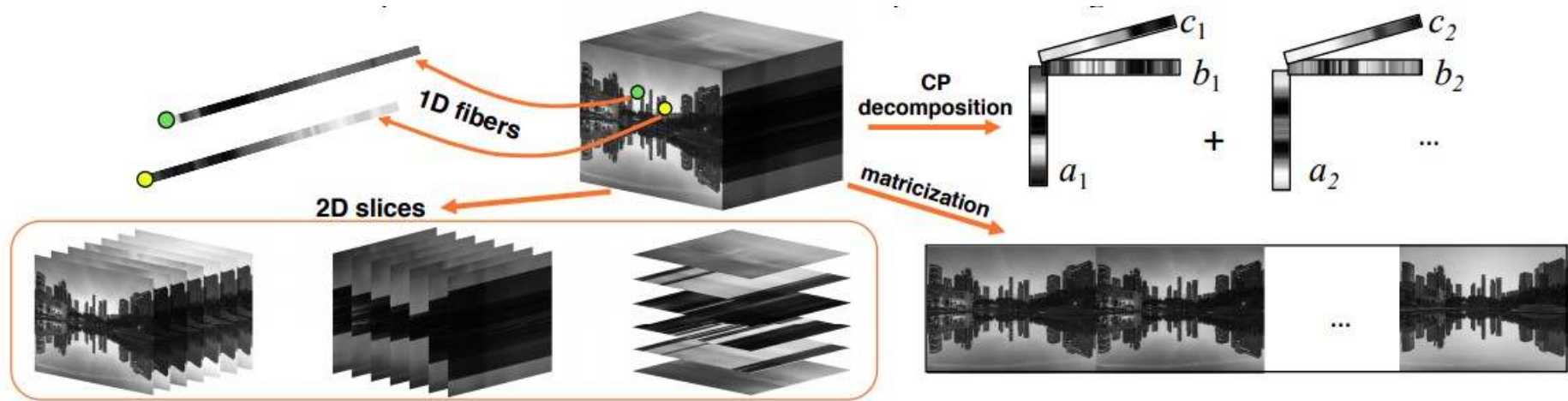
<https://arxiv.org/pdf/1602.04434.pdf>



Graph signal processing is still young

└ What about spatio-temporal-others data?

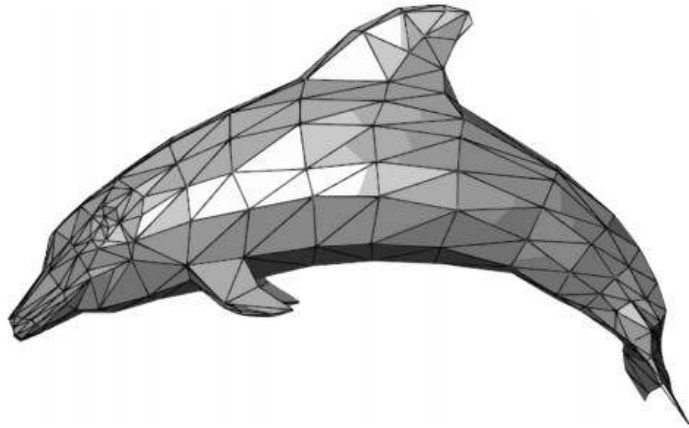
<https://arxiv.org/pdf/2007.00041.pdf>



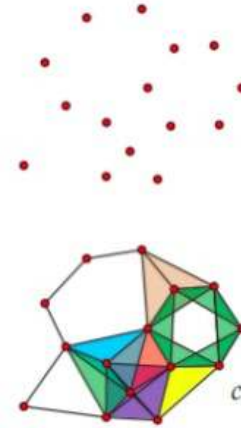
Graph signal processing is still young

└ Why stopping at graphs when one can abstract to simplicial complexes?

<https://arxiv.org/pdf/2004.02392.pdf>



(a)



(b)

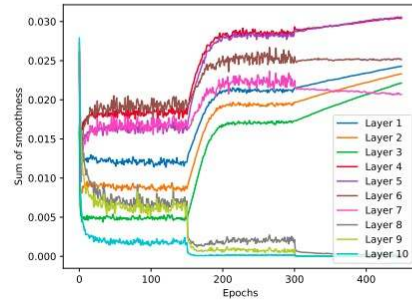
Fig. 1. (a) Approximation of a 2D surface with a 2-complex from Wikimedia Commons. (b) Model of a point cloud (top), together with a simplicial complex (bottom).

Graph signal processing is still young

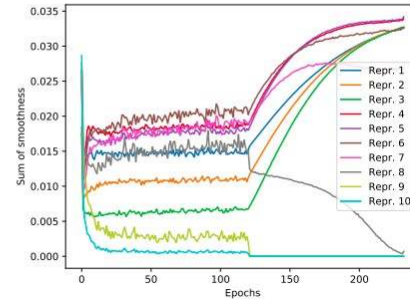
└ Can GSP help understanding neural networks?

<https://arxiv.org/pdf/2012.07439.pdf>

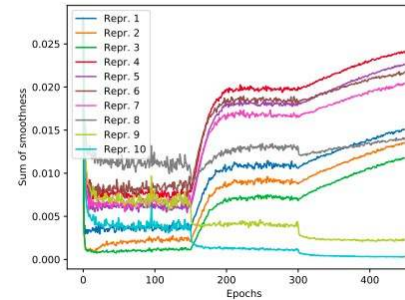
Reference



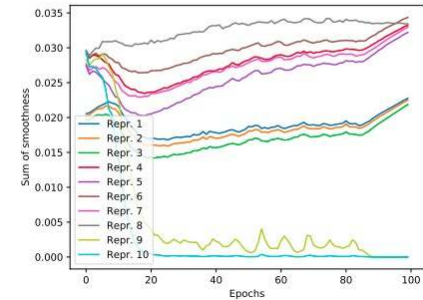
Slight overfit



Underfit



Extreme overfit



Graph Signal Processing

